机器学习

线性回归

多特征线性回归

在多特征线性回归中，假设有 n 个特征，输入数据为 $\mathbf{x} \in \mathbb{R}^{m \times n}$（$m$ 个样本，n 个特征），目标值为 $\mathbf{y} \in \mathbb{R}^m$，模型参数为 $\mathbf{w} \in \mathbb{R}^n$ 和偏置 $b \in \mathbb{R}$。

---

1. 线性回归函数

模型的预测值为：

$$\hat{\mathbf{y}} = \mathbf{x}\mathbf{w} + b$$

其中：

- $\hat{\mathbf{y}} \in \mathbb{R}^m$ 是预测值向量。
- $\mathbf{x} \in \mathbb{R}^{m \times n}$ 是输入特征矩阵。
- $\mathbf{w} \in \mathbb{R}^n$ 是权重向量。
- $b \in \mathbb{R}$ 是偏置。

---

2. 损失函数（均方误差，MSE）

损失函数衡量预测值与真实值之间的误差，通常使用均方误差（Mean Squared Error, MSE）：

$$L(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2$$

写成矩阵形式：

$$L(\mathbf{w}, b) = \frac{1}{2m} \|\hat{\mathbf{y}} - \mathbf{y}\|^2 = \frac{1}{2m} (\mathbf{x}\mathbf{w} + b - \mathbf{y})^\top (\mathbf{x}\mathbf{w} + b - \mathbf{y})$$

---

3. 损失函数对 $\mathbf{w}$ 求导

为了优化模型，需要对损失函数关于 $\mathbf{w}$ 求导。步骤如下：

损失函数展开：

$$L(\mathbf{w}, b) = \frac{1}{2m} (\mathbf{x}\mathbf{w} + b - \mathbf{y})^\top (\mathbf{x}\mathbf{w} + b - \mathbf{y})$$

对 $\mathbf{w}$ 求导：

$$\frac{\partial L}{\partial \mathbf{w}} = \frac{1}{m} \mathbf{x}^\top (\mathbf{x}\mathbf{w} + b - \mathbf{y})$$

对 b 求导：

$$\frac{\partial L}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i)$$

---

4. 梯度更新规则

使用梯度下降法更新参数：

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial L}{\partial \mathbf{w}}$$

$$b \leftarrow b - \alpha \frac{\partial L}{\partial b}$$

其中 α 是学习率。

概率角度看回归

从概率的角度来看回归问题，我们假设输出变量 y 在给定输入变量 x 和参数 w 的条件下服从正态分布：

$$p(y|x;w) = \mathcal{N}(y;xw,\sigma^2)$$

我们取这个分布的均值作为预测值：

$$\hat{y} = xw$$

我们的目标是最大化对数似然函数：

$$\max_w \log p(y|x;w)$$

这等价于最小化平方误差：

$$\min_w (y-xw)^2$$

对于 N 个独立同分布的样本，联合条件概率密度函数为：

$$p(y^{(1)},\cdots,y^{(N)}|x^{(1)},\cdots,x^{(N)};w) = \prod_{i=1}^{N} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y^{(i)}-x^{(i)}w)^2\right)$$

对数似然函数为：

$$\log p(y|X;w) = -\frac{N}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{N}(y^{(i)}-x^{(i)}w)^2$$

最大化对数似然等价于最小化平方和：

$$\min_{w} \sum_{i=1}^{N} (y^{(i)} - x^{(i)} w)^2$$

线性分类器

多分类器

1. 模型预测函数

给定输入特征矩阵 $\mathbf{X}$ 和权重向量 $\mathbf{W}$，预测值通过 softmax 函数计算。

$$\hat{\mathbf{Y}} = \text{softmax}(\mathbf{X}\mathbf{W})$$

其中，$\mathbf{X}$ 是 $n \times 1$ 的矩阵，$\mathbf{W}$ 是 $1 \times K$ 的矩阵，$\hat{\mathbf{Y}}$ 是 $n \times K$ 的预测概率矩阵。

$$\hat{y}_{ik} = \frac{\exp(x_i w_k)}{\sum_{j=1}^{K} \exp(x_i w_j)}$$

其中，x_i 是第 i 个样本的特征值，w_k 是第 k 个类别的权重，$\hat{y}_{ik}$ 是第 i 个样本属于第 k 个类别的预测概率。

2. 损失函数（交叉熵损失）

损失函数定义为真实标签与预测概率之间的交叉熵。

$$L = -\frac{1}{n} \cdot \text{Tr}\left(\mathbf{Y}^T \log \hat{\mathbf{Y}}\right)$$

其中，$\mathbf{Y}$ 是 $n \times K$ 的独热编码矩阵，$\hat{\mathbf{Y}}$ 是预测概率矩阵，$\log$ 是元素-wise 对数运算，Tr 表示矩阵的迹。

$$L = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} y_{ik} \log \hat{y}_{ik}$$

3. 损失函数对 $\mathbf{W}$ 求导

损失函数关于权重向量 $\mathbf{W}$ 的梯度。

$$\frac{\partial L}{\partial \mathbf{W}} = \frac{1}{n}\mathbf{X}^T(\hat{\mathbf{Y}} - \mathbf{Y})$$

其中，$\mathbf{X}^T$ 是 $1 \times n$ 的矩阵，$(\hat{\mathbf{Y}} - \mathbf{Y})$ 是 $n \times K$ 的矩阵。

$$\frac{\partial L}{\partial w_k} = \frac{1}{n}\sum_{i=1}^{n} x_i(\hat{y}_{ik} - y_{ik})$$

4. 梯度更新规则

使用梯度下降法更新权重向量 $\mathbf{W}$。

$$\mathbf{W} \leftarrow \mathbf{W} - \alpha \cdot \frac{\partial L}{\partial \mathbf{W}}$$

$$w_k \leftarrow w_k - \alpha \cdot \frac{\partial L}{\partial w_k}$$

其中，α 是学习率。

概率角度看分类

假设输出变量 y 是一个独热编码的向量，表示属于哪个类别。例如，对于 K 个类别，y 是一个 K 维向量，只有一个元素是 1，其他是 0。

$\boldsymbol{x} \in \mathbb{R}^D$ 是输入特征向量，表示样本的特征。

W 是 $D \times K$ 的权重矩阵，每一列对应一个类别的权重向量 W_k。

$\pi_k = \mathrm{softmax}(Wx)_k$，即：

$$\pi_k = \frac{\exp(x^T W_k)}{\sum_{k'=1}^{K} \exp(x^T W_{k'})}$$

假设 $p(y|x;W)$ 是一个多分类分布，使用 softmax 函数来表示类别概率：

$$p(y|x;W)=\prod_{k=1}^{K}\pi_k^{y_k}$$

为了简化计算，通常对似然函数取对数，得到对数似然函数：

$$\log p(y|x;W)=\sum_{k=1}^{K}y_k\log\pi_k$$

最大化对数似然等价于最小化负对数似然，即最小化交叉熵损失：

$$-\log p(y|x;W)=-\sum_{k=1}^{K}y_k\log\pi_k$$

非线性模型、过拟合与正则化

非线性模型

用基函数去非线性化原来的线性模型。

原来的线性回归可写为$f(x)=w_0+w_1x=[1,x]w$。
如果采用更复杂的基函数，则可以写为$f(x)=w_0+w_1x+w_2x^2+w_3x^3=\boldsymbol{\phi}(x)\boldsymbol{w}$

一般地，基函数模型可以写为：$f(x)=\phi(x)\boldsymbol{w}=[\phi_1(\boldsymbol{x}),\phi_2(\boldsymbol{x}),\cdots,\phi_n(\boldsymbol{x})]\boldsymbol{w}$
基函数模型对x不一定是线性的，但是对参数w仍然是线性的。

经过非线性变换后，损失函数可以写为：

$$L(\boldsymbol{w})=\frac{1}{N}\|\phi(\boldsymbol{X})\boldsymbol{w}-\boldsymbol{y}\|^2$$

其中

$$\phi(X)=\begin{bmatrix}\phi(\boldsymbol{x}^{(1)})\\ \vdots \\ \boldsymbol{\phi}(\boldsymbol{x}^{(N)})\end{bmatrix}$$

最小化损失函数(求驻点=0)，最优参数为：

$$w^*=(\phi^T\phi)^{-1}\phi^T\boldsymbol{y}$$

这就是将原来线性回归中的X替换为$\phi=\phi(X)$的结果。

分类问题中同理。交叉熵变为：

$$L(\boldsymbol{W})=-\frac{1}{N}\sum_{l=1}^{N}\sum_{k=1}^{K}y_k^{(l)}\log[\mathrm{softmax}_k(\boldsymbol{\phi}(\boldsymbol{x}^{(l)})\boldsymbol{W})]$$

梯度变为：

$$\frac{\partial L(\boldsymbol{W})}{\partial \boldsymbol{w}_j}=\frac{1}{N}\sum_{l=1}^{N}\left(\mathrm{softmax}_j(\boldsymbol{\phi}^{(l)}\boldsymbol{W})-\boldsymbol{y}^{(l)}\right)\boldsymbol{\phi}^{(l)T}$$

同样是将原来的$x^{(l)}$替换为$\phi^{(l)}=\phi(x^{(l)})$的结果

过拟合

基函数的维数越高，就能更好的拟合训练数据。但是相对的，在测试集上的表现就会越来越差。

模型在**模式数据上**的表现称为这个模型的**泛化能力**。

> 模式数据是模型在训练过程中未见过的、但与训练数据具有相似分布的数据

定义

当验证集上的错误率随着复杂度提升而下降时，称模型此时是**欠拟合**的；
当验证集上的错误率随复杂度提升而提升时，称模型此时是**过拟合**的。

K-fold 交叉验证

训练数据通常是比较少的，如果还划分一部分用于验证集，则没有足够的数据样本用于训练，此时可以采取K-fold 交叉验证。

也即把训练集分为 K 份，第 i 轮训练用第 i 份作为验证集，其他 K-1 份用于训练。

信息准则 (Information Criteria)

用于评估模型的复杂度。

Akaike Information Criteria (AIC,赤池信息准则) :

$$\mathrm{AIC} = 2M - 2\log(L)$$

其中M是模型的参数量，L是该模型下的最大似然。一般，AIC越小，模型越好。

Bayesian Information Criteria (BIC,贝叶斯信息专责我)

$$\mathrm{BIC} = M\log N - 2\log(L)$$

其中N是训练数据数量。

这两个标准一般只能用于概率模型中，因为需要用到对数似然。

> 似然函数表示在给定模型参数的情况下，观测数据出现的概率。假设我们有一个概率模型 $P(X|\theta)$，其中：
>
> - X 是观测数据，
> - θ 是模型的参数。
>
> 似然函数定义为：
>
> $$L(\theta) = P(X|\theta)$$
>
> 对于独立同分布的数据，似然函数可以写成：
>
> $$L(\theta) = \prod_{i=1}^{n} P(x_i|\theta)$$

> 其中 x_i 是第 i 个观测数据点。
> 由于似然函数通常是连乘的形式，直接计算可能会遇到数值下溢的问题（即结果太小，计算机无法表示）。因此，通常对似然函数取对数，将连乘转化为累加：
>
> $$\ln L(\theta) = \sum_{i=1}^{n} \ln P(x_i|\theta)$$
>
> 对数似然函数不仅简化了计算，还保持了似然函数的单调性，因此最大化对数似然等价于最大化似然函数。

正则化

过拟合一般是由高阶项对应的参数过大导致的。引入正则项来平衡这些影响

> 正则化是机器学习中一种用于**防止模型过拟合**的技术

L_2正则化:

$$\tilde{L}(\boldsymbol{w}) = L(\boldsymbol{w}) + \lambda \|\boldsymbol{w}\|_2^2$$

其中λ是超参数，用于控制正则项的影响。它将参数趋向于 0。
正则化后得到的模型曲线更平滑。

L_1正则化：

$$\tilde{L}(\boldsymbol{w}) = L(\boldsymbol{w}) + \lambda \|\boldsymbol{w}\|_1$$

这种正则化可以让参数变为 0 (而不仅仅是趋向 0),所以经常趋向于得到松弛解。

> 正则化通过添加惩罚项间接限制了模型参数的范围，类似于凸优化中的可行域限制

支持向量机

线性分类器的决策边界

之前的线性分类器的基础上加上一个偏置项，交叉熵损失函数变为：

$$L(\boldsymbol{w}, b) = -y \log(\sigma(\boldsymbol{w}^T \boldsymbol{x} + b)) - (1 - y) \log(1 - \sigma(\boldsymbol{w}^T \boldsymbol{x} + b))$$

损失函数最小值为 0,记使其为 0 的最优的w^*, b^*,则$L(\boldsymbol{w}, b) = 0$表示的超平面为:
$\{w | w^{*T} x + b^* = 0\}$

这个超平面:

- 垂直于w^*
- 到原点的距离是$-b^* / ||w^*||$

线性最大间隔分类器

用点到直线(决策超平面)的距离表示间隔

“间隔”取决于距离超平面最近的那些数据点 $\mathrm{Margin} = \min_l \frac{y^{(l)}(\boldsymbol{w}^T \boldsymbol{x}^{(l)} + b)}{\|w\|}$

原问题

为了将其转化为凸优化问题，最小化 $\frac{1}{2}\|w\|^2$，并加入约束条件 $y^{(l)}(w^T x^{(l)} + b) \geq 1$
:

$$\begin{aligned} &\min_{w,b} \quad \frac{1}{2}\|w\|^2 \\ &\text{subject to} \quad y^{(l)}(w^T x^{(l)} + b) \geq 1, \quad \forall l = 1, 2, \ldots, n \end{aligned}$$

> 实际上是认为规定分子为1,然后最大化剩余的,相当于最小化倒数
> 约束条件可以由决策平面上方的y=1,下方的y=-1推出

预测分类结果$\hat{y} = \mathrm{sign}(\boldsymbol{w}^{*T} \boldsymbol{x} + b^*)$

对偶问题

> 引入拉格朗日乘数法，构造拉格朗日函数:
>
> $$\mathcal{L}(w, b, \alpha) = \frac{1}{2}\|w\|^2 - \sum_{l=1}^{n} \alpha_l [y^{(l)}(w^T x^{(l)} + b) - 1]$$
>
> 对 w 和 b 求偏导并设偏导为零:

$$\frac{\partial \mathcal{L}}{\partial w} = w - \sum_{l=1}^{n} \alpha_l y^{(l)} x^{(l)} = 0 \quad \Rightarrow \quad w = \sum_{l=1}^{n} \alpha_l y^{(l)} x^{(l)}$$

$$\frac{\partial \mathcal{L}}{\partial b} = -\sum_{l=1}^{n} \alpha_l y^{(l)} = 0 \quad \Rightarrow \quad \sum_{l=1}^{n} \alpha_l y^{(l)} = 0$$

将 w 的表达式代入拉格朗日函数：

$$\mathcal{L} = \sum_{l=1}^{n} \alpha_l - \frac{1}{2} \sum_{l=1}^{n} \sum_{k=1}^{n} \alpha_l \alpha_k y^{(l)} y^{(k)} x^{(l)} \cdot x^{(k)}$$

对偶问题为：

$$\begin{aligned} \max_{\alpha} \quad & \sum_{l=1}^{n} \alpha_l - \frac{1}{2} \sum_{l=1}^{n} \sum_{k=1}^{n} \alpha_l \alpha_k y^{(l)} y^{(k)} x^{(l)} \cdot x^{(k)} \\ \text{subject to} \quad & \alpha_l \geq 0, \quad \forall l = 1, 2, \ldots, n \\ & \sum_{l=1}^{n} \alpha_l y^{(l)} = 0 \end{aligned}$$

预测分类结果$\hat{y} = \operatorname{sign}\left(\sum_{l=1}^{N} \alpha^{(l)} y^{(l)} \boldsymbol{x}^{(l)T} \boldsymbol{x} + b^*\right)$

软线性最大间隔分类器

如果数据不是线性可分的，就需要引入软间隔。

约束变为： $y^{(l)}(\boldsymbol{w}^T \boldsymbol{x}^{(l)} + b) \geqslant 1 - \xi^{(l)}$

其中$\xi^{(l)} \geqslant 0$是一个松弛变量，用于表示最多能容忍的误差。我们希望同时最小化间隔和误差，所以优化目标变为：

$$\min_{\boldsymbol{w},b,\boldsymbol{\xi}} \left(\frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{l=1}^{N} \xi^{(l)} \right)$$

其中C是惩罚因子，它表示了误差的权重。C取无穷时，就退化到上一个线性最大间隔分类器
同样可以利用拉格朗日乘子法求解。最终的分类器的形式仍然是：

$$\hat{y} = \text{sign}\left(\sum_{l=1}^{N} \alpha^{(l)} y^{(l)} \boldsymbol{x}^{(l)T} \boldsymbol{x} + b^* \right)$$

核技巧

上述的分类器仍然是线性的，引入基函数后，可以得到非线性的分类器：

$$\hat{y} = \text{sign}\left(\sum_{l=1}^{N} \alpha^{(l)} y^{(l)} \phi(\boldsymbol{x}^{(l)})^T \phi(\boldsymbol{x}) + b^* \right)$$

越高维的空间中，数据点之间的距离越大，也就越容易找到一个可行的"超平面”,所以我们希望变换后的特征空间$\phi(x)$尽可能高维。但是也不能过高，因为维度越高，计算量也就越大。

整个问题的难点在于计算$\phi(x^{(l)})^T \phi(x)$,这在高维空间中的计算量是非常大的。

> 核技巧无需将原本的低维向量转为高维再做内积,而是可以直接算出两个低维向量在高维的内积

核函数是一个包含两个变量的函数$K(x, x')$,可以表示为两个向量的内积：

$$K(\boldsymbol{x}, \boldsymbol{x}') = \phi(\boldsymbol{x})^T \phi(\boldsymbol{x}')$$

Mercer 定理

如果一个函数$K(x, x')$是对称正定的，也即

$$\iint K(\boldsymbol{x},\boldsymbol{x}')f(\boldsymbol{x})f(\boldsymbol{x}')d\boldsymbol{x}d\boldsymbol{x}' \geqslant 0, \forall f(\cdot) \in L^2$$

则一定存在一个特征空间$\phi(x)$,使得$K(\boldsymbol{x},\boldsymbol{x}') = \boldsymbol{\phi}(\boldsymbol{x})^T\boldsymbol{\phi}(\boldsymbol{x}')$

神经网络

定义

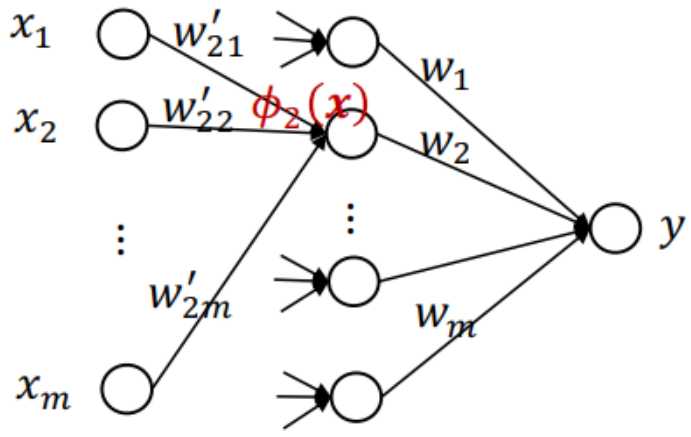
目前提到的非线性化模型的方法有两种

- 利用基函数ϕ(·)来进行特征变换
- 利用核技巧，可以理解为一个无限维的基函数变换

以上两种方法中，基函数（的参数）都是固定的，不能根据数据的特征进行调整。上述的两种方法可以用下图表示：



设想让“非线性化”过程中的参数是可学习的（可变的）：



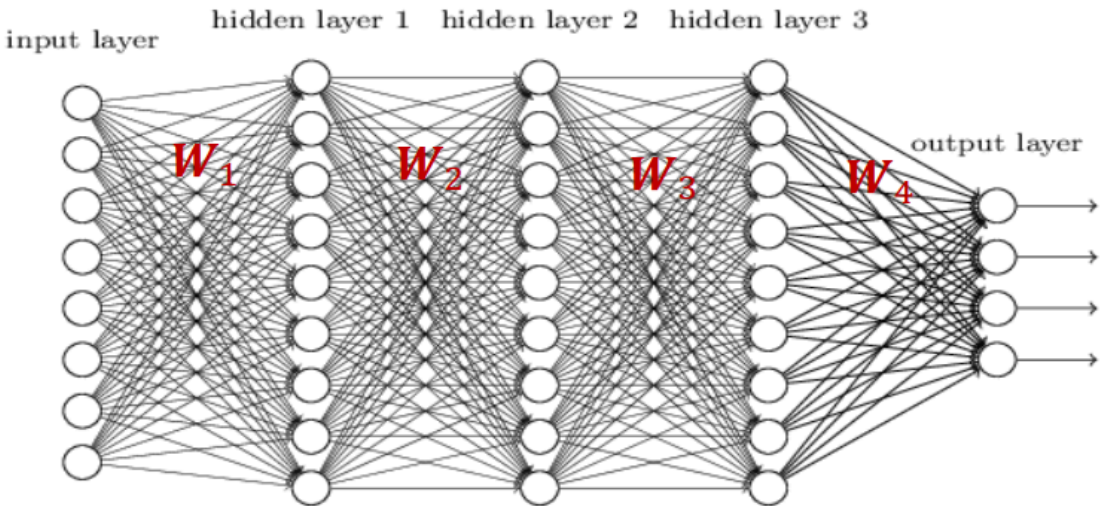
其中$w_21', w'_{22}, \ldots, w'_{2m}$都是可变的。

一般地，令$\phi_i(\boldsymbol{x}) = a(\sum_{l=1}^{m} w'_{il}x_l)$,其中$a(\cdot)$是**激活函数**。激活函数一定要有(而且不能是线性函数),否则最终y相对于x还是线性的。

于是上图的输出可以写为：

$$\hat{y} = \boldsymbol{w}_2^T a(\boldsymbol{W}_1 x)$$

在此基础上，不断增加堆叠的层数：



一个L层的神经网络可以写为：

$$\hat{y} = \boldsymbol{W}_L a(\cdots a(\boldsymbol{W}_2 a(\boldsymbol{W}_1 \boldsymbol{x})))$$

其中最外层到底是矩阵W_L还是一个向量w_L取决于需要怎样的输出。

- 如果是回归问题，输出是一个实数；
- 如果是分类问题，输出是一个向量(输出是一个向量)最后需要再套一个softmax函数。

给定训练集$\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}$

回归问题的损失函数可写为：

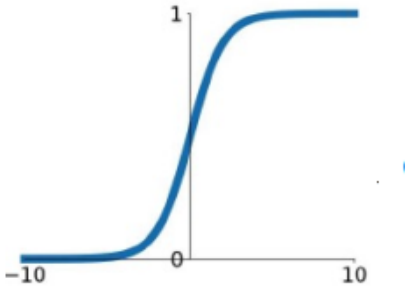
$$L = \frac{1}{N} \sum_{(\boldsymbol{x},y)\in\mathcal{D}} |y - \hat{y}|^2$$

多分类问题的损失函数可写为：

$$L = -\frac{1}{N} \sum_{(\boldsymbol{x},y)\in\mathcal{D}} \sum_{k=1}^{K} y_k \log \hat{y}_k$$

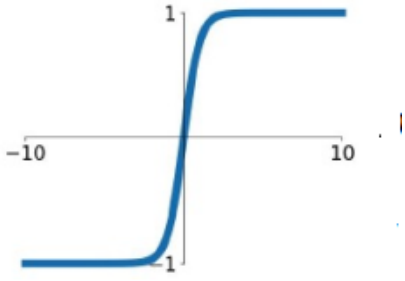
常见的激活函数

$\text{sigmoid}(x) = \frac{1}{1+e^{-x}}$



- 易有梯度消失现象
- 只输出正值
- 指数函数的计算代价高

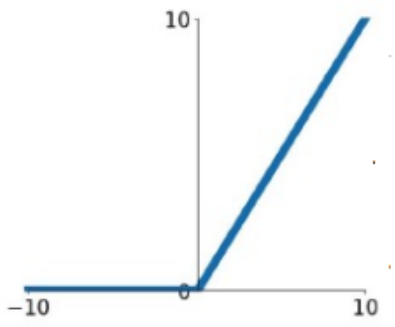
$\tanh(x) = \frac{1-e^{-2x}}{1+e^{-2x}}$



- 易有梯度消失现象

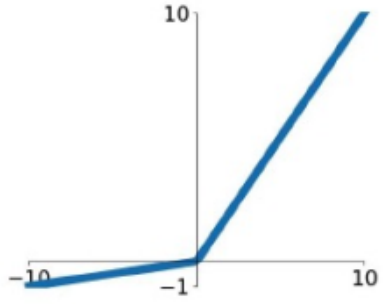
- 指数函数的计算代价高
- 同时输出正值和负值

$$\mathrm{ReLU}(x) = \max(0, x)$$



- 在x>0时没有梯度消失现象；x<0时梯度为 0
- 计算效率高
- 只输出正值

$$\text{Leaky Relu}(x) = \max(0.1x, x)$$

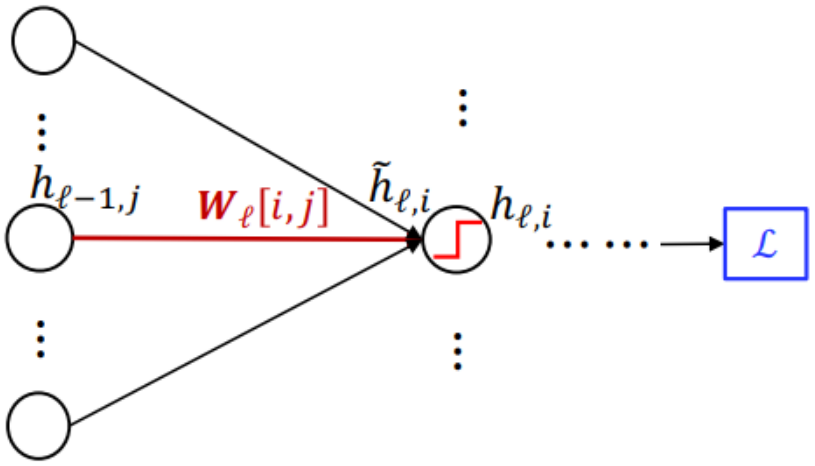


- 对所有的x都没有梯度消失线性
- 计算效率高

反向传播

考虑递归的方式来求每个参数的梯度。

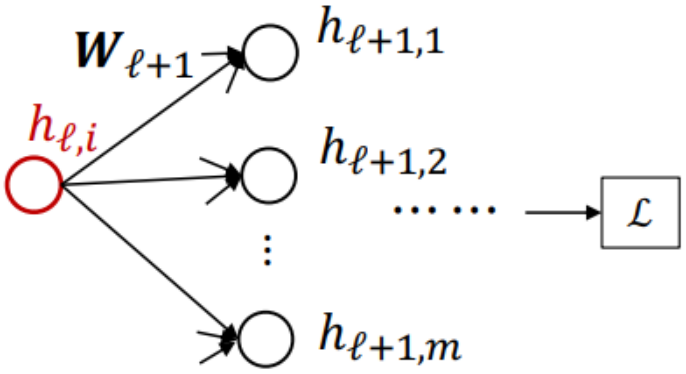
$$\tilde{h}_{l,i} = \sum_k W_\varphi[i,k] h_{l-1,k} \quad h_{l,i} = a\left(\sum_k W_\varphi[i,k] h_{l-1,k}\right)$$



这里$W_{l[i,j]}$是第l层的参数中，第l个隐藏层第i个节点的第j个参数。则有：

$$\begin{aligned}\frac{\partial L}{\partial W_{l[i,j]}} &= \frac{\partial L}{\partial h_{l,i}} \frac{\partial h_{l,i}}{\partial \tilde{h}_{l,i}} \frac{\partial \tilde{h}_{l,i}}{\partial W_{l[i,j]}} \\ &= \frac{\partial L}{\partial h_{l,i}} \cdot a'(\tilde{h}_{l,i}) \cdot h_{l-1,j}\end{aligned}$$

下面看怎么计算$\frac{\partial L}{\partial h_{l,i}}$



- 根据链式法则，$\frac{\partial L}{\partial h_{l,i}}$ 可以表示为：

$$\frac{\partial L}{\partial h_{l,i}} = \sum_{k=1}^{m} \frac{\partial L}{\partial h_{l+1,k}} \cdot \frac{\partial h_{l+1,k}}{\partial \tilde{h}_{l+1,k}} \cdot \frac{\partial \tilde{h}_{l+1,k}}{\partial h_{l,i}}$$

- 其中：

$$\frac{\partial h_{l+1,k}}{\partial \tilde{h}_{l+1,k}} = a'(\tilde{h}_{l+1,k})$$

$$\frac{\partial \tilde{h}_{l+1,k}}{\partial h_{l,i}} = W_{l+1[k,i]}$$

- 因此：

$$\frac{\partial L}{\partial h_{l,i}} = \sum_{k=1}^{n_{l+1}} \left(\frac{\partial L}{\partial h_{l+1,k}} \cdot a'(\tilde{h}_{l+1,k}) \cdot W_{l+1[k,i]} \right)$$

如果用向量和矩阵来表示，可以写成：

$$\frac{\partial L}{\partial \mathbf{h}_l} = \mathbf{W}_{l+1}^T \cdot \left(\frac{\partial L}{\partial \mathbf{h}_{l+1}} \odot a'(\tilde{\mathbf{h}}_{l+1}) \right)$$

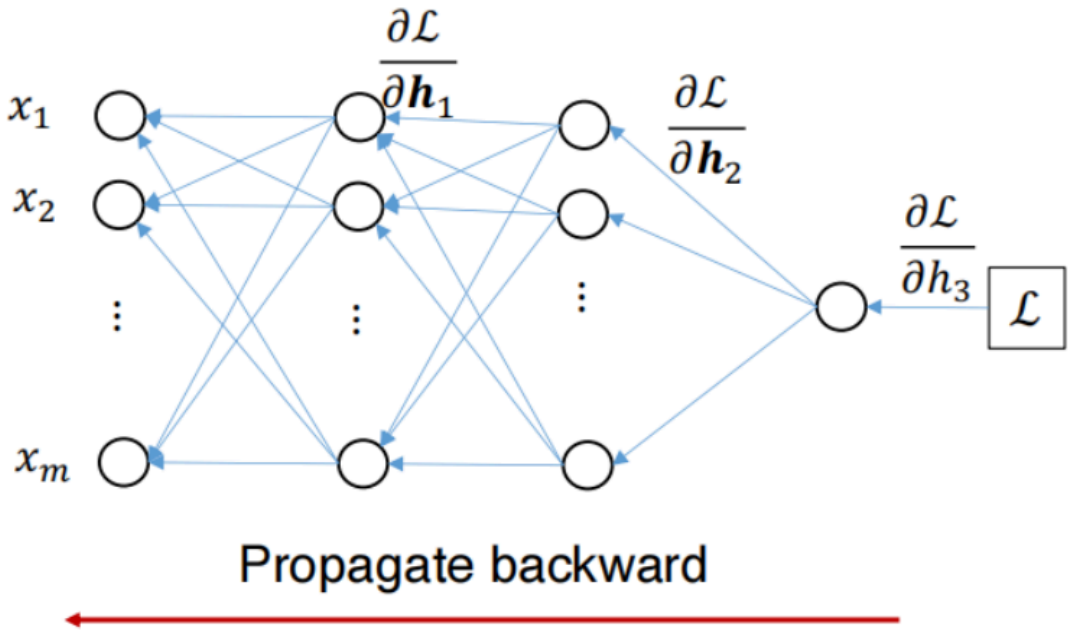
其中，$\mathbf{h}_l$ 是第 l 层的输出向量，$\mathbf{W}_{l+1}$ 是第 $l+1$ 层的权重矩阵，$\odot$ 表示元素-wise 乘法(相同位置的元素进行乘法运算)。

先正向传播算值,再反向传播算梯度

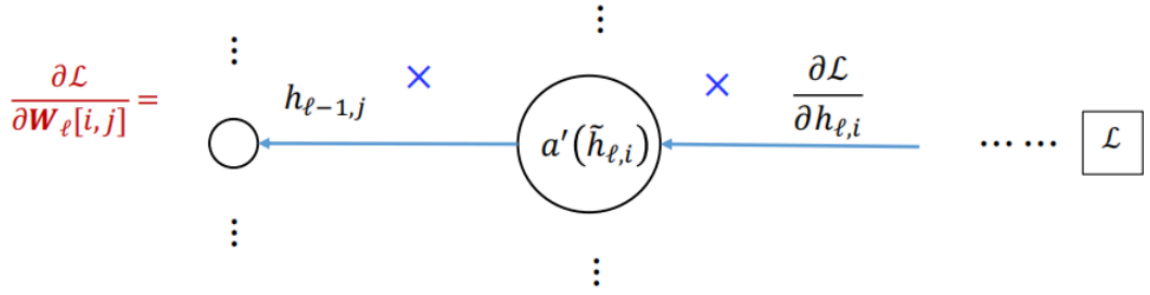
反向传播

1.计算 $\frac{\partial L}{\partial h_l}$

$$\frac{\partial L}{\partial h_l} = \begin{bmatrix} \frac{\partial L}{\partial h_{l,1}} \\ \frac{\partial L}{\partial h_{l,2}} \\ \vdots \\ \frac{\partial L}{\partial h_{l,n_l}} \end{bmatrix}$$



2.计算$\frac{\partial L}{\partial W_l}$



元素形式

$$\frac{\partial L}{\partial W_{l[i,j]}} = \frac{\partial L}{\partial h_{l,i}} \cdot a'(h_{l,i}) \cdot h_{l-1,j}$$

矩阵形式

$$\frac{\partial L}{\partial W_l} = \left(\frac{\partial L}{\partial h_l} \odot a'(h_l) \right) \cdot h_{l-1}^T$$

各种类型的神经网络

CNN

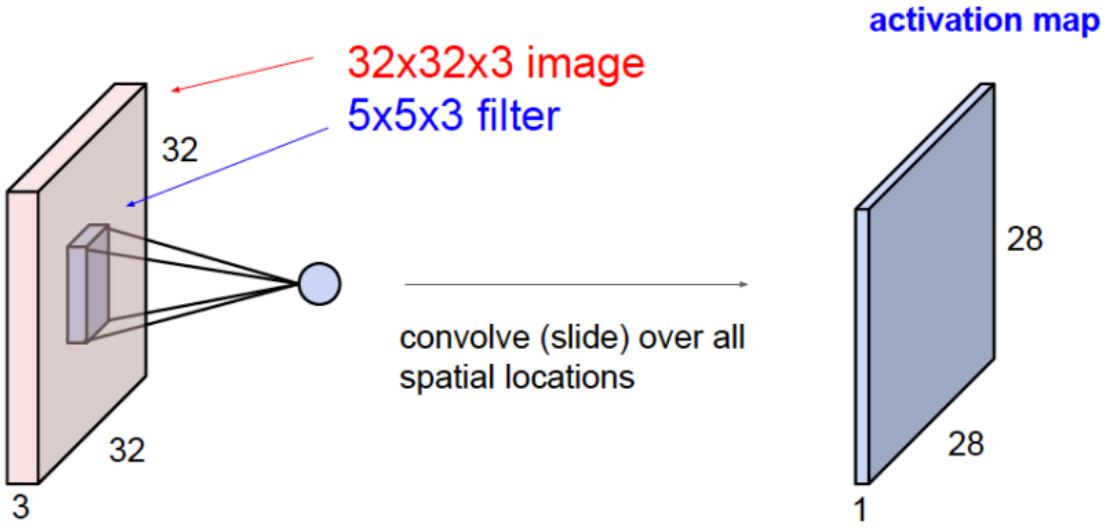
Convolutional Neural Networks，卷积神经网络

普通全连接网络（多层感知机，MLP）有以下缺点：

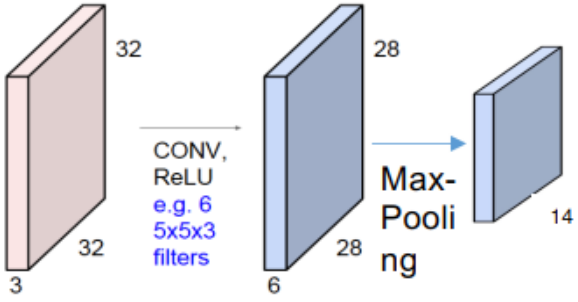
- 参数数量巨大，容易**过拟合**
- 没有考虑数据数据中**潜在的结构信息**

CNN 就是一种可以利用图片/视觉信息中潜在结构的网络。

二维图像可以理解为二维矩阵，用过滤器（Filter）对原始图像做卷积，“提取”出特征信息：



CNN 中，基本的结构是卷积层+ReLU+池化层：



这种结构可以多次堆叠。其特点是：

- 减少维度

- 使模型非线性化
- 保留重点特征

卷积层

计算输出层（池化层的输出尺寸也是）大小

N输出大小，W输入大小，P填充大小，S步长 F 卷积核大小

$$N = \frac{(W-F+2P)}{S} + 1$$

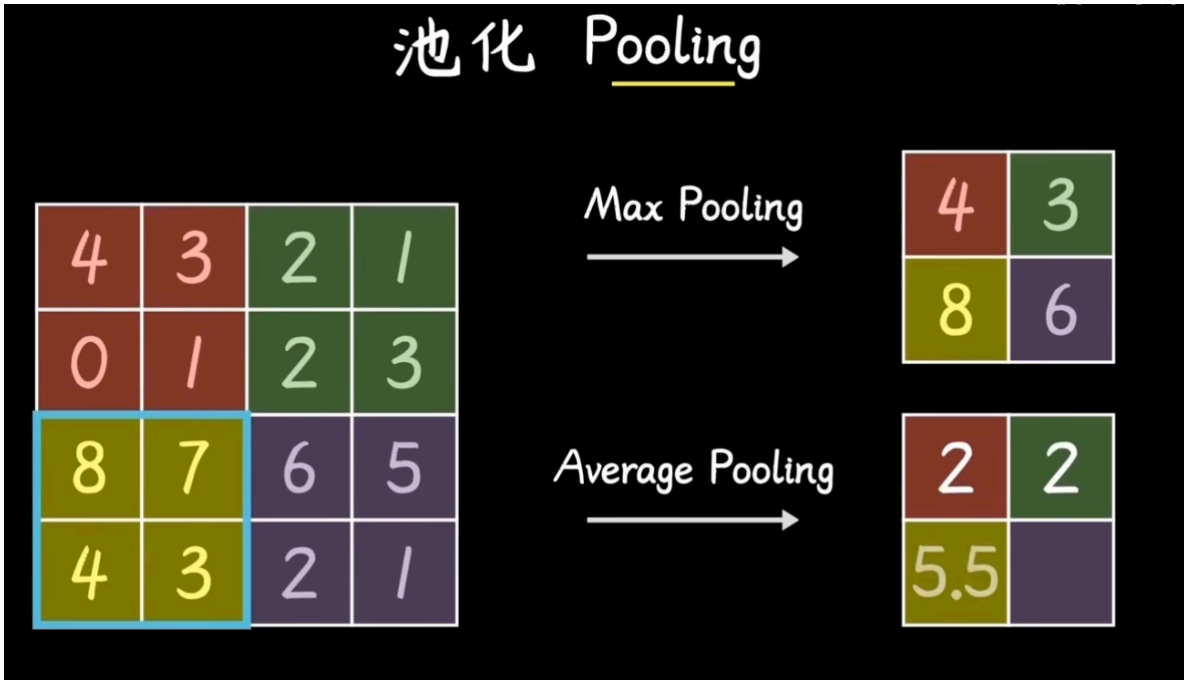
输出通道数 = 卷积核数量

汇聚/池化

输出通道数 = 输入通道数

降低特征数量

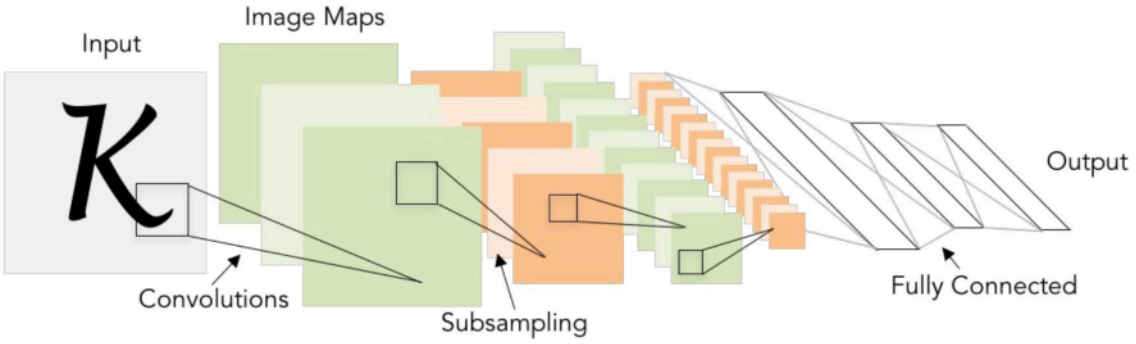
选最大或者取平均



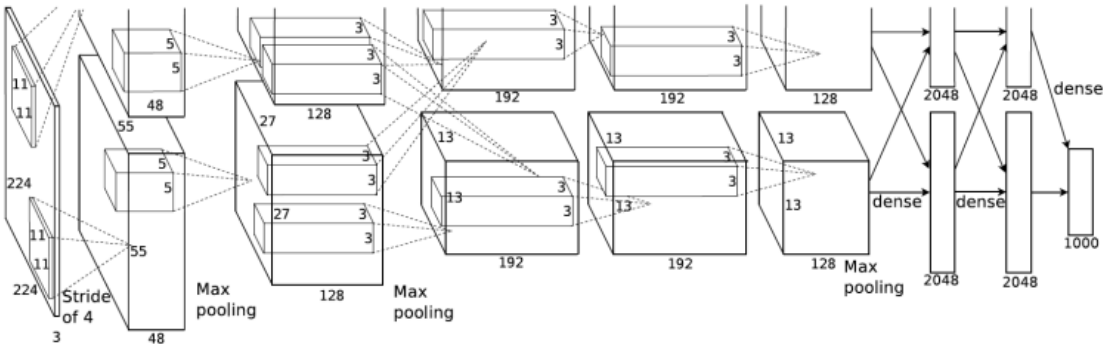
全连接

具体的 CNN

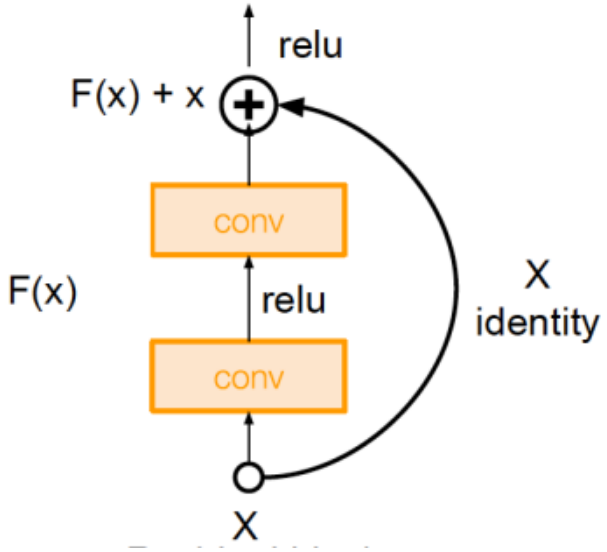
LeNet，其结构是卷积层+池化层+卷积层+池化层+全连接层+全连接层：



AlexNet 则结构更多更复杂，包括五个卷积层和三个全连接层：



ResNet 则有 152 层。层数不是越多越好，层数越多，意味着参数越多，也越难以训练。ResNet 还引入了残差连接（Shortcut Connection），创建了一些直接让相隔的层相连的连接。如下图中的弧线就是残差连接：



RNN

特点

序列处理能力

- RNN能够处理任意长度的序列数据，如时间序列、文本、语音等。
- 通过时间步展开，RNN可以捕捉序列中的时间依赖关系。

共享参数

- RNN在不同时间步共享相同的参数，减少了模型参数数量，提高了计算效率。

记忆能力

- RNN通过隐藏状态（hidden state）传递历史信息，能够记住之前时间步的信息。
- 这种记忆能力使其适合处理具有长期依赖关系的任务。

梯度问题

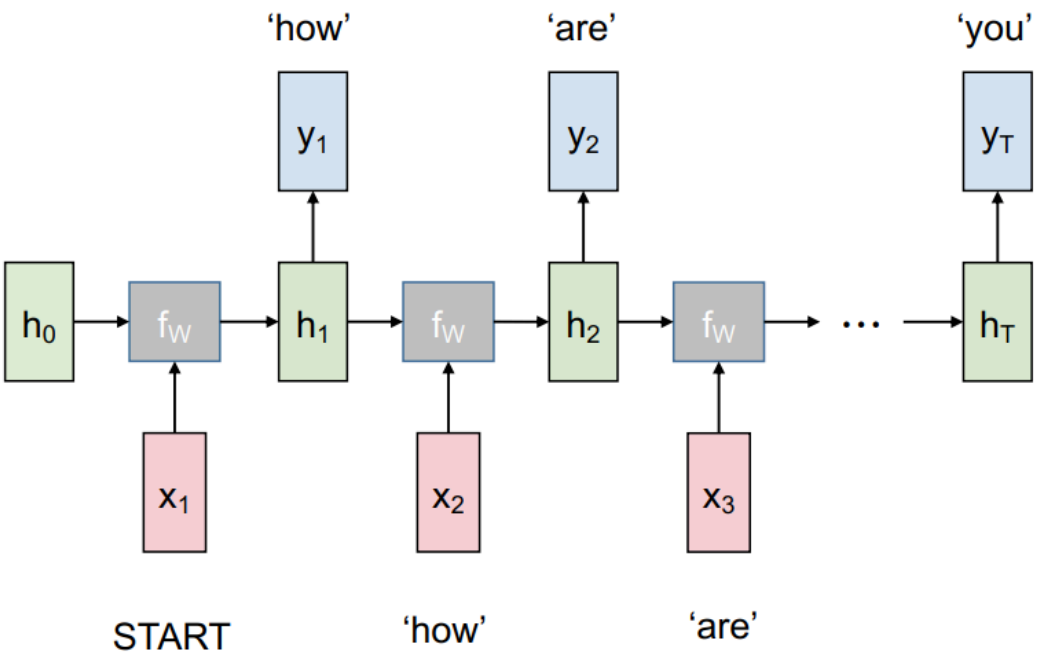
- RNN在训练过程中容易遇到梯度消失或梯度爆炸问题，导致难以学习长期依赖关系。
- 长短期记忆网络（LSTM）和门控循环单元（GRU）等变体通过引入门控机制缓解了这一问题。

灵活性

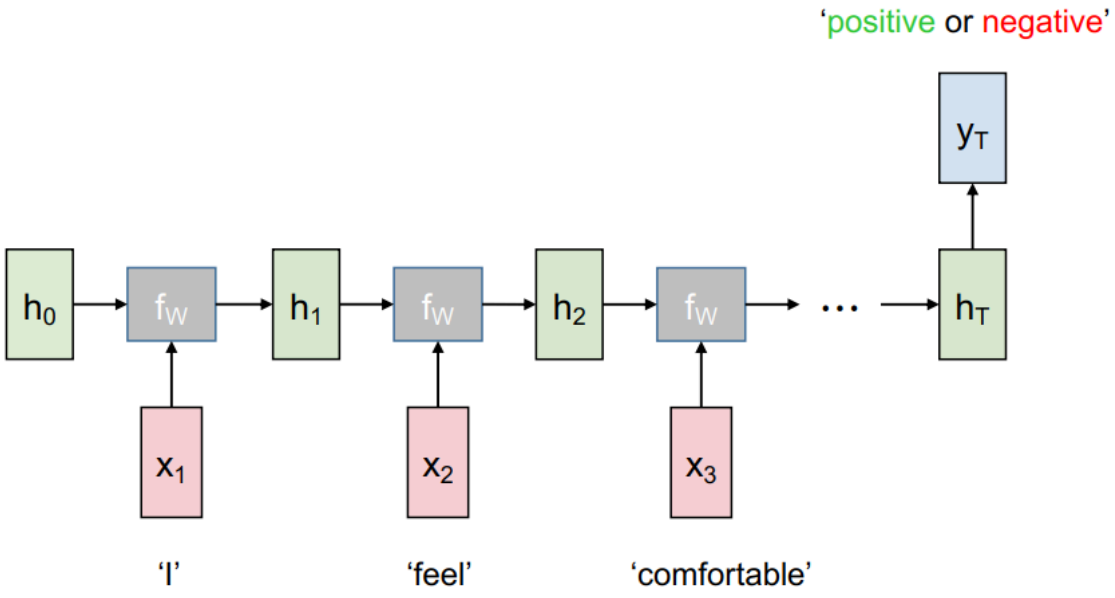
- RNN可以处理多种输入输出结构，如一对一、一对多、多对一和多对多，适用于不同任务。

例子

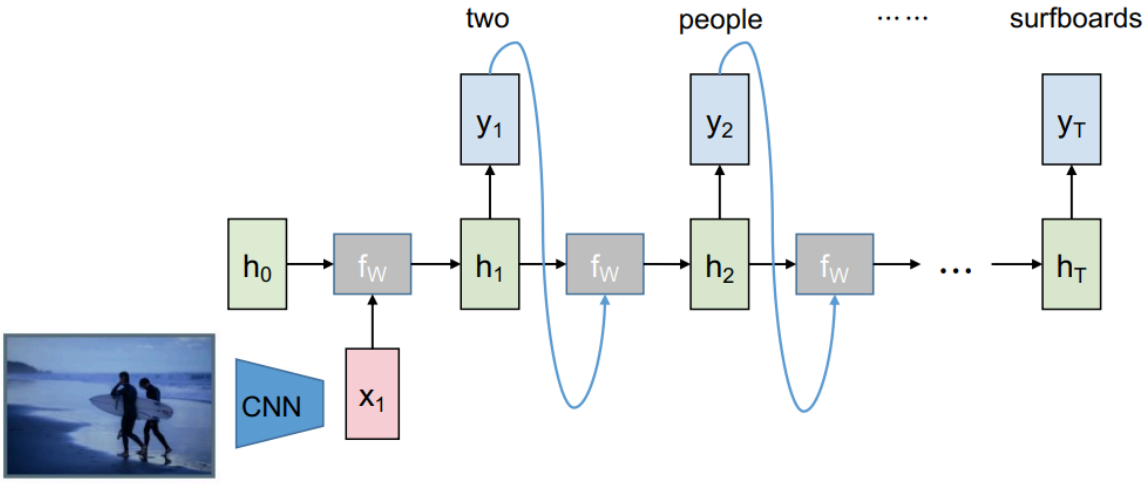
多到多的 RNN：语言模型、视频帧分类等



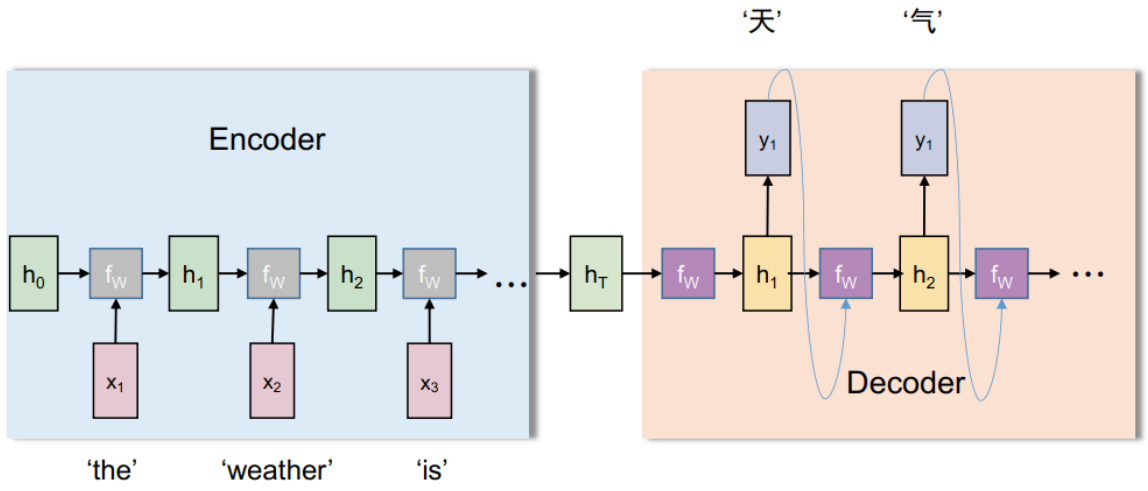
多到一的 RNN：序列分类、情感分析等



一到多的 RNN：图片内容分析等

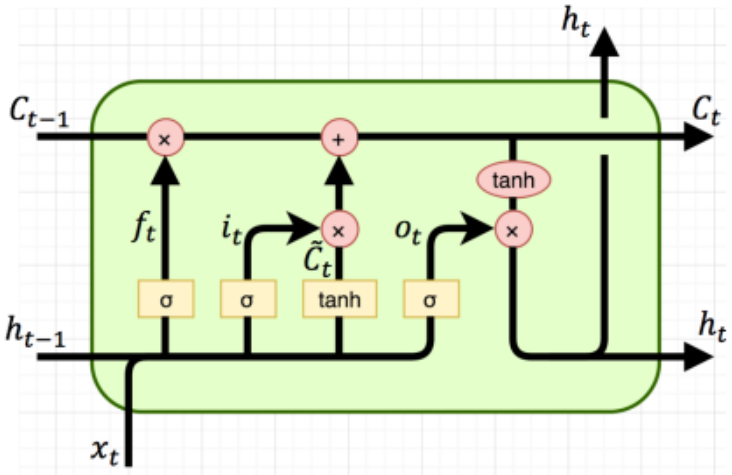


多到一（编码器）+一到多（解码器）的 RNN：机器翻译、语句特征提取等。

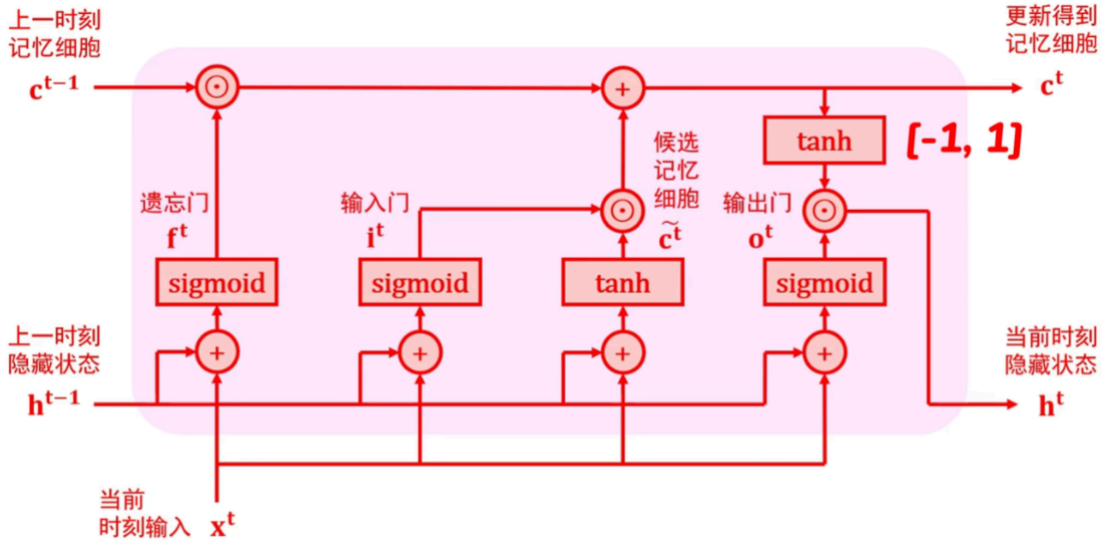


LSTM，长短期记忆网络 解决梯度爆炸消失

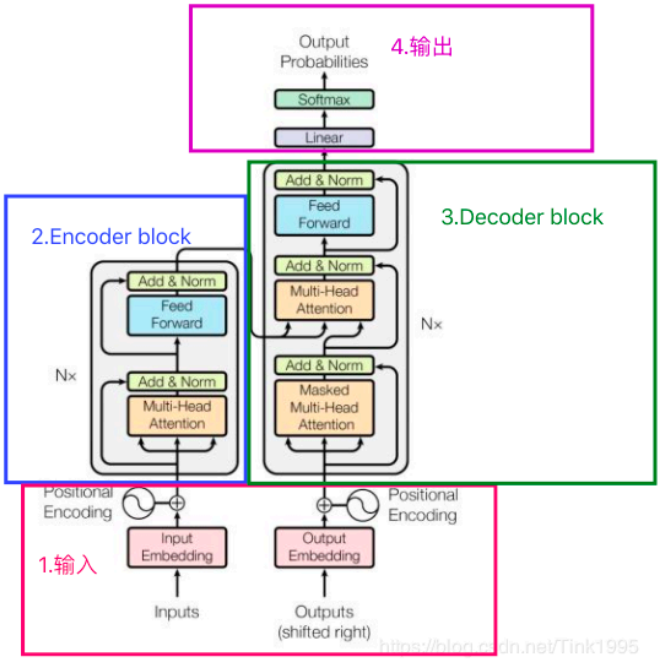
使用sigmod和tanh激活



1. **遗忘门（Forget Gate）**
   决定哪些信息从记忆单元中丢弃。它通过sigmoid函数输出一个0到1之间的值，0表示完全丢弃，1表示完全保留。
2. **输入门（Input Gate）**
   决定哪些新信息被存储到记忆单元中。它包含两个部分：
   - 一个sigmoid函数决定更新的部分。
   - 一个tanh函数生成候选值。
3. **输出门（Output Gate）**
   决定记忆单元中的哪些信息被输出到下一时刻的隐藏状态。它通过sigmoid函数和tanh函数共同作用。



Transformer



https://blog.csdn.net/Tink1995
https://blog.csdn.net/Tink1995
CSDN @告白气球~

优化算法与训练技术

优化算法

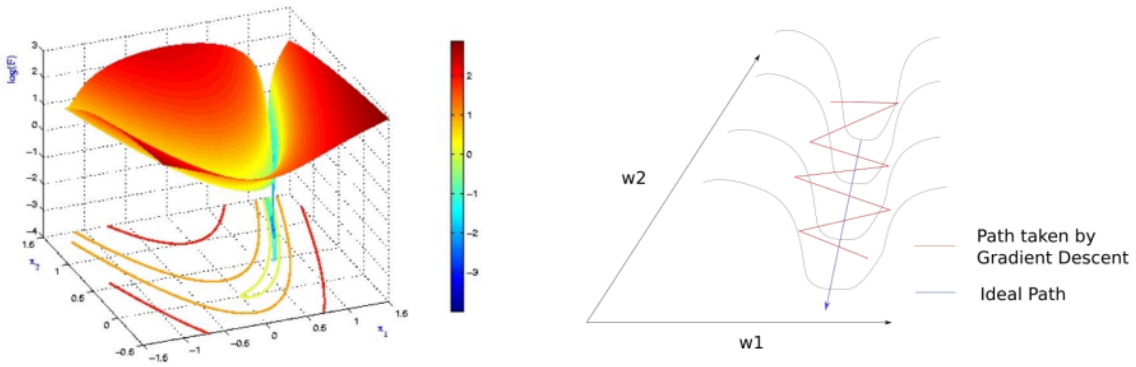
随机梯度下降(SGD)

使用部分样本计算损失，然后继续梯度下降

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta \nabla L(\boldsymbol{x}, y; \boldsymbol{w}_t)$$

其中η是学习率,$\nabla L(\boldsymbol{x}, y; \boldsymbol{w}_t)$是**部分样本**的平均损失。

SGD 的问题是，非常容易陷入局部最优，且解空间中一些病态弯曲（pathological curvature）造成收敛缓慢，如下图：



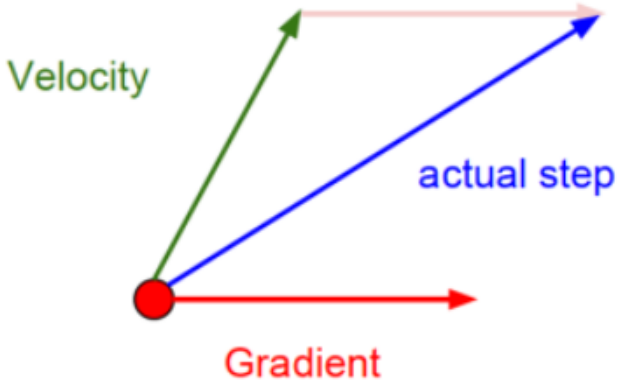
SGD 的路线会是图中的折线，而不是我们期望的直线。

SGD+Momentum

SGD+Momentum 是在 SGD 的基础上引入了动量项，其更新公式为：

$$\boldsymbol{v}_t = \rho \boldsymbol{v}_{t-1} + \nabla L(\boldsymbol{x}, y; \boldsymbol{w}_t)$$
$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta \boldsymbol{v}_t$$

其中$\rho \in (0, 1)$是动量因子，常选择的值是 0.9、0.95、0.99。



这里的动量项与物理学中的动量类似。可以想象，一个小球从坡上滚下，首先由于有动量，下坡速度越来越快；其次如果遇到一个上坡，积累的速度有益于克服上坡的阻力帮其脱出。也就是说，引入动量项可以加速收敛，避免陷入局部最优。

RMSProp

Root Mean Square Propagation,均方根传播。其更新公式为：

$$\boldsymbol{s}_t = \rho \boldsymbol{s}_{t-1} + (1-\rho)(\nabla L(\boldsymbol{x}, y; \boldsymbol{w}_t))^2$$
$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta \nabla L(\boldsymbol{x}, y; \boldsymbol{w}_t) \oslash \sqrt{\boldsymbol{s}_t}$$

其中$\oslash$表示两个相同形状的矩阵**对应元素相除**，s_t是之前梯度的平方的加权。

$\oslash \sqrt{s_t}$这个操作相当于将加权梯度进行了归一化，拉到相同的水平，这样可以避免病态弯曲(两边不会那么陡峭)。

将更新式子展开后即可发现，在衰减因子的作用下，**时间越早的梯度对更新的影响越小。**

$$\boldsymbol{s}_1 = \rho \cdot \boldsymbol{s}_0 + (1-\rho)\big(\nabla f(\boldsymbol{w}_1)\big)^2$$
$$\boldsymbol{s}_2 = \rho^2 \cdot \boldsymbol{s}_0 + \rho(1-\rho)\big(\nabla f(\boldsymbol{w}_1)\big)^2 + (1-\rho)\big(\nabla f(\boldsymbol{w}_2)\big)^2$$
$$\boldsymbol{s}_3 = \rho^3 \cdot \boldsymbol{s}_0 + \rho^2(1-\rho)\big(\nabla f(\boldsymbol{w}_1)\big)^2 + \rho(1-\rho)\big(\nabla f(\boldsymbol{w}_2)\big)^2 + (1-\rho)\big(\nabla f(\boldsymbol{w}_3)\big)^2$$
$$\vdots$$

Weighting

Adam

Adaptive Moment Estimation 的缩写，是一种结合了 Momentum 和 RMSProp 的优化算法。其更新公式为：

$$\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1)\nabla L(\boldsymbol{x}, y; \boldsymbol{w}_t)$$
$$\boldsymbol{s}_t = \beta_2 \boldsymbol{s}_{t-1} + (1-\beta_2)(\nabla L(\boldsymbol{x}, y; \boldsymbol{w}_t))^2$$
$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta \hat{\boldsymbol{m}}_t \oslash \sqrt{\boldsymbol{s}_t}$$

其中m_t是过去梯度的移动平均。当$\beta_1 = 0$时，Adam 就退化为了 RMSProp。

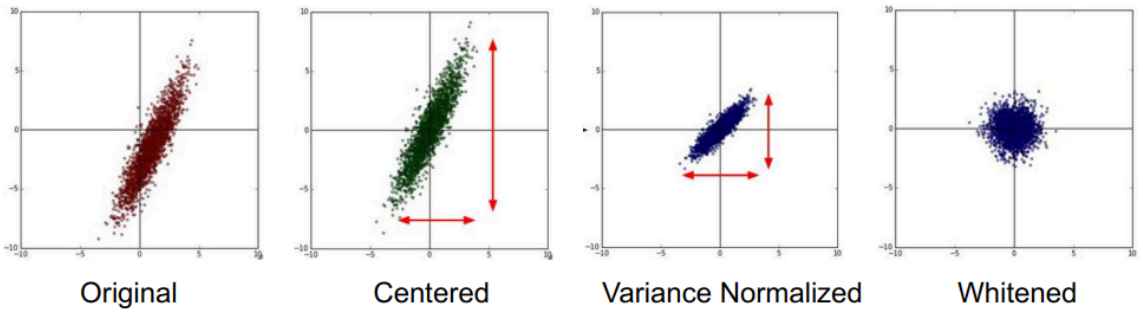
参数的一组常见设置是$\beta_1 = 0.9, \beta_2 = 0.999, \eta = 0.001$

训练技术

一些经常使用的数据处理方法包括：

- 中心化：例如同时减去均值
- 归一化：每个维度的数据都除以标准差

- 白化（Whitening）数据



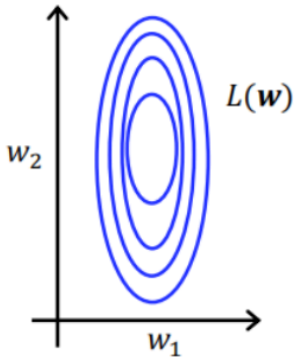
可能的问题

- 当特征值的尺度差异较大时，损失函数 $L(w)$ 的等高线在参数空间中会呈现“拉长的椭圆”形状。
    - 这是因为不同维度的梯度（导数）变化速率不同（例如某些维度的梯度较大，另一些较小）。
- 这种情况下，梯度下降（gradient descent）优化算法在靠近全局最优点时，路径会“震荡”且收敛缓慢。

解决方法

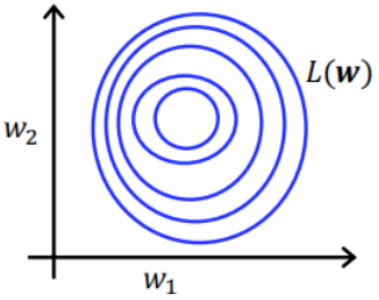
- 通过预处理，将所有特征值缩放到相似的尺度（通常通过中心化和归一化）。
- 当特征值的尺度相似时，损失函数 $L(w)$ 的等高线会变得更接近于“圆形”。
- 这样一来，梯度下降法的每次迭代都会更稳定，优化路径更直，收敛速度更快。

Obviously, the cost function $L(\boldsymbol{w})$ decreases at different rates for different dimensions



This results in the *slow convergence* of gradient descent. *Why??*

If the features are preprocessed to similar scales, then the loss function looks like



初始化

从**固定高斯分布随机取样**来初始化神经网络中各层的**权重**，尤其是在较浅的神经网络(层数小于8)时，会取得较好
的效果。

一个常用的高斯分布是：

$$w \sim N(0, 0.01^2)$$

如果激活函数使用的是ReLU，则可以使用截断的高斯分布，其是通过将标准高斯分布中**超出特定范围的值的概率设
为零**，并对剩余部分进行归一化处理得到的。

$$w \sim N_T(0, 0.01^2)$$

如果神经网络较深，可以通过对以下高斯分布进行采样，来对计算出第i层节点值要用到的参数初始化

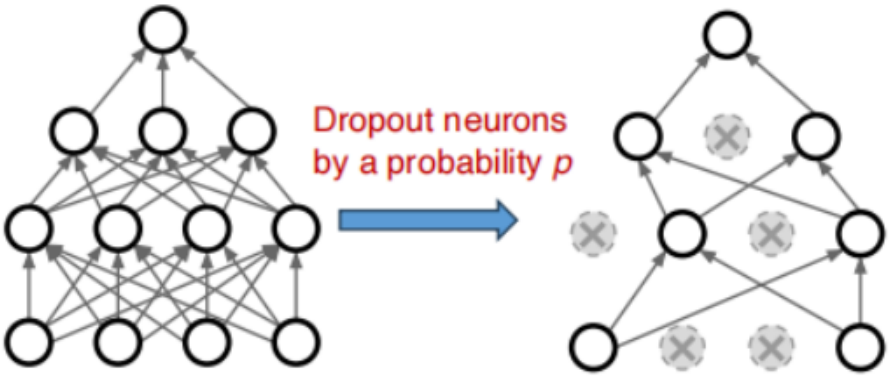
$$w \sim N(0, \sigma_i^2)$$

其中σ_i是第$i-1$层的神经元个数的倒数的平方根。

Dropout

一种用于防止神经网络过拟合的正则化技术

它的核心思想是在训练过程中随机“丢弃”（即暂时移除）神经网络中的一部分神经元，从而减少神经元之间的复杂共适应性（co-adaptation），使网络更加鲁棒。



训练阶段的 Dropout

- 核心思想：
  - 在每次前向传播（forward pass）中，随机将一部分神经元的输出设置为 0。
  - 被丢弃的神经元在本次训练迭代中不参与计算。
- 丢弃概率 p：
  - 丢弃神经元的概率 p 是一个超参数，通常设置为 0.5（即每次训练时，大约 50% 的神经元会被丢弃）。
  - 例如，如果 $p = 0.5$，则每个神经元有 50% 的概率被丢弃。
- 作用：
  - Dropout 可以有效缓解深度神经网络中的过拟合问题。
  - 通过随机丢弃神经元，Dropout 迫使网络**不依赖于某些特定的神经元**，从而提高模型的**泛化能力**。

测试阶段的 Dropout

- 核心思想：
  - 在测试阶段，模型的输出应该是确定的（即对于相同的输入，输出应该是固定的）。
  - 因此，测试阶段不使用 Dropout，所有神经元都参与计算。
- 缩放操作：
  - 为了保持训练和测试时的一致性，测试阶段需要对每个神经元的输出进行缩放。
  - 每层的输入也要加上 Dropout 的概率

$$h_{l+1} = \sigma(p\boldsymbol{W}_l\boldsymbol{h}_l)$$

  - 假设某个神经元的输出为 h，则测试时的输出为：

$$h_{\text{test}} = h \cdot (1-p)$$

  - 这种缩放操作确保了训练和测试时每个神经元的输入范围一致。

批量归一化

Batch Normalization, BN。

对每一层的输入进行标准化处理，使其均值为 0，方差为 1。这样可以减少内部协变量偏移（Internal Covariate Shift），即网络层输入分布的变化，从而加速训练并提高模型的稳定性。

一种解决方法就是引入 BN 层，将某一个隐藏层的输出$\{\boldsymbol{h}_l^{(i)}\}_{i=1}^N$变为标准正态分布：

$$\hat{\boldsymbol{h}}_l^{(i)} = (\boldsymbol{h}_l^{(i)} - \boldsymbol{\mu}_l) \oslash \boldsymbol{\sigma}_l$$

其中μ_l, σ_l^2分别是$h_l^{(i)}$的均值和标准差。进一步地，可以引入两个可学习的参数γ_l, β_l，来调整归一化后的值：

$$\tilde{\boldsymbol{h}}_l^{(i)} = \gamma_l \odot \hat{\boldsymbol{h}}_l^{(i)} + \beta_l$$

其中⊙是对应元素相乘。

使用所有样本计算均值和标准差代价太大，一般用某一个小批次的均值和方差来近似.

在训练阶段，使用当前 mini-batch 的均值和方差来标准化输入。然而，在测试阶段，这种方法的两个主要问题是：

- 输出依赖于 mini-batch：如果测试时仍然使用当前 mini-batch 的均值和方差，模型的输出会依赖于具体的 mini-batch，导致不一致的结果。
- mini-batch 大小可能为 1：在测试时，mini-batch 的大小可能为 1（即单个样本），此时无法准确计算均值和方差。

一般有两个**解决方法**

方法 1：计算精确的均值和方差

- 使用整个训练数据集的均值和方差，而不是 mini-batch 的均值和方差。
- 具体公式：

  $$\mu_p = \frac{1}{N} \sum_{i=1}^{N} \hat{R}_p^{(i)}$$

  $$\sigma_p^2 = \frac{1}{N} \sum_{i=1}^{N} (\hat{R}_p^{(i)} - \mu_p)^2$$

  其中：

  - N 是训练数据集的大小。
  - $\hat{R}_p^{(i)}$ 是第 i 个样本在某一层的输出。
- 优点：计算精确，结果稳定。
- 缺点：需要遍历整个训练数据集，计算成本较高。

方法 2：使用移动平均（Moving Average）

- 在训练阶段，通过移动平均的方式逐步估计均值和方差，并在测试阶段使用这些估计值。
- 具体步骤：
  1. 在训练时，记录每个 mini-batch 的均值和方差。
  2. 使用指数移动平均（Exponential Moving Average, EMA）更新全局的均值和方差：

$$\mu_{\text{global}} = \alpha \cdot \mu_{\text{global}} + (1 - \alpha) \cdot \mu_{\text{batch}}$$

$$\sigma^2_{\text{global}} = \alpha \cdot \sigma^2_{\text{global}} + (1 - \alpha) \cdot \sigma^2_{\text{batch}}$$

     其中，α 是平滑系数（通常接近 1，如 0.9 或 0.99）。
  3. 在测试阶段，直接使用训练时估计的全局均值和方差。
- 优点：计算高效，适合在线学习或大规模数据集。
- 缺点：估计值可能不如精确计算准确

最后，整个BN层的计算过程可以写为：

$$\text{BN}(\boldsymbol{h}_l) = \gamma_l \odot (\boldsymbol{h}_l - \boldsymbol{\mu}_l) \oslash \boldsymbol{\sigma}_l + \boldsymbol{\beta}_l$$

几种归一化对比

1. Batch Normalization

Batch Normalization 是最常见的一种归一化方法，主要对批次中的数据进行归一化。它的作用是加速收敛、减小内部协变量偏移（Internal Covariate Shift），并起到一定的正则化效果。

Batch Normalization 是对 **批次数据** 的归一化，在训练和测试中表现可能有所不同，因为测试时通常使用训练中的移动平均均值和方差。

2. Layer Normalization

Layer Normalization 通常用于 **Recurrent Neural Networks (RNNs)** 或 Transformer 中，与 Batch Normalization 的主要区别在于它对 **样本数据的每一层** 进行归一化。

优势：

- Layer Normalization 对批次大小不敏感，适合于小批次训练或序列任务（如 RNN）。
- 在训练和测试阶段行为一致。

---

3. Instance Normalization

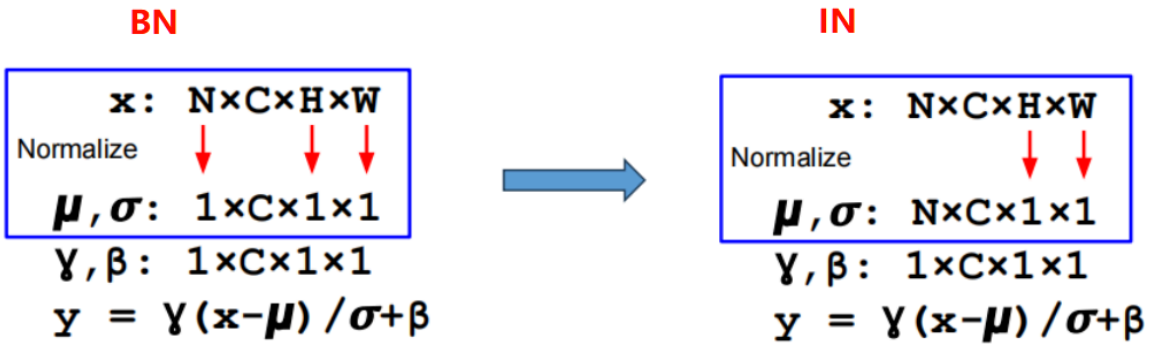
Instance Normalization 常用于风格迁移（Style Transfer）等任务，注重每个样本的独立性。

优势：

- Instance Normalization 的行为与测试和训练一致，非常适合处理每个样本的独立性要求（如图像风格）。

N是样本数量,D是特征维度,**C**: 通道数，H 和 W 分别表示卷积神经网络中特征图（Feature Map）的高度和宽度



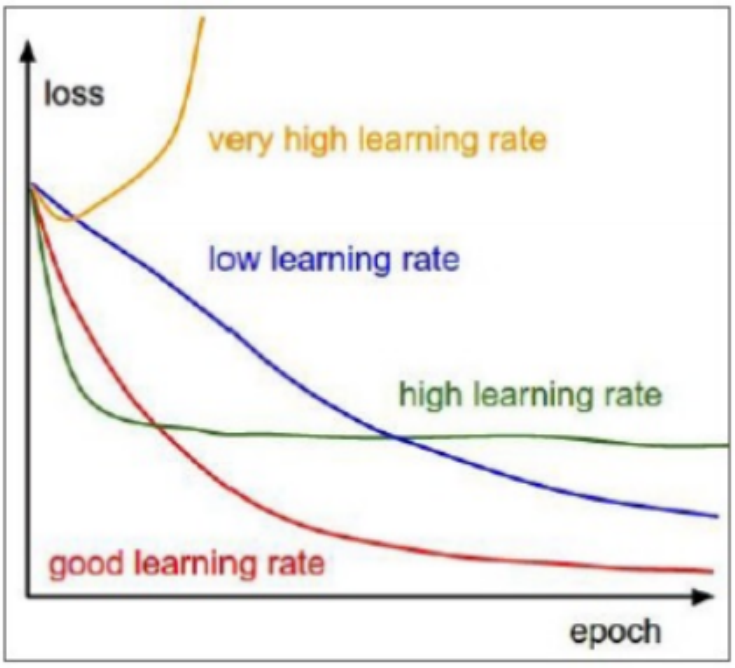


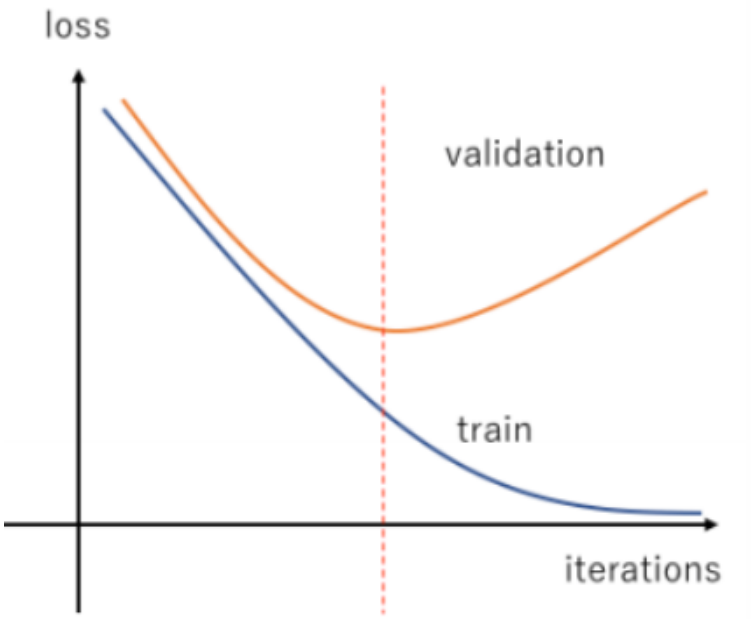
超参数调整

学习率等超参数的调整也不是盲目的，有一些一般性的方法：

1. 首先尝试**在许多问题上表现优异**的超参数**(经验)**
2. 根据训练结果逐渐来调整

训练损失曲线暗示了良好的学习率的可能范围　　在训练集和验证集上的性能之间的关系表明，该模型过拟合





决策树

Decision Tree，DT。

基本概念

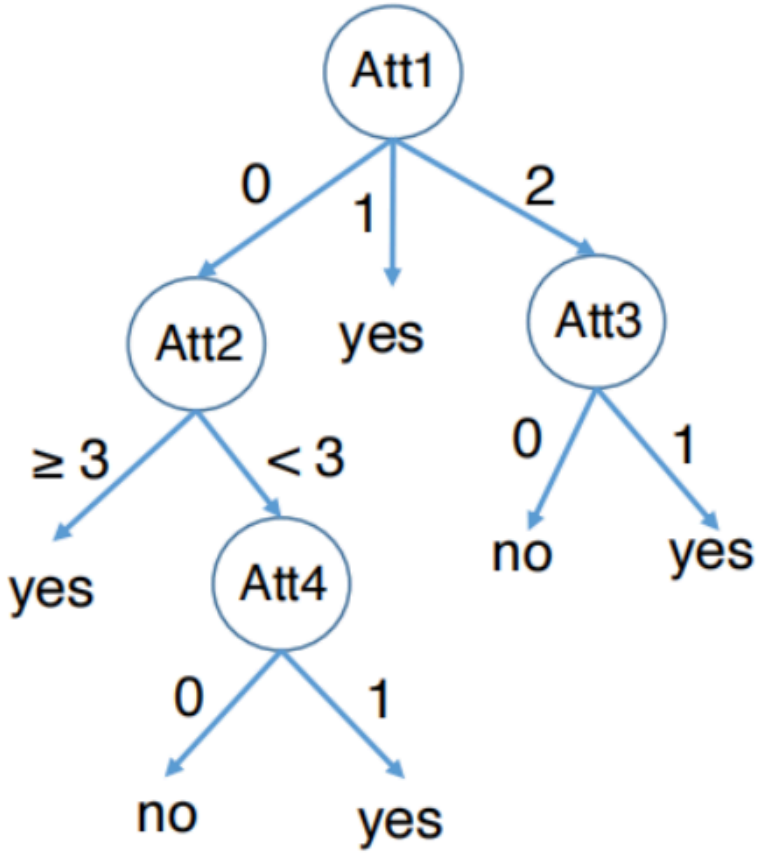
决策树中：

1. 根节点和内部节点表示一个**属性**（特征）
2. 叶子节点对应一个（最终被划分为的）**类别**
3. 边对应某个**属性值**

利用决策树进行分类，就是从根节点开始，根据属性值逐步向下，直到到达叶子节点。叶子节点的类别就是最终的分类结果。

学习出一个好的决策树，要考虑：

- 每个节点上用**哪个属性**进行划分（可以重复使用属性）
- 从某个节点如何引入边（如何**划分属性值**）
- **何时停止**拓展决策树



选择属性标准

从最明显的特征入手，或者说从“包含最多信息”的特征入手，

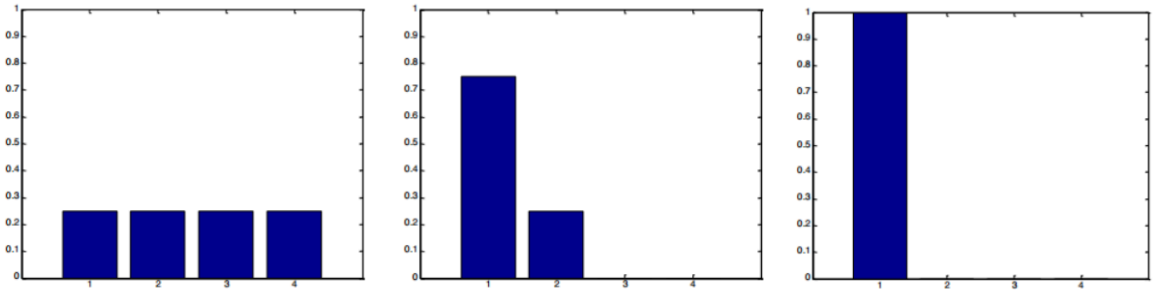
熵 (Entropy) 是对随机变量的不确定性的度量。

给定一个随机变量$Z \sim p(z)$,则其熵是

$$H(Z) = -\sum_{z \in \mathcal{Z}} p(z) \log p(z)$$

其中$\mathcal{Z}$是随机变量Z可能取值的集合。

熵越大，变量取值的不确定性越大，分布越平坦（均匀）



(a) (b) (c)

a) $H(Z) = -4 \times 0.25 \log_2 0.25 = 2$ bits

b) $H(Z) = -0.75 \log_2 0.75 - 0.25 \log_2 0.25 \approx 0.8133$ bits

c) $H(Z) = -1 \log_2 1 = 0$ bits

熵越大,越不确定

条件熵:在给定一个随机变量Y的情况下，另一个随机变量Z的熵:

$H(Z|Y = y) = -\sum_{z \in \mathcal{C}} p(z|y) \log p(z|y)$

$$H(Z|Y) = \sum_{y \in \mathcal{Y}} p(Y = y) H(Z|Y = y)$$

我们总有$H(Z) \geqslant H(Z|Y)$,可以这样直观的理解：获取额外信息 (Y的取值)后，把握一般都会变大，对Z的不
确定性会减小。

信息增益:

Y带来的信息增益$IG(Y)$,是知道随机变量Y取值，使另一个随机变量Z的熵减少的程度:

$$IG(Y) = H(Z) - H(Z|Y)$$

构建决策树

从根节点开始，计算知道各个属性对确定最后分类结果带来的**信息增益**，选择信息增益**最大的属性作为根节点。**

接下来，根据根节点各个属性的取值，将数据集**划分为若干个子集，**对每个子集递归地**重复**上述过程，用其他属性来构建子树。

停止拓展条件:

- 当某个节点的**所有样本都属于同一类别**时
- 所有属性都已使用过
- 树达到最大深度
- 信息增益值小于某个阈值

树过大或者过复杂时，容易过拟合，可以通过**剪枝**来解决这个问题。

利用验证集，可以在构建过程中或者构建结束后使用下面的策略：

- 减去某个节点不影响验证集的准确率
- 减去对验证集准确率提升最小的节点
- 验证集上的准确率开始下降即停止扩展

例子

https://www.bilibili.com/video/BV1Cq4y1S7k1

这里的info相当于H(Z)

序号	天气	气温	湿度	风	打网球
1	晴	热	高	无	N
2	晴	热	高	有	N
3	多云	热	高	无	Y
4	雨	温暖	高	无	Y
5	雨	凉爽	正常	无	Y
6	雨	凉爽	正常	有	N
7	多云	凉爽	正常	有	Y
8	晴	温暖	高	无	N
9	晴	凉爽	正常	无	Y
10	雨	温暖	正常	无	Y
11	晴	温暖	正常	有	Y
12	多云	温暖	高	有	Y
13	多云	热’	正常	无	Y
14	雨	温暖	高	有	N

第1步：先看序号，计算总的信息量

$$\inf o([9,5]) = -\frac{9}{14}\log\frac{9}{14} - \frac{5}{14}\log\frac{5}{14} = 0.94$$

([Yes9,No5])

(注：这里我把log默认底数是2)

第2步：在天气、气温、湿度、风 4个指标中选一个作为根节点

这里的E(天气)相当于H(Z|Y),里面的xx/14就是不同的P(Y)

序号	天气	气温	湿度	风	打网球
1	晴	热	高	无	N
2	晴	热	高	有	N
3	多云	热	高	无	Y
4	雨	温暖	高	无	Y
5	雨	凉爽	正常	无	Y
6	雨	凉爽	正常	有	N
7	多云	凉爽	正常	有	Y
8	晴	温暖	高	无	N
9	晴	凉爽	正常	无	Y
10	雨	温暖	正常	无	Y
11	晴	温暖	正常	有	Y
12	多云	温暖	高	有	Y
13	多云	热’	正常	无	Y
14	雨	温暖	高	有	N

第2步：在天气、气温、湿度、风 4个指标中选一个作为根节点

天气-晴：

inf o ([2,3]) = -2/5log2/5 - 3/5log3/5=0.97

天气-雨：

$$\inf o([3,2]) = -\frac{3}{5}\log\frac{3}{5} - \frac{2}{5}\log\frac{2}{5} = 0.97$$

天气-云：

$$\inf o([4,0]) = 0.00$$

天气各种情况加权和：

E（天气）= $$\inf o([2,3],[4,0],[3,2]) = \frac{5}{14}\inf o[2,3] + \frac{4}{14}\inf o[4,0] + \frac{5}{14}\inf o[3,2] = 0.69$$

inf o:信息量
E：信息熵
gain：信息增益

基于天气决定是否打网球的信息增益为：

$$gain(天气) = \inf o([9,5]) - \inf o([2,3],[4,0],[3,2]) = 0.25$$

第一步算的总的信息量=0.94

第2步天气的三种情况加权之和=0.69

继续算下一个的信息增益

序号	天气	气温	湿度	风	打网球
1	晴	热	高	无	N
2	晴	热	高	有	N
3	多云	热	高	无	Y
4	雨	温暖	高	无	Y
5	雨	凉爽	正常	无	Y
6	雨	凉爽	正常	有	N
7	多云	凉爽	正常	有	Y
8	晴	温暖	高	无	N
9	晴	凉爽	正常	无	Y
10	雨	温暖	正常	无	Y
11	晴	温暖	正常	有	Y
12	多云	温暖	高	有	Y
13	多云	热’	正常	无	Y
14	雨	温暖	高	有	N

第2步：在天气、气温、湿度、风 4个指标中选一个作为根节点

气温-热：

inf o（[2,2]）= -2/4log2/4 - 2/4log2/4=

气温-温暖：

inf o（[5,1]）= -5/6log5/6 - 1/6log1/6=

气温-凉爽：

inf o（[3,1]）= -3/4log3/4 - 1/4log1/4=

天气各种情况加权和：

inf o（[2,2],[5,1],[3,1]）=
4/14inf o（[2,2]）+6/14inf o（[5,1]）+4/14inf o（[3,1]）=

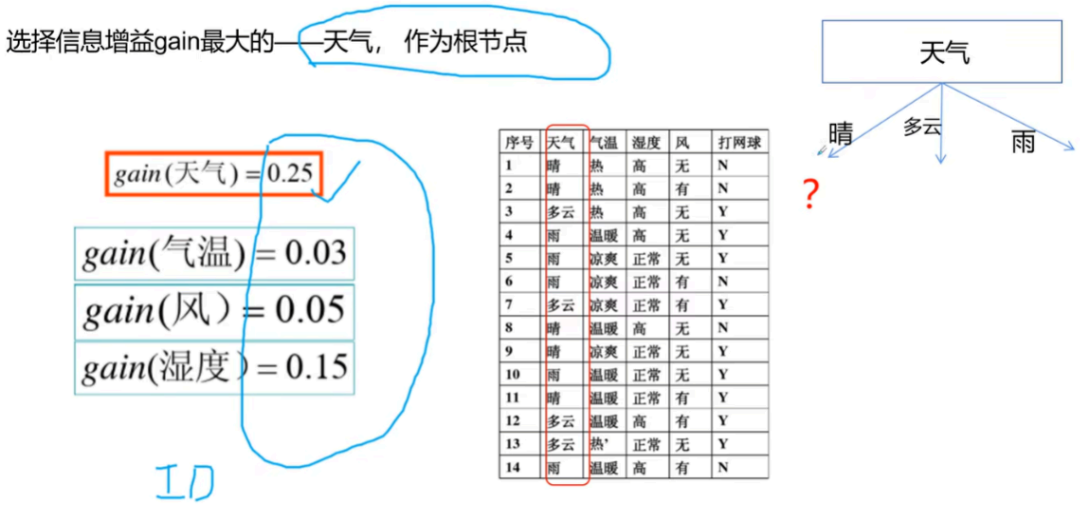
基于气温决定是否打网球的信息增益为：

gain(气温)=inf o([9,5])-inf o([2,2],[5,1],[3,1])=0.03

第一步算的总的信息量=0.94

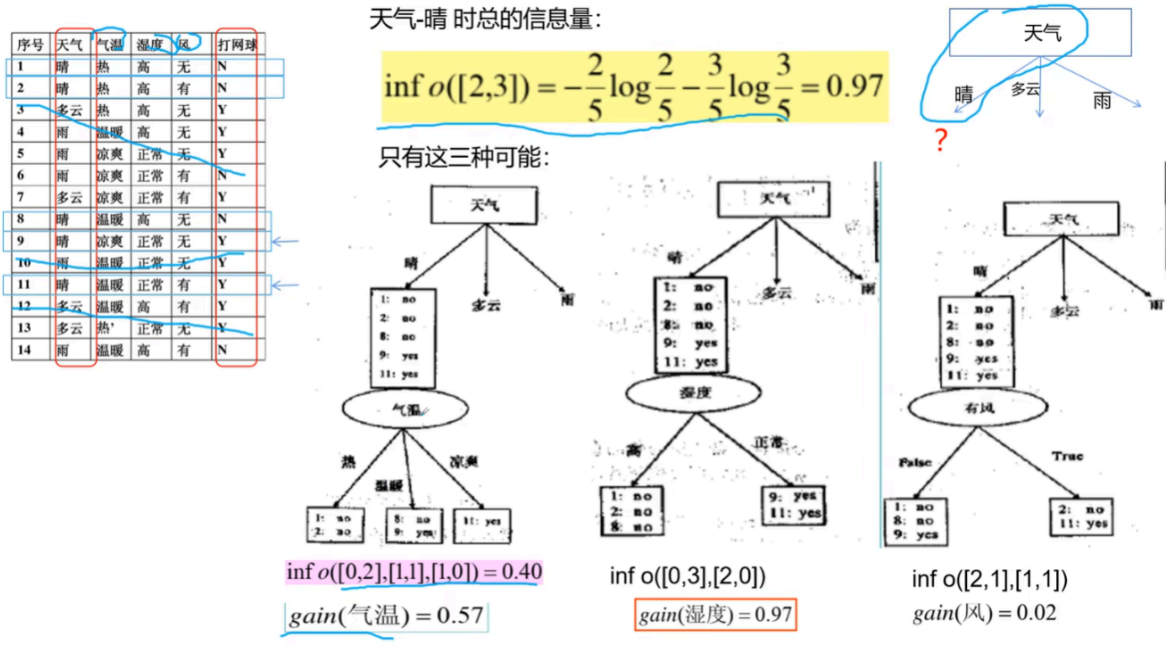
第2步天气的三种情况加权之和=0.797

假设全部算完开始构建

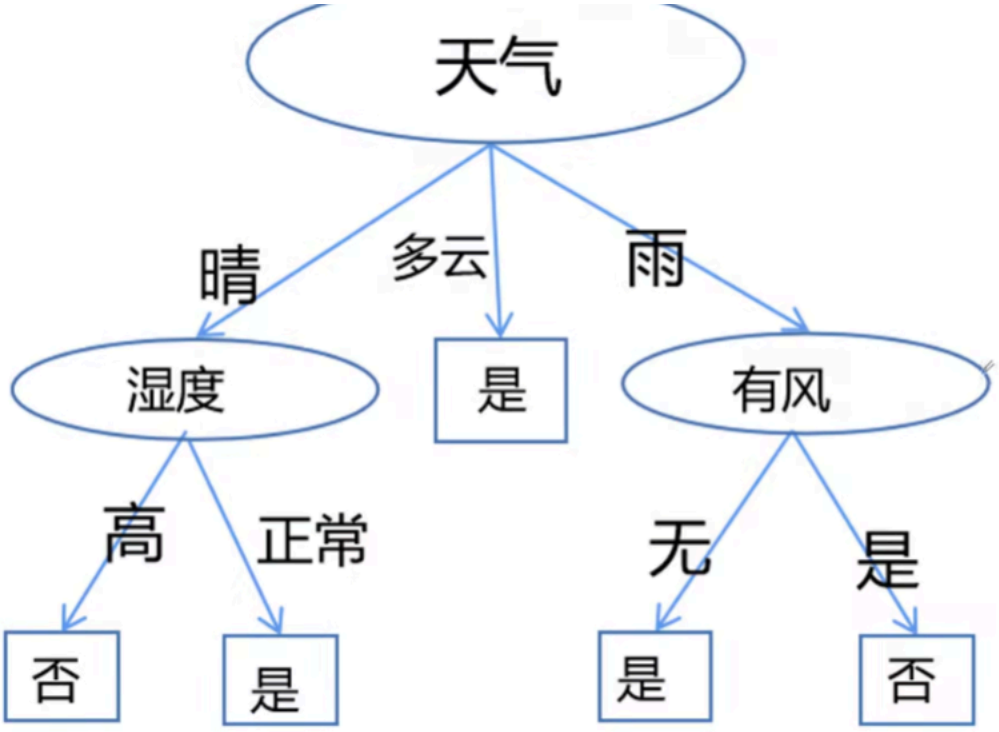


序号	天气	气温	湿度	风	打网球
1	晴	热	高	无	N
2	晴	热	高	有	N
3	多云	热	高	无	Y
4	雨	温暖	高	无	Y
5	雨	凉爽	正常	无	Y
6	雨	凉爽	正常	有	N
7	多云	凉爽	正常	有	Y
8	晴	温暖	高	无	N
9	晴	凉爽	正常	无	Y
10	雨	温暖	正常	无	Y
11	晴	温暖	正常	有	Y
12	多云	温暖	高	有	Y
13	多云	热’	正常	无	Y
14	雨	温暖	高	有	N

看每一个分支,就只需要**考虑满足分支这个条**件的这些行,算该条件每个**剩余属性**的**信息增益**



结果



集成学习

集成学习是一种机器学习策略，通过**结合多个弱分类器**（Weak Classifier）的预测来提高整体模型的性能。

基本思想是，虽然单个弱分类器在整体数据集上的表现可能一般，但它们可能在数据集的不同**子集上表现优异**。

通过合理地组合这些弱分类器，可以得到一个性能更强大的强分类器（Strong Classifier）。

Bagging

Bootstrap Aggregating, 自举汇聚法。

抽样构建方法

1. 创建训练数据的子集

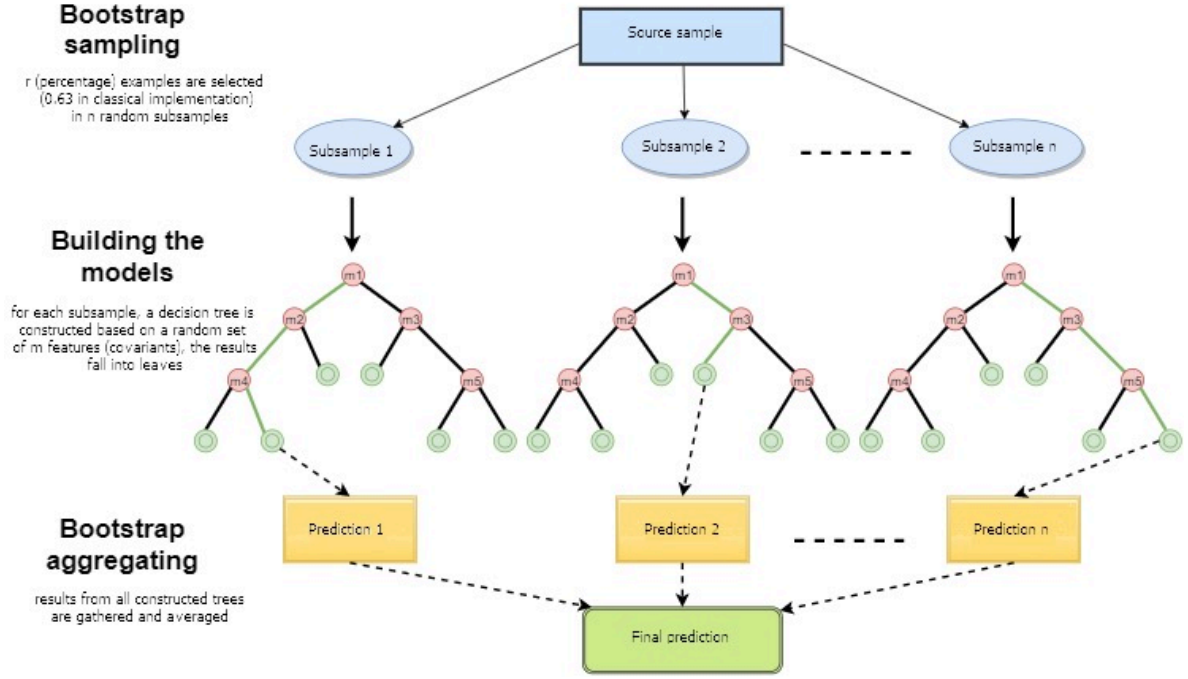
- Bootstrap采样：从原始训练数据集中通过**有放回地随机抽样**生成多个子集。假设有N个样本，每次**随机抽取N'个样本**（通常N' = N），生成一个子集。这个过程**重复K次**，生成K个不同的子集S1, S2, ..., SK。
  样本被抽中后会放回原数据集，因此同一个样本可能在同一个子集中被多次抽中，也可能有些样本在某个子集中没有被抽中。

2. 训练多个基模型

- 训练基模型：在每个生成的子集SK上训练一个独立的决策树模型。因此，最终会**得到K个决策树模型**。

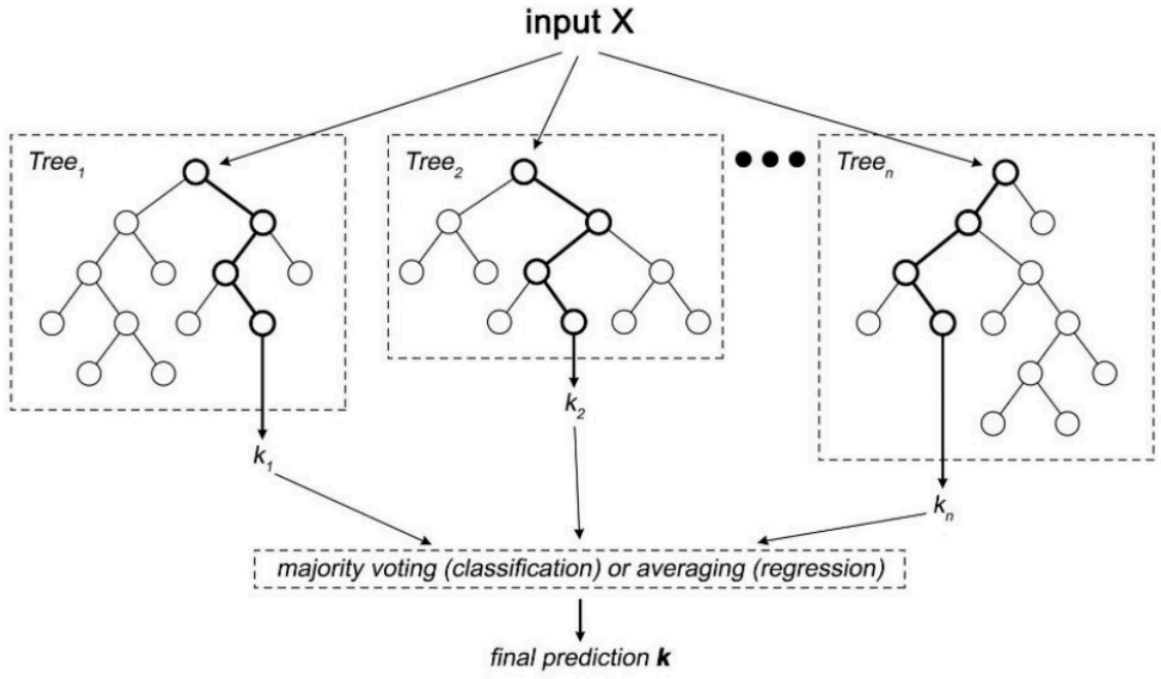
3. 结合多个模型的预测

- 结合预测结果：对于分类任务，通过**多数投票**（majority voting）来确定最终的预测类别；对于回归任务，则通常取所有基模型预测值的**平均值**作为最终预测结果。



可见 Bagging 是一种**无权**的组合方式。

这种方式下，各个弱分类器的误差可能还是有较大的相关性，我们可以进一步增加随机性。例如，如果弱分类器采用的是决策树，我们可以在构建决策树是，只使用**随机地抽取的部分属性**。这样每个决策树的形状都不尽相同，是一片随机森林(Random Forest) 。



Boosting

每次训练一个弱分类器，对于**被误分类的样本**，给予其**更大的权重**，然后投入到下一个弱分类器的训练中。最后将所有弱分类器带权组合起来。

AdaBoost (Adaptive Boosting自适应增强) 算法：

1.初始化样本权重$w_i^{(0)} = \frac{1}{N}, i = 1, 2, \cdots, N$

2.对于第k轮迭代，训练一个分类器$h_k(\boldsymbol{x})$,用带权$\boldsymbol{w}^{(k-1)}$的样本
3.训练结束后，计算错误率：

$$\epsilon_k = \frac{\sum_{i=1}^{N} w_i^{(k-1)} I(h_k(\boldsymbol{x}_i) \neq y_i)}{\sum_{i=1}^{N} w_i^{(k-1)}}$$

也即是被**误分类的样本**的权重之和占**所有**样本权重之和的比例

> I是指示函数,()里面条件为真的时候值是1,否则是0

4.该分类器在最终组合中的权重为

$$\alpha_k = \frac{1}{2} \ln \frac{1 - \epsilon_k}{\epsilon_k}$$

> 错的越多,ϵ_k越大,权重α_k越小

5.更新样本权重，用于下一轮训练：

$$w_i^{(k)} = w_i^{(k-1)} \exp(-\alpha_k y_i h_k(\boldsymbol{x}_i))$$

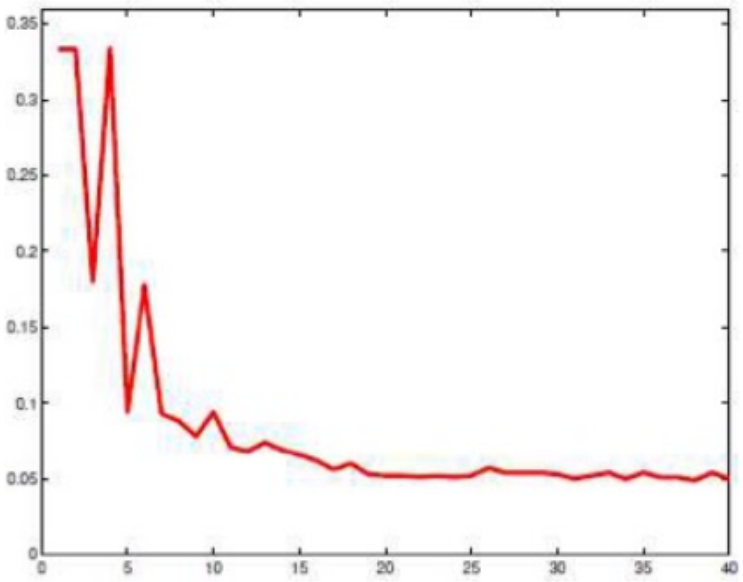
> y_i 是样本的真实标签，$h_k(\boldsymbol{x}_i)$ 是第 k 个分类器的预测结果

由上述训练过程可见，误分类的样本权重会增大，正确分类的样本权重会减小，这样可以使得下一个弱分类器更关注于误分类的样本，这是一个**增强**的过程。同时还可以发现，最后的组合是带权的，与弱分类器的表现有关。

$$
\begin{aligned}
&1: \mathcal{D}_1(\boldsymbol{x}) = 1/m \\
&2: \text{for } t = 1, 2, \ldots, T \text{ do} \\
&3: \quad h_t = \mathfrak{L}(D, \mathcal{D}_t) \\
&4: \quad \epsilon_t = P_{\boldsymbol{x} \sim \mathcal{D}_t}(h_t(\boldsymbol{x}) \neq f(\boldsymbol{x})) \\
&5: \text{if } \epsilon_t > 0.5 \text{ then break} \\
&6: \quad \alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right) \\
&7: \quad \mathcal{D}_{t+1}(\boldsymbol{x}) = \frac{\mathcal{D}_t(\boldsymbol{x})}{Z_t} \times \begin{cases} \exp(-\alpha_t), & \text{if } h_t(\boldsymbol{x}) = f(\boldsymbol{x}) \\ \exp(\alpha_t), & \text{if } h_t(\boldsymbol{x}) \neq f(\boldsymbol{x}) \end{cases} \\
&\quad = \frac{\mathcal{D}_t(\boldsymbol{x}) \exp(-\alpha_t f(\boldsymbol{x}) h_t(\boldsymbol{x}))}{Z_t} \\
&8: \text{end for} \\
&\text{输出: } H(\boldsymbol{x}) = \operatorname{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(\boldsymbol{x})\right)
\end{aligned}
$$

典型的弱分类器 决策树;逻辑回归;神经网络

一个典型的误分类率曲线如下图所示：



考虑定义一个指数损失函数：

$$L(\boldsymbol{w}) = \sum_{i=1}^{N} \exp(-y_i h(\boldsymbol{x}_i))$$

$h(\boldsymbol{x}_i)$是组合分类器

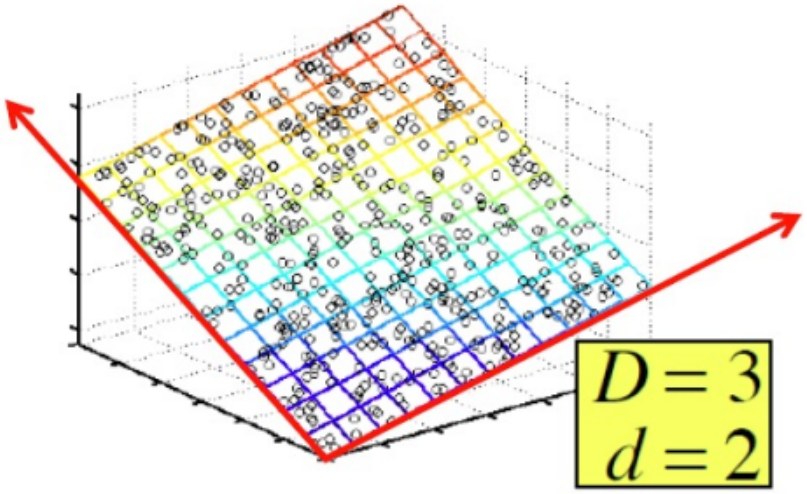
$$h(\boldsymbol{x}) = \sum_{i=1}^{K} \alpha_i h_i(\boldsymbol{x})$$

可以证明，AdaBoost 算法等价于在最小化指数损失函数。

PCA

Principal Component Analysis，主成分分析。

现实问题中，许多数据的维度都是巨大的，如果直接拿来计算，代价非常高。而且，数据的维度往往是冗余的，经常出现高维数据驻留在近似的低维空间的情况，如下图：



原始数据虽然是三维的，但近似可以认为都在一个二维平面上。

所以如果我们能找到数据分布的主要方向，就能用较少的维度来表示数据，从而节省计算资源。

角度一——最小化误差

由于抛弃了一些维度，误差是客观存在的，但是希望重建后数据与原始数据的误差尽可能小。

如果给定一组正交基，即一组向量$u_1, u_2, \cdots, u_M$满足：

$$\boldsymbol{u}_i^T \boldsymbol{u}_j = \delta_{i,j} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$

在这组正交基张成的**子空间**中，某个向量x的最佳近似是：

$$\tilde{\boldsymbol{x}} = \sum_{i=1}^{M} \alpha_i \boldsymbol{u}_i$$

其中$\alpha_i = \boldsymbol{u}_i^T \boldsymbol{x}$。

> 因为是子空间,$\tilde{\boldsymbol{x}}$是x在子空间的正交**投影**

证明方法如下

$$\min \|\boldsymbol{x} - \tilde{\boldsymbol{x}}\|^2 = \min \|\boldsymbol{x} - \sum_{i=1}^{M} \alpha_i \boldsymbol{u}_i\|^2 = \min(\|\boldsymbol{x}\|^2 - \sum_{i=1}^{M} \alpha_i \boldsymbol{u}_i^T \boldsymbol{x} + \sum_{i=1}^{M} \alpha_i^2)$$

这是一个凸优化问题，其解就是$\alpha_i = \boldsymbol{u}_i^T \boldsymbol{x}$.

> 最小化 $\|\mathbf{x} - \tilde{\mathbf{x}}\|^2$ 的过程实际上是在求正交投影，因为在欧几里得空间中，最近的点就是正交投影

接下来就是确定一组正交基，能够最小化这个最小的误差。
一般，我们处理$x - \overline{x}$,而不是直接处理x,其中$\bar{x}$是整个数据集的均值。

$\widetilde{x}^{(n)} = \sum_{i=1}^{M} \alpha_i^{(n)} \boldsymbol{u}_i$

$\alpha_i^{(n)} = \boldsymbol{u}_i^T \left(x^{(n)} - \overline{x} \right)$

我们的目标即使最小化整个数据集的平均重建误差：

$$E=\frac{1}{N}\sum_{n=1}^{N}\|(x^{(n)}-\overline{x})-\widetilde{x}^{(n)}\|^2=\frac{1}{N}\sum_{n=1}^{N}\|(x^{(n)}-\overline{x})-\sum_{i=1}^{M}\alpha_i^{(n)}u_i\|^2$$

展开二次项,之后代入$\alpha_i^{(n)}$

$E=$
$\frac{1}{N}\left(\sum_{n=1}^{N}\left\|\mathbf{x}^{(n)}-\overline{\mathbf{x}}\right\|^2-2\sum_{n=1}^{N}\sum_{i=1}^{M}\alpha_i^{(n)}\left(\mathbf{x}^{(n)}-\overline{\mathbf{x}}\right)^T\mathbf{u}_i+\sum_{n=1}^{N}\sum_{i=1}^{M}\left(\alpha\right.\right.$
$=\frac{1}{N}\Sigma_{n=1}^{N}\|\boldsymbol{x}^{(n)}-\overline{\boldsymbol{x}}\|^2-\sum_{i=1}^{M}\boldsymbol{u}_i^T\frac{1}{N}\Sigma_{n=1}^{N}\left(\boldsymbol{x}^{(n)}-\overline{\boldsymbol{x}}\right)\left(\boldsymbol{x}^{(n)}-\overline{\boldsymbol{x}}\right)^T\boldsymbol{u}_i$

注意到协方差矩阵$\mathbf{S}=\frac{1}{N}\mathbf{X_c}{\mathbf{X_c}}^T$ 把前面写成矩阵$\mathbf{X}=[x^{(1)},x^{(2)},\ldots,x^{(N)}]$

$$E=\frac{1}{N}\|X-\overline{X}\|_F^2-\frac{1}{N}\sum_{i=1}^{M}\boldsymbol{u}_i^T\mathbf{S}\boldsymbol{u}_i$$

可以证明，与其(最小化误差)等价的约束问题：

$$\max_{\boldsymbol{u}_i}\sum_{i=1}^{M}\boldsymbol{u}_i^T\boldsymbol{S}\boldsymbol{u}_i\quad\text{s.t.}\boldsymbol{u}_i^T\boldsymbol{u}_i=\delta_{i,j}$$

解是$\boldsymbol{S}$从大到小排序前M个特征向量。由于$S\in\mathbb{R}^{D\times D}$是实对称矩阵，所以其一定存在$D$个相互正交的特征向
量。

角度二——最大化方差

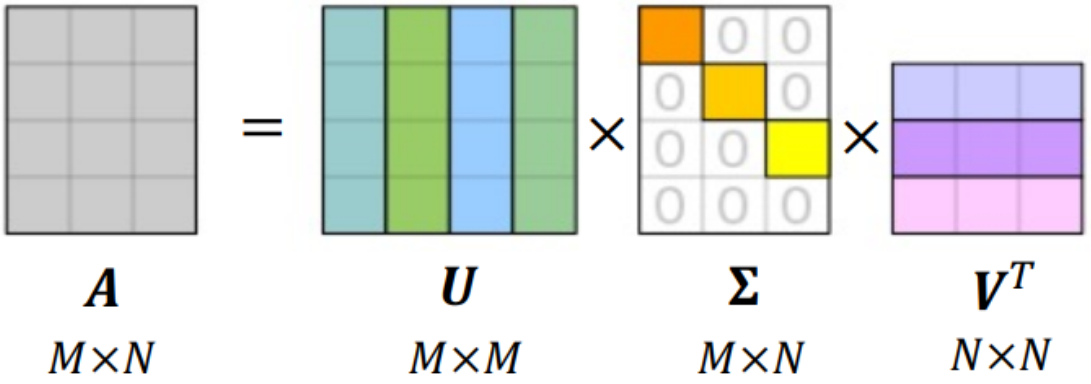
另一个角度是最大化重建数据的方差。可以想象，把面上数据投影到一条线上，如果所有的数据都集中**在线上某个点附近**，则几乎无法区分数据，携带的信息**很少**；如果数据比较**松散地分布在较大的范围内**，容易区分出每个点，携带的信息就比**较多。**

由方差的定义，可以写出在某个方向u_j上的整个数据集方差：

$$
\begin{aligned}
\text{var} &= \frac{1}{N}\sum_{n=1}^{N}\left(\boldsymbol{u}_j^T(\boldsymbol{x}^{(n)}-\overline{\boldsymbol{x}})\right)^2 \\
&= \boldsymbol{u}_j^T\frac{1}{N}\sum_{n=1}^{N}(\boldsymbol{x}^{(n)}-\overline{\boldsymbol{x}})(\boldsymbol{x}^{(n)}-\overline{\boldsymbol{x}})^T\boldsymbol{u}_j \\
&= \boldsymbol{u}_j^T\boldsymbol{S}\boldsymbol{u}_1
\end{aligned}
$$

这与我们在上一个角度得到的形式是一样的。

角度三——奇异值分解



一个矩阵$A \in \mathbb{R}^M \times N$可以做奇异值分解：

$$
A = U\Sigma V^T
$$

其中$U = [u_1, u_2, \cdots, u_M] \in \mathbb{R}^{M\times M}$和$V = [v_1, v_2, \cdots, v_N] \in \mathbb{R}^{N\times N}$是正交矩阵。$u_i$和$v_i$分别是$AA^T$和
$A^T A$的特征向量。Σ是一个对角矩阵，**对角线上的元素是奇异值(**奇异值 σ_i 是特征值的平方根**)**。

由于Σ只有对角线上有非零值，原矩阵A可以表达为：

$$
A = \sum_{i=1}^{r}\Sigma_{i,i}\boldsymbol{u}_i\boldsymbol{v}_i^T = \boldsymbol{U}'\boldsymbol{\Sigma}'\boldsymbol{V}'
$$

矩阵A的SVD分解中的向量u_i是矩阵AA^T关于其第i大特征值的特征向量。

令$A=\widetilde{X}$，其中$\widetilde{X} \triangleq [x^{(1)}-\overline{x}, x^{(2)}-\overline{x}, \cdots, x^{(N)}-\overline{x}]$，我们可以看到

$$\widetilde{X}\widetilde{X}^T=\sum_{n=1}^{N}\left(x^{(n)}-\overline{x}\right)\left(x^{(n)}-\overline{x}\right)^T=N\cdot S$$

> 样本数量N;矩阵S表示的是数据的协方差矩阵$S=\frac{1}{N}\sum_{n=1}^{N}(x^{(n)}-\overline{x})(x^{(n)}-\overline{x})^T$

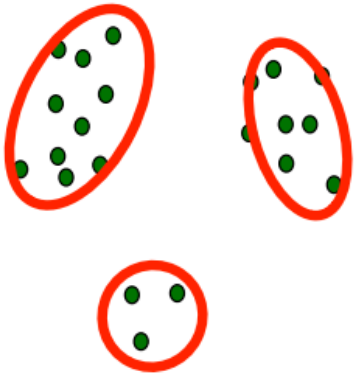
这意味着$\widetilde{X}\widetilde{X}^T$的特征向量与矩阵$S$的特征向量相同。

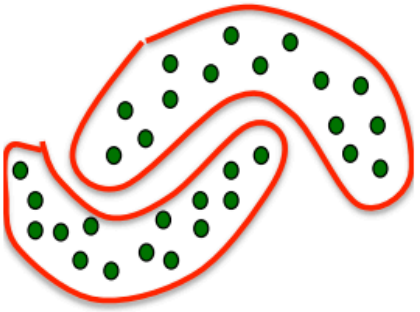
通过对$\widetilde{X}$进行SVD分解，可以直接获得数据$\{x^{(n)}\}_{n=1}^{N}$的主方向，这些主方向就是SVD分解中左矩阵U的列向量。

Kmeans-聚类

给定一组数据$\{x_i\}$,聚类就是将数据分成若干个簇：

簇内的数据点**尽可能相似**，簇间的数据点**尽可能不同。**





K-means 是一种常见的聚类算法，流程如下：

1.假如要将数据分成K个簇，首先生成K个**初始中心点** μ_k,每个中心点代表一个簇
2.对于每一个数据点，分配到最近的中心点所代表的簇。
若用$r_{i,k}\in\{0,1\}$表示x_i**是否属于**第k个簇，则有：

$$r_{i,k} = \begin{cases} 1, & \text{if } k = \arg\min_j \|\boldsymbol{x}_i - \boldsymbol{\mu}_j\|^2 \\ 0, & \text{otherwise} \end{cases}$$

3.更新中心点，使得每个簇的中心点是簇内所有数据点的平均值：

$$\mu_k = \frac{\sum_{i=1}^{N} r_{i,k} \cdot \boldsymbol{x}_i}{\sum_{i=1}^{N} r_{i,k}}$$

判断K-means 是否收敛

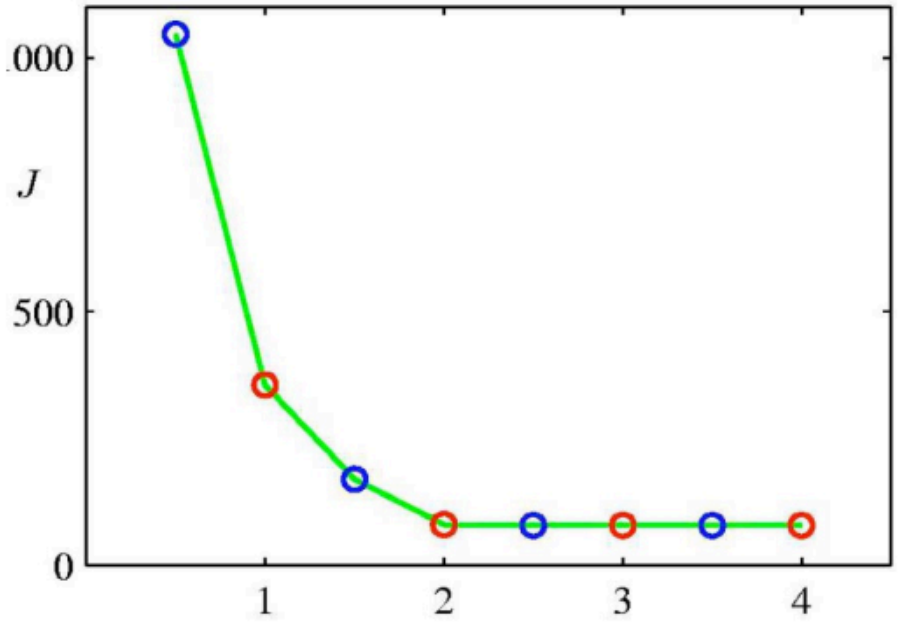
可以定义如下目标函数：

$$J = \sum_{i=1}^{N} \sum_{k=1}^{K} r_{i,k} \|\boldsymbol{x}_i - \boldsymbol{\mu}_k\|^2$$

这个目标函数的含义是，每个数据点到其所属簇中心的距离之和。K-means 就相当于在最小化这个目标函数。易知
J是单调递减的，因为每次迭代都会使得J减小。所以 K-means 一定会收敛。

易知，随着K增大，J总是减小的。一般，可以用**肘部法则**来确定最佳的K值：

可以通过寻找曲线的“肘部点”来确定 K 的最佳值。在这个例子中，$K = 2$ 是肘部点，即在这个点之后，J 的减少速度明显减缓，增加聚类数对结果的改进有限。



选择 K 的两种方法：

- 方法1（肘部法则）：通过观察肘部点确定 K，如图所示 $K=2$ 是肘部点。
- 方法2：根据下游任务的性能来决定最佳的 K。比如，使用不同的 K 值进行聚类后，测试其对后续任务的影响，从而选择一个在应用中效果最优的 K。

K-means 聚类的效果还与**初始中心点高度相关**。

几种初始化方法：

- 随机初始化。这种方法可能导致**初始生成邻近**的中心点，使得最终聚类效果**不好**
- 基于距离的初始化。第一个中心点随机生成，接下来选择**离现存中心点最远**的数据点作为下一个中心点。这种
  方法的问题是可能选择到一些离群点。
- 随机+基于距离的初始化。先随机生成一个中心点，然后基于距离选择下一个中心点，剩下的距离离目前的中
  心点**越远**，被选中的**概率越大**。

隐变量模型

引入

在第三讲中我们提到，分类和回归这种有监督学习都可以看成是在学习某个条件概率分布$p(y|x;w)$。

无监督学习也可以看成是在学习某个概率分布$p(\boldsymbol{x};\boldsymbol{w})$。

由于没有标签y,对x建模要困难许多。

一种最简单的想法是将x的分布限定为某个高斯分布$\mathcal{N}(\mu,\boldsymbol{\Sigma})$

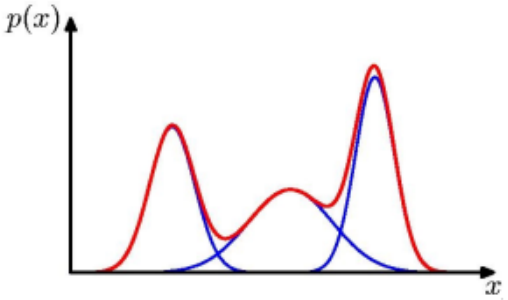
- μ 是数据的均值，表示数据的中心位置。
- Σ 是数据的协方差矩阵，表示数据的分布形状和方向。

通过优化 μ 和 Σ，使得高斯分布能够最好地拟合数据点 $\left\{x^{(n)}\right\}_{n=1}^{N}$。

设想引入一个简单的分类分布$p(z)=Cat(K,\pi)$,满足$p(z_i)=\pi_i,i=1,2,\cdots,K$。我们将其与一个高斯分布组合起来则我们可以获得一个更复杂的分布：

> **Cat(K, π)** 是一种描述离散随机变量的分布，其中 K 是类别数，π 是每个类别的概率向量。
>
> 例子,有一个骰子，$K=6$（骰子有 6 个面），且骰子是公平的，则 $\pi=\left(\frac{1}{6},\frac{1}{6},\frac{1}{6},\frac{1}{6},\frac{1}{6},\frac{1}{6}\right)$。此时，分类分布描述了骰子每个面朝上的概率。

$$p(\boldsymbol{x})=\sum_{i=1}^{K}p(\boldsymbol{x}|\boldsymbol{z}_i)p(\boldsymbol{z}_i)=\sum_{i=1}^{K}\boldsymbol{\pi}_i\mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_i,\boldsymbol{\Sigma}_i)$$



这里的$p(z)$可以代表数据隐藏的结构。一般地，隐变量模型是带有隐变量的概率模型：

$$p(\boldsymbol{x},z)$$

其中z就称为隐变量。隐变量也可以有多个，如$p(\boldsymbol{x}, \boldsymbol{z}_1, \boldsymbol{z}_2, \cdots, \boldsymbol{z}_K)$。

相应的，关于x的概率$p(x)$是一个边缘概率，可以写为：

$$p(\boldsymbol{x}) = \int_z p(\boldsymbol{x}, \boldsymbol{z}) d\boldsymbol{z}$$

高斯隐变量模型

若隐变量的分布是一个高斯分布：

$$p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{0}, I)$$

引入参数W写出条件概率$p(x|z)$,假定其也是一个高斯分布：

$$p(\boldsymbol{x}|\boldsymbol{z}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{W}\boldsymbol{z} + \boldsymbol{\mu}, \sigma^2\boldsymbol{I})$$

我们可以通过最大化其(对数)似然概率：

$$\log p(\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_N) = \log \prod_{i=1}^N p(\boldsymbol{x}_i) = \sum_{i=1}^N \log p(\boldsymbol{x}_i)$$

来得到能最好拟合真实数据的参数。

> 在两者都是高斯分布的情况下，目标函数可以有一个解析解。首先，由于：
>
> $$p(\boldsymbol{x}_i) = p(\boldsymbol{x}_i|\boldsymbol{z}_i) \cdot p(\boldsymbol{z}_i) = \mathcal{N}(\boldsymbol{x}_i|\boldsymbol{W}\boldsymbol{z}_i + \boldsymbol{\mu}, \sigma^2\boldsymbol{I})\mathcal{N}(\boldsymbol{z}_i|\boldsymbol{0}, \boldsymbol{I})$$
>
> 则：
>
> $$\boldsymbol{x}_i = \boldsymbol{\mu} + W\boldsymbol{z}_i + \boldsymbol{\epsilon}_i$$
>
> 其中$\epsilon_i \sim \mathcal{N}(0, \sigma^2\boldsymbol{I})$。因为高斯分布的线性组合仍然是高斯分布，所以$x_i$也是服从高斯分布的。
>
> 这个多维高斯分布的期望为：

$$E[\boldsymbol{x}_i] = \boldsymbol{\mu} + \boldsymbol{W}\mathbb{E}[\boldsymbol{z}_i] + \mathbb{E}[\boldsymbol{\epsilon}_i] = \boldsymbol{\mu}$$

协方差为：

$$\mathrm{Cov}[\boldsymbol{x}_i] = \mathbb{E}[(\boldsymbol{x}_i - \boldsymbol{\mu})(\boldsymbol{x}_i - \boldsymbol{\mu})^T] = WW^T + \sigma^2\boldsymbol{I}$$

所以有：

$$p(\boldsymbol{x}_i) = \mathcal{N}(\boldsymbol{x}_i|\boldsymbol{\mu}, \boldsymbol{W}\boldsymbol{W}^T + \sigma^2\boldsymbol{I})$$

回头看对数似然概率，可以写成：

$$\log p(\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_N) = \log \prod_{i=1}^{N} p(\boldsymbol{x}_i) = \sum_{i=1}^{N} \log \mathcal{N}(\boldsymbol{x}_i|\boldsymbol{\mu}, \boldsymbol{W}\boldsymbol{W}^T + \sigma^2\boldsymbol{I}) =$$
$$-\frac{1}{2}\left[ND\log 2\pi + N\log det(\boldsymbol{W}\boldsymbol{W}^T + \sigma^2\boldsymbol{I}) + \sum_{i=1}^{N}(\boldsymbol{x}_i - \boldsymbol{\mu})^T(\boldsymbol{W}\boldsymbol{W}^T + \sigma^2\boldsymbol{I}$$

记$\boldsymbol{\Sigma} = \boldsymbol{W}\boldsymbol{W}^T + \sigma^2\boldsymbol{I}, \boldsymbol{S} = \sum_{i=1}^{N}(\boldsymbol{x}_i - \boldsymbol{\mu})(\boldsymbol{x}_i - \boldsymbol{\mu})^T$,则：

$$\frac{\partial \log p(\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_N)}{\partial \boldsymbol{\mu}} = 0 \Rightarrow \sum_{i=1}^{N}(\boldsymbol{x}_i - \boldsymbol{\mu}) = 0 \Rightarrow \boldsymbol{\mu} = \frac{1}{N}\sum_{i=1}^{N}\boldsymbol{x}_i$$

$$\begin{aligned}\frac{\partial \log p(\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N)}{\partial \boldsymbol{\Sigma}} &= -\frac{N}{2}\Sigma^{-1} + \frac{1}{2}\sum_{n=1}^{N}\Sigma^{-1}(x_n - \mu_n)(x_n - \mu_n)^T\Sigma^{-1} \\ &= -\frac{N}{2}\boldsymbol{\Sigma}^{-1} + \frac{N}{2}\boldsymbol{\Sigma}^{-1}\boldsymbol{S}\boldsymbol{\Sigma}^{-1} = 0 \\ &\Rightarrow \boldsymbol{\Sigma} = \boldsymbol{S}\end{aligned}$$

综上，在最大化似然概率时，易知$x_i \sim \mathcal{N}(\mu, \Sigma) = \mathcal{N}(\overline{x}, S)$。

进一步可求出$W = U\sqrt{\Lambda - \sigma^2 I}$,其中

- U由S的前M个特征向量组成

- Λ是S的前M个特征值组成的对角矩阵

所以W就可以视作没有经过正则化的数据x的主成分，其中每i个成分被放大了$\sqrt{\lambda_i - \sigma^2}$倍

所以高斯隐变量模型也被称为概率 PCA。

高斯混合模型

在高斯混合模型中，我们假设数据是由多个高斯分布混合而成的：

$$p(\boldsymbol{x}) = \sum_{i=1}^{K} \pi_i \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$$

其中π_i是权重，满足$\sum_i^K \pi_i = 1$。

高斯混合模型可以写作隐变量模型的形式。引入隐变量z,且z是一个独热编码，记z_i表示第i个元素为 1,其他元素
为0的向量。

隐变量的分布写为$p(z)$,满足$p(\boldsymbol{z}_i) = \pi_i$。条件概率$p(\boldsymbol{x}|\boldsymbol{z})$写为$p(\boldsymbol{x}|\boldsymbol{z}_i) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$。这就得到了高斯混合
模型的隐变量模型形式。

通过最大化似然概率，可以得到最优参数$\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i, \pi_i$。

有了x的分布之后，可以计算后验概率：

$$p(\boldsymbol{z}_i|\boldsymbol{x}) = \frac{p(\boldsymbol{x}|\boldsymbol{z}_i)p(\boldsymbol{z}_i)}{p(\boldsymbol{x})} = \frac{\pi_i \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

这个后验概率的含义就是，数据点x属于第i个高斯分布的概率，相当于在做聚类。

LVM (隐变量模型)还运用在隐式马尔科夫模型、Sigmoid 信念网络 (SBN)、隐式狄利克雷分布 (LDA)等模型中。

EM算法

Expectation Maximization Algorithm, 期望最大化算法。

EM 算法是用于解决下面的问题：假如有联合分布$p(x, z; \theta)$,我们需要最大化对数似然概率(一个边缘概率)
$p(\boldsymbol{x}; \boldsymbol{\theta})$

$p(x; \theta) = \sum_z p(x, z; \theta)$

$$\theta^* = \arg\max_\theta \log p(\boldsymbol{x}; \boldsymbol{\theta})$$

EM 算法是一种迭代算法，每次迭代分两步：

1. E-step: 计算下面这个期望：

$$Q(\boldsymbol{\theta}; \boldsymbol{\theta}^{(t)}) = \mathbb{E}_{\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)}}[\log p(\boldsymbol{x}, \boldsymbol{z}; \boldsymbol{\theta})]$$

这实际上是在计算在给定数据x和参数$\theta^{(t)}$的情况下，隐变量z的期望对数似然概率
2.M-step:更新参数是这个期望最大化：

$$\theta^{(t+1)} = \arg\max_\theta Q(\boldsymbol{\theta}; \boldsymbol{\theta}^{(t)})$$

理论推导

将对数似然概率重写。

> x和θ相互独立

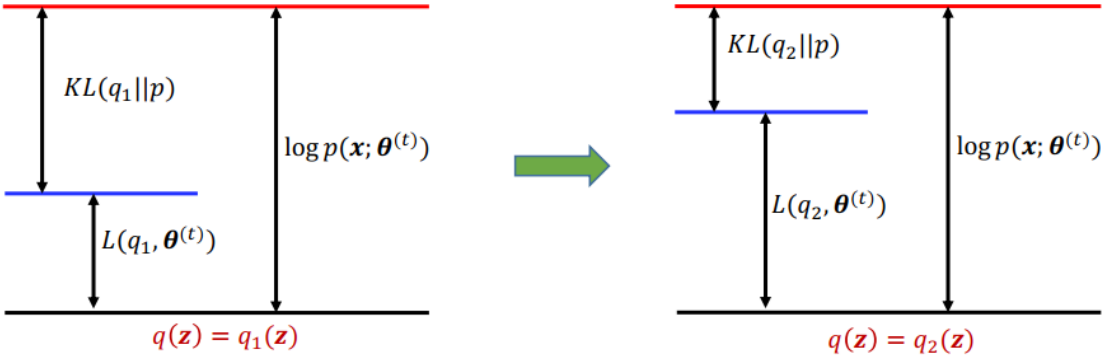
假定q(z)是任意的分布，有：

$\log p(x; \theta) = \sum_Z q(z) \log p(x) = \sum_z q(\boldsymbol{z}) \log p(x; \theta) =$
$\sum_Z q(z) \log \frac{p(x, z; \theta) q(z)}{p(z|x; \theta) q(z)}$
$= \sum_{\mathbf{z}} q(\mathbf{z}) \log \frac{p(\mathbf{x}, \mathbf{z}; \theta)}{q(\mathbf{z})} + \sum_{\mathbf{z}} q(\mathbf{z}) \log \frac{q(\mathbf{z})}{p(\mathbf{z}|\mathbf{x}; \theta)}$
$= \mathcal{L}(q, \theta) + KL(q||p(z|x; \theta))$

后一项我们称之为 KL 散度，它用于衡量两个分布的相似程度。

一般的定义为：$\mathrm{KL}(q\|p)=\int q(\boldsymbol{z})\log\frac{q(\boldsymbol{z})}{p(\boldsymbol{z})}d\boldsymbol{z}\geqslant 0$

下面这张图可以直观地解释 KL 散度：不同的q(z)有不同的分解

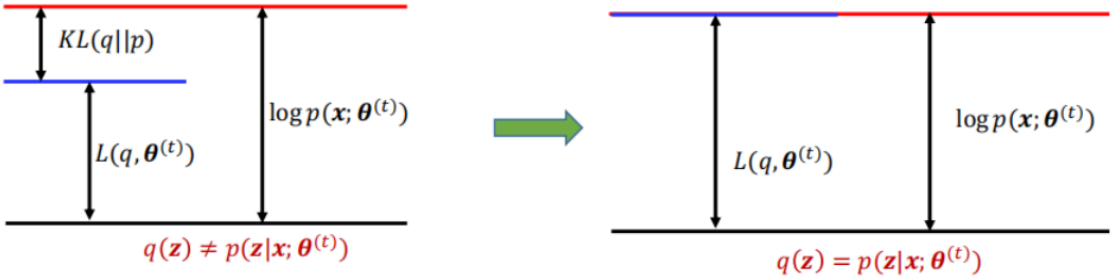


$\log p(x;\theta^{(t)})=\sum_{\mathbf{z}}q(\mathbf{z})\log\frac{p\left(x,\mathbf{z};\boldsymbol{\theta}^{(t)}\right)}{q(\mathbf{z})}+\sum_{\mathbf{z}}q(\mathbf{z})\log\frac{q(\mathbf{z})}{p(\mathbf{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)})}$

若取$q(\boldsymbol{z})=p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)})$,则$KL(q||p(\mathbf{z}|\mathbf{x};\boldsymbol{\theta}^{(t)}))=0$

因此,参数为$\boldsymbol{\theta}^{(t)}$时的对数似然概率为:

$$\log p(\boldsymbol{x};\boldsymbol{\theta}^{(t)})=\mathcal{L}(p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)}),\boldsymbol{\theta}^{(t)})=\sum_{z}p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)})\log\frac{p(\boldsymbol{x},\boldsymbol{z};\boldsymbol{\theta}^{(t)})}{p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)})}$$



如果我们更新参数:

$$\boldsymbol{\theta}^{(t+1)}=\arg\max_{\theta}\mathcal{L}(p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)}),\boldsymbol{\theta})$$

因为θ是最大化$\mathcal{L}$得到的参数,则一定有更新后的$\mathcal{L}$更大:

$$\mathcal{L}(p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)}),\boldsymbol{\theta}^{(t+1)}) \geqslant \mathcal{L}(p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)}),\boldsymbol{\theta}^{(t)}) = \log p(\boldsymbol{x};\boldsymbol{\theta}^{(t)})$$

我们再取$q(\boldsymbol{z}) = p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)})$,计算参数为$\boldsymbol{\theta}^{(t+1)}$时的对数似然概率:

$$\log p(\boldsymbol{x};\boldsymbol{\theta}^{(t+1)}) = \mathcal{L}(p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)}),\boldsymbol{\theta}^{(t+1)}) + \mathrm{KL}(p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)})\|p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t+1)})) \geqslant$$

> kl散度总是是非负的

所以按照上面更新参数的方法，总有:

$$\log p(\boldsymbol{x};\boldsymbol{\theta}^{(t+1)}) \geqslant \log p(\boldsymbol{x};\boldsymbol{\theta}^{(t)})$$

$\arg\max_\theta \mathcal{L}(p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)}),\boldsymbol{\theta})$能保证每一步的似然性都能增加

接下来，只需证明 EM 算法和上述过程等价。因为:

$$\mathcal{L}(q,\theta) = \sum_z q(z)\log\frac{p(\boldsymbol{x},\boldsymbol{z};\boldsymbol{\theta})}{q(\boldsymbol{z})} = \sum_z q(\boldsymbol{z})\log p(\boldsymbol{x},\boldsymbol{z};\boldsymbol{\theta}) - \sum_z q(\boldsymbol{z})\log q(\boldsymbol{z})$$

后半部分其实就是$q(z)$的熵，可以忽略。而前半部分正是 E-step 中的期望。所以 EM 算法和上述过程等价。

> “上述过程”指的是在 EM 算法中通过选择 $q(\mathbf{z}) = p(\mathbf{z}|\mathbf{x};\boldsymbol{\theta}^{(t)})$ 并最大化 $\mathcal{L}(q,\theta)$ 来更新参数 $\boldsymbol{\theta}$ 的过程。具体来说，该过程包括以下步骤:
>
> 1. **选择** $q(\mathbf{z}) = p(\mathbf{z}|\mathbf{x};\boldsymbol{\theta}^{(t)})$：在第 t 轮迭代中，选择隐变量 $\mathbf{z}$ 的分布为当前参数 $\boldsymbol{\theta}^{(t)}$ 下的后验分布。
> 2. **最大化** $\mathcal{L}(q,\theta)$ **来更新参数**：通过最大化期望对数似然函数 $\mathcal{L}(q,\theta)$ 来得到新的参数 $\boldsymbol{\theta}^{(t+1)}$。
>
> 这个过程确保了每一步对数似然概率 $\log p(\mathbf{x};\boldsymbol{\theta})$ 都会增加，并且与 EM 算法的 E 步和 M 步相对应:

- **E 步**：计算期望 $\mathcal{L}(p(\mathbf{z}|\mathbf{x};\boldsymbol{\theta}^{(t)}),\boldsymbol{\theta})$。
- **M 步**：最大化这个期望来得到新的参数 $\boldsymbol{\theta}^{(t+1)}$。

例子-训练GMM

GMM 可以利用 EM 算法来训练。

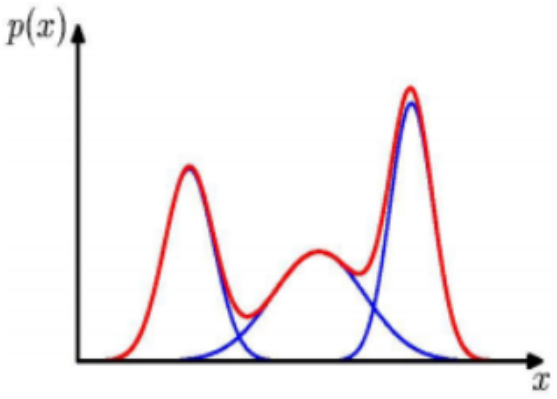
高斯混合分布$p(x)=\sum_{k=1}^{K}\pi_k\mathcal{N}(x;\mu_k,\Sigma_k)$

联合分布 $p(\boldsymbol{x},\boldsymbol{z})$ 引入了潜在变量 $\boldsymbol{z}=[z_1,z_2,\cdots,z_K]$，其中 z_k 是一个二元指示变量，如果数据点 $\boldsymbol{x}$ 属于第 k 个成分，则 $z_k=1$，否则为 0。联合分布表示为：

$$p(\boldsymbol{x},\boldsymbol{z})=p(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})=\prod_{k=1}^{K}[\pi_k N(\boldsymbol{x};\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k)]^{z_k}$$

这里，$p(\boldsymbol{z})$ 是 $\boldsymbol{z}$ 上的分类分布，参数为 $\pi=[\pi_1,\pi_2,\cdots,\pi_K]$，即：

$$p(\boldsymbol{z})=\prod_{k=1}^{K}\pi_k^{z_k}$$



E-step

$$p(\boldsymbol{z}_j|\boldsymbol{x}_i;\boldsymbol{\theta}^{(t)})=\frac{p(x,z_j;\theta^{(t)})}{\sum_{i=1}^{K}p(x,z_j;\theta^{(t)})}=\frac{\mathcal{N}(\boldsymbol{x}_i|\boldsymbol{\mu}_j^{(t)},\boldsymbol{\Sigma}_j^{(t)})\pi_j^{(t)}}{\sum_{k=1}^{K}\mathcal{N}(\boldsymbol{x}_i|\boldsymbol{\mu}_k^{(t)},\boldsymbol{\Sigma}_k^{(t)})\pi_k^{(t)}}=\gamma_{i,j}^{(t)}$$

所以：

$$\log p(\boldsymbol{x}_i, \boldsymbol{z}; \boldsymbol{\theta}^{(t)}) = \sum_{j=1}^{K} [\log \mathcal{N}(\boldsymbol{x}_i | \boldsymbol{\mu}_j^{(t)}, \boldsymbol{\Sigma}_j^{(t)}) + \log \pi_j^{(t)}]$$

$$\mathbb{E}_{p(z|x;\theta^{(t)})}[\log p(x, z; \theta)] = \sum_{k=1}^{K} \mathbb{E}_{p(z|x;\theta^{(t)})}[z_k][\log \mathcal{N}(x_n; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) + \log \pi_k]$$

所以对于单个数据点x_i,期望为:

$$Q\left(\boldsymbol{\theta}; \boldsymbol{\theta}^{(t)}\right) = \frac{1}{N} \sum_{n=1}^{N} \mathbb{E}_{p(\mathbf{z}^{(n)}|\boldsymbol{x}^{(n)};\boldsymbol{\theta}^{(t)})} \left[\log p(\boldsymbol{x}^{(n)}, \boldsymbol{z}^{(n)}; \boldsymbol{\theta})\right] = \sum_{j=1}^{K} \gamma_{i,j}^{(t)} [\log \mathcal{N}(\boldsymbol{x}_i$$

将高斯分布也展开:

$$Q(\boldsymbol{\theta}; \boldsymbol{\theta}^{(t)}) = \sum_{j=1}^{K} \gamma_{i,j}^{(t)} \left[-\frac{1}{2} \log |\boldsymbol{\Sigma}_j| - \frac{1}{2} (\boldsymbol{x}_i - \boldsymbol{\mu}_j)^T \boldsymbol{\Sigma}_j^{-1} (\boldsymbol{x}_i - \boldsymbol{\mu}_j) + \log \pi_j\right]$$

考虑所有数据，平均期望就是:

$$Q(\boldsymbol{\theta}; \boldsymbol{\theta}^{(t)}) = \frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{K} \gamma_{i,j}^{(t)} \left[-\frac{1}{2} \log |\boldsymbol{\Sigma}_j| - \frac{1}{2} (\boldsymbol{x}_i - \boldsymbol{\mu}_j)^T \boldsymbol{\Sigma}_j^{-1} (\boldsymbol{x}_i - \boldsymbol{\mu}_j) + \text{lo}$$

M-step

上面已经求出了期望的式子，分别对$\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j$求偏导并令其为 0,可以得到

$$\boldsymbol{\mu}_j^{(t+1)} = \frac{1}{N_j} \sum_{i=1}^{N} \gamma_{i,j}^{(t)} \boldsymbol{x}_i$$

$$\boldsymbol{\Sigma}_j^{(t+1)} = \frac{1}{N_j} \sum_{i=1}^{N} \gamma_{i,j}^{(t)} (\boldsymbol{x}_i - \boldsymbol{\mu}_j^{(t+1)})(\boldsymbol{x}_i - \boldsymbol{\mu}_j^{(t+1)})^T$$

其中$N_j = \sum_{i=1}^{N} \gamma_{i,j}^{(t)}$,可以理解为所有数据点分到第$j$类的概率之和。

对于π_j,在约束$\sum_j = 1^K \pi_j = 1$下，可以求出解：

$$\pi_j^{(t+1)} = \frac{N_j}{N}$$

总结一下，如果第t轮的参数为$\mu^{(t)}, \mathbf{\Sigma}^{(t)}, \pi^{(t)}$,首先利用其计算$\gamma_{i,j}^{(t)}$,然后根据上面的公式更新参数
$\mu^{(t+1)}, \mathbf{\Sigma}^{(t+1)}, \pi^{(t+1)}$。重复这个过程直到收敛。

如果将$\mathbf{\Sigma}_k$限定为对角矩阵$\sigma^2 I$的 形 式 , 且 固 定 $\pi_k = \frac{1}{K}$,则 GMM 就退化为$\beta = \frac{1}{2\sigma^2}$的软 K-means。

总结

基于当前估计 $\{\boldsymbol{\mu}_k, \mathbf{\Sigma}_k, \pi_k\}_{k=1}^K$, 更新$\gamma_{nk}$

$$\gamma_{nk} \leftarrow \frac{\mathcal{N}(x^{(n)}; \boldsymbol{\mu}_k, \mathbf{\Sigma}_k)\pi_k}{\sum_{i=1}^K \mathcal{N}(\boldsymbol{x}^{(n)}; \boldsymbol{\mu}_i, \mathbf{\Sigma}_i)\pi_i}$$

给定 γ_{nk}, 更新 μ_k, Σ_k and π_k

$$N_k \leftarrow \sum_{n=1}^N \gamma_{nk}$$

$$\mu_k \leftarrow \frac{1}{N_k} \Sigma_{n=1}^N \gamma_{nk} \boldsymbol{x}^{(n)}$$

$$\mathbf{\Sigma}_k \leftarrow \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (\boldsymbol{x}^{(n)} - \boldsymbol{\mu}_k)(\boldsymbol{x}^{(n)} - \boldsymbol{\mu}_k)^T$$

$$\pi_k \leftarrow \frac{N_k}{N}$$

EM变体

上面我们讨论了 EM 在训练 GMM 中的应用，实际问题中的后验分布不总是高斯分布，期望和参数更新都不一定有闭式解。

EM+SGD

推导 EM 算法的过程中，我们发现最大化期望，只是在保证似然概率不减。事实上，保证似然概率不减，不一定要最大化期望。

对期望利用梯度上升法来更新参数：

$$\boldsymbol{\theta}^{(t+1)}=\boldsymbol{\theta}^{(t)}+\gamma\cdot\frac{\partial Q\left(\boldsymbol{\theta};\boldsymbol{\theta}^{(t)}\right)}{\partial\boldsymbol{\theta}}|_{\boldsymbol{\theta}=\boldsymbol{\theta}^{(t)}}$$

显然有：

$$Q(\boldsymbol{\theta};\boldsymbol{\theta}^{(t)})\geqslant Q(\boldsymbol{\theta}^{(t+1)};\boldsymbol{\theta}^{(t)})$$

由于$p(z)$的熵可以认为是不变的，所以根据 EM 算法的推导过程可以反向推导出，仍然有：

$$\log p(\boldsymbol{x};\boldsymbol{\theta}^{(t+1)})\geqslant\log p(\boldsymbol{x};\boldsymbol{\theta}^{(t)})$$

MCMC EM

Markov Chain Monte Carlo EM。

其中 MCMC 是一种采样方法，由于我们并不总是知道后验分布的形式，所以可以通过 MCMC 来近似采样。

所以期望可以用这些采样值来近似：

$$Q(\boldsymbol{\theta};\boldsymbol{\theta}^{(t)})\approx\frac{1}{S}\sum_{s=1}^{S}\log p(\boldsymbol{x},\boldsymbol{z}_s;\boldsymbol{\theta})$$

再结合 SGD,就得到了 MCMC EM, 这是常用于训练隐变量模型的方法。

VB EM

Variational Bayes EM,变分贝叶斯 EM。

直接在高维空间中采样出z_s代价比较大。VB EM 的基本思想是用一个简单的分布$q(z;\phi)$来近似$p(z|x;\theta)$,然后在这个简单的分布上进行 EM 算法。

这个简单的分布，可以是每个维度用一个高斯分布的组合：

$$z_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$$

然后找到能最小化与原后验分布 KL 散度的参数ϕ。

将最小化 KL 散度进行转化：

$$\min_{\phi} \mathrm{KL}(q(\boldsymbol{z};\boldsymbol{\phi})\|p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta})) \Leftrightarrow \max_{\phi} \int q(\boldsymbol{z};\boldsymbol{\phi}) \log \frac{p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta})}{q(\boldsymbol{z};\boldsymbol{\phi})} d\boldsymbol{z} \Leftrightarrow \max_{\phi} \int q($$

中间，我们舍去了$\log p(\boldsymbol{x};\boldsymbol{\theta})$,因为它与$\phi$无关。
后续利用这个近似分布最大化梯度可以写为：

$$\max_{\theta} \mathbb{E}_{q(\boldsymbol{z};\boldsymbol{\phi})}[\log p(\boldsymbol{x},\boldsymbol{z};\boldsymbol{\theta})] = \max_{\theta} \int q(\boldsymbol{z};\boldsymbol{\phi}) \log p(\boldsymbol{x},\boldsymbol{z};\boldsymbol{\theta}) d\boldsymbol{z} \Leftrightarrow \max_{\theta} \int q(\boldsymbol{z}$$

这里添加了与θ无关的项log $p(x;\theta)$,不影响最大化。
可以发现**最小化散度和最大化期望可以转化为相同的形式。**所以以上两个问题统一写为：

$$\max_{\theta,\phi} \int q(\boldsymbol{z};\boldsymbol{\phi}) \log \frac{p(\boldsymbol{x},\boldsymbol{z};\boldsymbol{\theta})}{q(\boldsymbol{z};\boldsymbol{\phi})} d\boldsymbol{z} = \max_{\boldsymbol{\theta},\phi} \int \mathcal{L}(\boldsymbol{x};\boldsymbol{\theta},\boldsymbol{\phi}) d\boldsymbol{z}$$

利用**梯度下降法**，同时更新θ, ϕ,这就是 VB EM。

可以证明，$\mathcal{L}(\boldsymbol{x};\boldsymbol{\theta},\boldsymbol{\phi})$是任意参数的对数似然概率的下界。VB EM 在最大化这个下界，也就是在最大化对数似然概率。

深度生成模型

数据的生成过程可以用一个联合概率分布函数来描述：

$$p(\boldsymbol{x}, \boldsymbol{z}) = p(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})$$

也即由隐变量z生成数据x。显然，这也可以看成是一个隐变量模型。这就是普通的生成模型(GM) 。

之前介绍的 PCA 是隐变量模型：

$$\begin{aligned} p(\boldsymbol{z}) &= \mathcal{N}(\boldsymbol{z}|\boldsymbol{0}, \boldsymbol{I}) \\ p(\boldsymbol{x}|\boldsymbol{z}) &= \mathcal{N}(\boldsymbol{x}|\boldsymbol{W}\boldsymbol{z} + \boldsymbol{\mu}, \sigma^2\boldsymbol{I}) \end{aligned}$$

GMM(高斯混合模型)属于生成模型 也是隐变量模型：

$$\begin{aligned} p(\boldsymbol{z}_i) &= \pi_i \\ p(\boldsymbol{x}|\boldsymbol{z}_i) &= \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) \end{aligned}$$

为了增强模型的表达能力，我们在$p(z)$和$p(x|z)$中引入深度神经网络，于是联合概率分布可以写为：

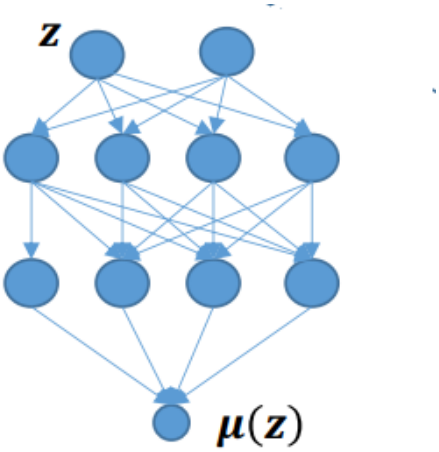
$$p(\boldsymbol{x}, \boldsymbol{z}) = p(\boldsymbol{x}|T(\boldsymbol{z}))p(\boldsymbol{z})$$

其中$T(\cdot)$就代表一个神经网络。这样，我们就得到了深度生成模型(DGM)。

例如在生成图片的时候，DGM 可以具体的写为：

$$p(\boldsymbol{x}, \boldsymbol{z}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}(\boldsymbol{z}), \boldsymbol{I})\mathcal{N}(\boldsymbol{z}|\boldsymbol{0}, \boldsymbol{I})$$

其中$\mu(z)$就是一个神经网络。如过神经网络的结构如下图：



则有：

$$\mu(\boldsymbol{z}) = \boldsymbol{W}_3 a(\boldsymbol{W}_2 a(\boldsymbol{W}_1 \boldsymbol{z} + \boldsymbol{b}_1) + \boldsymbol{b}_2) + \boldsymbol{b}_3$$

在PCA中，则有：

$$\mu(z) = Wz + b$$

后续，将神经网络涉及的参数简记一个符号θ,$\mu(z)$就可以写为$\mu(z;\theta)$。

VB EM 训练 DGM

我们知道 VB EM 的基本思想是用一个简单的分布$q(z;\phi)$来近似$p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta})$,然后在这个简单的分布上进行 EM 算法。
优化这个近似的分布和 E-step 中的期望可以统一的写为最大化下面这个式子：

$$\mathcal{L}(\boldsymbol{x};\boldsymbol{\theta},\phi) = \int q(\boldsymbol{z};\boldsymbol{\phi}) \log \frac{p(\boldsymbol{x},\boldsymbol{z};\boldsymbol{\theta})}{q(\boldsymbol{z};\boldsymbol{\phi})} d\boldsymbol{z}$$

进一步，由$p(x,\boldsymbol{z};\boldsymbol{\theta}) = p(\boldsymbol{x}|\boldsymbol{z};\boldsymbol{\theta})p(\boldsymbol{z})$,所以：

$$\begin{aligned}\mathcal{L}(\boldsymbol{x};\boldsymbol{\theta},\phi) &= \int q(\boldsymbol{z};\boldsymbol{\phi}) \log p(\boldsymbol{x}|\boldsymbol{z};\boldsymbol{\theta}) d\boldsymbol{z} - \int q(\boldsymbol{z};\boldsymbol{\phi}) \log \frac{q(\boldsymbol{z};\boldsymbol{\phi})}{p(\boldsymbol{z})} d\boldsymbol{z} \\ &= \mathbb{E}_{q(\boldsymbol{z};\boldsymbol{\phi})}[\log p(\boldsymbol{x}|\boldsymbol{z};\boldsymbol{\theta})] - \mathrm{KL}(q(\boldsymbol{z};\boldsymbol{\phi}) \| p(\boldsymbol{z}))\end{aligned}$$

接下来，考虑一个具体的例子：

$p(\boldsymbol{x}|\boldsymbol{z})$和$q(\boldsymbol{z})$都是高斯分布，且协方差矩阵是对角矩阵;$\boldsymbol{\mu}(\boldsymbol{z};\boldsymbol{\theta})$是参数为$\theta$的神经网络.$\phi$包括后验参数$\lambda, \eta$。

$p(\boldsymbol{x}|\boldsymbol{z};\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}(\boldsymbol{z};\boldsymbol{\theta}), \boldsymbol{I}), q(\boldsymbol{z};\phi) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{\lambda}, \mathrm{diag}(\boldsymbol{\eta}^2))$
要最大化$\mathcal{L}(x;\theta,\phi)$,考虑使以下两个偏导为 0:

$$\frac{\partial \mathcal{L}(\boldsymbol{x};\boldsymbol{\theta},\phi)}{\partial \theta} = 0, \frac{\partial \mathcal{L}(\boldsymbol{x};\boldsymbol{\theta},\phi)}{\partial \phi} = 0$$

对于$\mathcal{L}(x;\theta,\phi)$中散度的部分，由于$q(z;\phi)$和$p(z)$都是高斯分布，所以 这部分偏导为 0 有闭公式解
但是对于期望的部分，由于$\mu(z;\theta)$是一个神经网络，所以这部分的偏导没有闭式解。

重参数化技巧

在 MCMC EM 中提到，可以利用采样的方法来近似期望。在 DGM 中，我们同样可以这样做。但是，如果直接对
$q(z;\phi)$采样，这样采样出的只是单纯的数据点，并不包含对应的参数λ, η。
所以，我们可以利用重参数化技巧。先从一个标准正态分布$\mathcal{N}(0, I)$中采样出一个ϵ,然后做一个映射：

$$z = \lambda + \eta \odot \epsilon$$

有$z \sim \mathcal{N}(\boldsymbol{\lambda}, \mathrm{diag}(\boldsymbol{\eta}^2))$。所以$\boldsymbol{z}$就相当于从$q(z;\phi)$中采样出来的数据点，且显示带有参数$\lambda, \eta$。

由于$p(\boldsymbol{x}|\boldsymbol{z};\boldsymbol{\theta})$的协方差矩阵是对角矩阵，所以：

$$\log p(\boldsymbol{x}|\boldsymbol{z};\boldsymbol{\theta}) = C - \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{\mu}(\boldsymbol{z};\boldsymbol{\theta})\|^2 = C - \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{\mu}(\boldsymbol{\lambda} + \boldsymbol{\eta} \odot \boldsymbol{\epsilon};\boldsymbol{\theta})\|^2$$

于是，期望可以用采样数据来估计：

$$\mathbb{E}_{q(\boldsymbol{z};\boldsymbol{\phi})}[\log p(\boldsymbol{x}|\boldsymbol{z};\boldsymbol{\theta})] \approx \frac{1}{S}\sum_{s=1}^{S}[C - \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{\mu}(\boldsymbol{\lambda} + \boldsymbol{\eta} \odot \boldsymbol{\epsilon}_s;\boldsymbol{\theta})\|^2] = \tilde{L}(\boldsymbol{x};\boldsymbol{\theta},\boldsymbol{\phi}$$

这种形式下，期望对参数θ以及ϕ的偏导是可求的了。利用自动微分+反向传播，就能对这个期望的近似值进行优化。

在整个数据集上的训练

为了优化最终的目标，对于数据集上的每个数据点x_i,最好都要有一个对应的$q_i(\boldsymbol{z};\phi_i)$,也即有对应的参数$\phi_i = \{\lambda_i, \eta_i\}$
由于θ会在所有样本上更新，而ϕ是在单个样本上更新，所以后者的更新频率可能会更高。

但是为了保证能够有一个较好的后验分布近似$q_i(z;\phi_i)$,我们需要提高ϕ_i的更新频率。

我们引入两个深度神经网络$g(\cdot;\phi_1), g(\cdot;\phi_2)$,让参数$\lambda_i, \eta_i$由这两个网络输出，也即：

$$\lambda_i = g(\boldsymbol{x}_i;\phi_1), \boldsymbol{\eta}_i = g(\boldsymbol{x}_i;\phi_2)$$

所以只需要同时训练这两个网络就行，这两个网络就能给出所有数据点的λ_i, η_i。不仅更新频率提高了，且相比单独

训练每组参数，训练网络的效率更高。

最终，整个 DGM 模型的目标函数如下：

$$\mathcal{L} \approx \frac{1}{NK}\sum_{n=1}^{N}\sum_{k=1}^{K}\left(C - \frac{1}{2}\left\|\boldsymbol{x}_n - \boldsymbol{\mu}_{\boldsymbol{\theta}}\left(g_{\boldsymbol{\phi}_1}(\boldsymbol{x}_n) + g_{\boldsymbol{\phi}_2}(\boldsymbol{x}_n)\cdot\boldsymbol{\epsilon}_n^{(k)}\right)\right\|^2\right) - \frac{1}{N}\sum_{n=1}^{N} KL\left(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}_n)||p(\boldsymbol{z})\right)$$

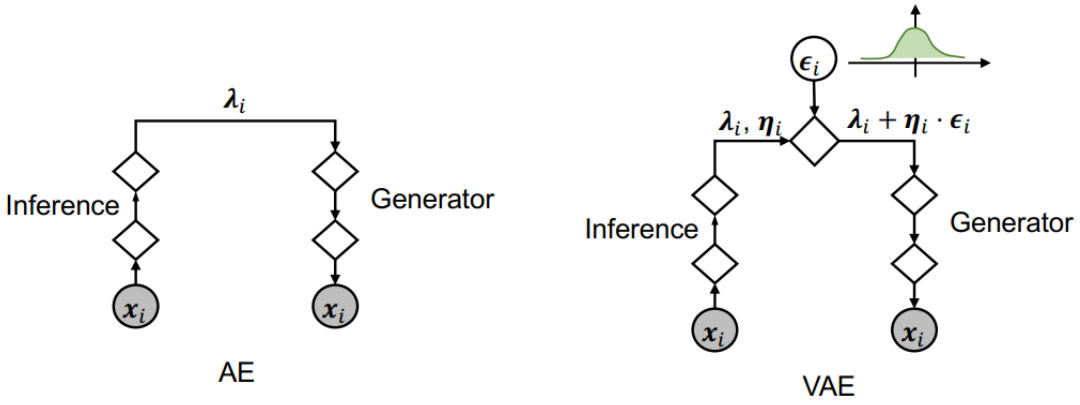
noise

decoder

encoder

网络$g(\cdot;\phi_1)$可以看作是编码器。由于ϵ是从标准正态分布中采样的，可以将$g(\cdot;\phi_2)\odot\epsilon$看作是一个高斯噪声。
由z到x的网络$\mu(z;\theta)$可以看作是解码器。我们还加入了一个 KL 散度作为正则项。

以上，其实就是所谓的变分自编码器(VAE):



相比普通的自编码器（AE），VAE 引入了一个正则项 KL 散度，同时还引入了高斯噪声。

表示学习

Representation Learning

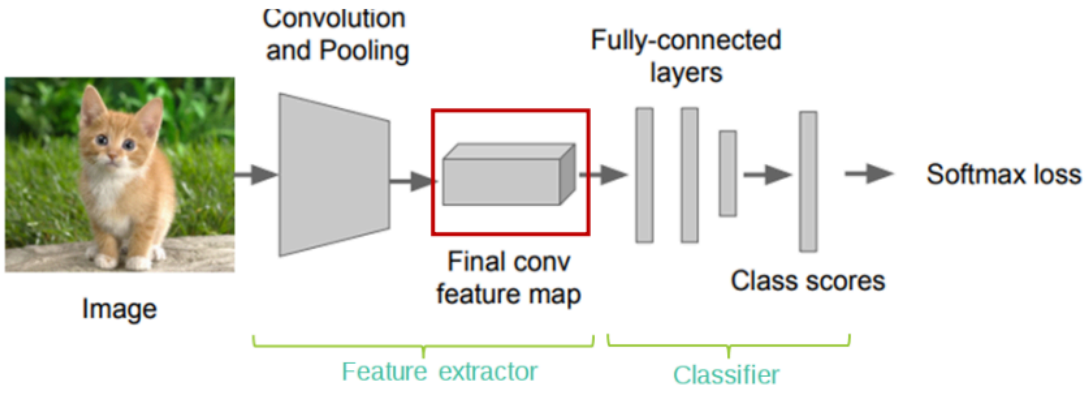
引入

表示学习的目标是将数据点x_i映射到空间$\mathbb{R}^d$中的一个向量。显然，这种映射方式有无数种，我们希望找到这样一种映射，可以保留尽可能多的**语义信息** (Semantic Information) 。这里的x_i不是标量的意思，可以代表各种类型的数据，例如图片、文本等。

例如，数据为手写数字的图片。我们希望映射后，相同数字的图片在空间中尽可能相近。
这样重新表示后，数据更为紧凑 (Compact),而且仅保留了重要的语义信息，便于后续的处理，也很容易使用一些
下游的方法来处理：例如分类、聚类等。

图片表示

对于有监督的情况，之前介绍过用 CNN 来训练一个分类器：

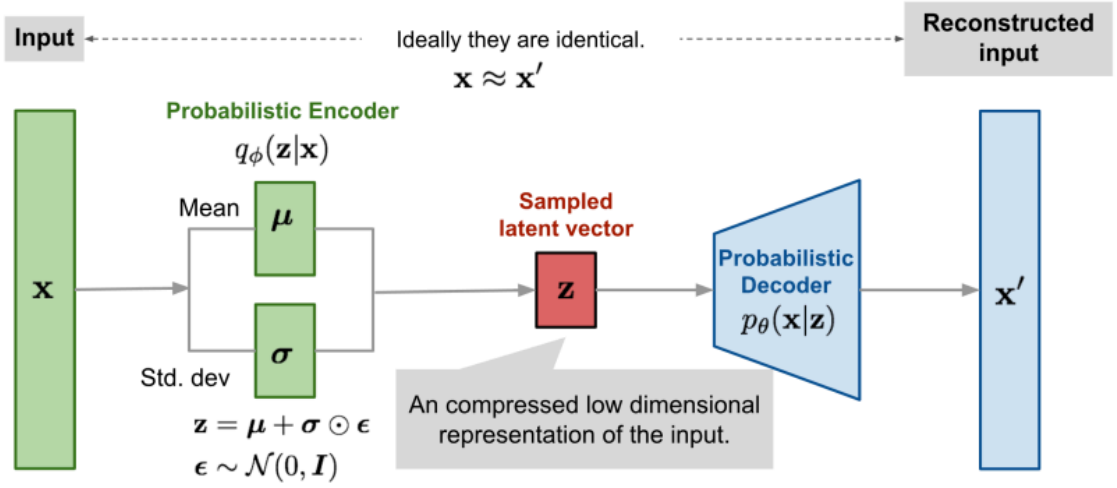


卷积-池化层的部分可以看成在做特征提取，全连接层的部分可以看成在做分类。则最后一个卷积-池化层的输出就可
以看作是对原始图片的一个重新表示。有时，全连接层的输出也可以看作是一个表示。

对于无监督的情况，可以先用一个带**标签的辅助数据集**训练一个 CNN,这个预训练的 CNN 同样可以看为是特征提取。

再在无标签的数据集上，用这个预训练的 CNN,就得到了一个新的表示。

上一讲介绍到的变分自编码器，其中的编码器就相当于做特征提取，编码后的结果也可以看做是一个表示。PCA可以
看做是一个线性的自编码器。

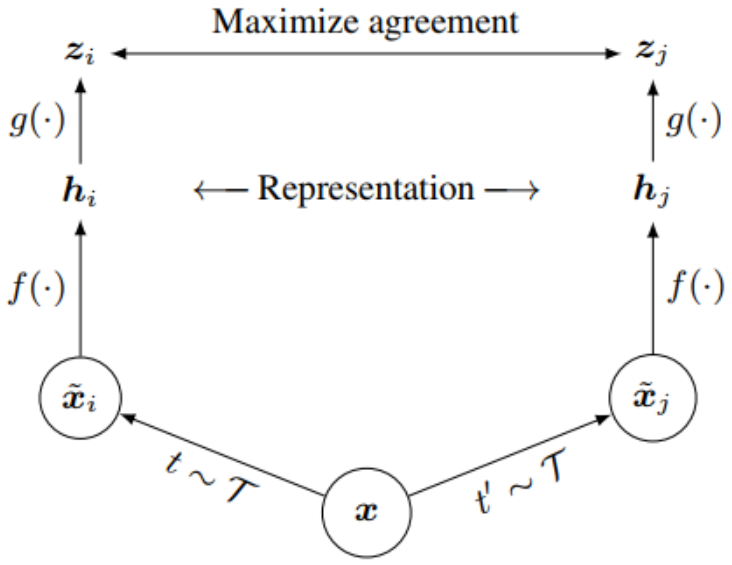


SimCLR

Simple Framework for Contrastive Learning, 是一种对比学习(Contrastive Learning)框架，
对于一张图片，我们可以对其做一些变换，例如裁剪、旋转、翻转等，这个过程也称为数据增广(Data)
Augmentation) :

$$\boldsymbol{x}_i^+ = \mathrm{aug}(\boldsymbol{x}_i)$$

SimCLR 的基本思想就是让x_i和x_i^+在表示空间中尽可能接近，而与其他图片x_j及其增广x_j^+更远



图中$\boldsymbol{h}_i, \boldsymbol{h}_j$就可以看作是$\tilde{\boldsymbol{x}}_i, \tilde{\boldsymbol{x}}_j$的表示。$g(\cdot)$被称为投影头 (Projection Head),通常是一个较浅的神经网络，类似
于 CNN 分类器中最后的全连接层。

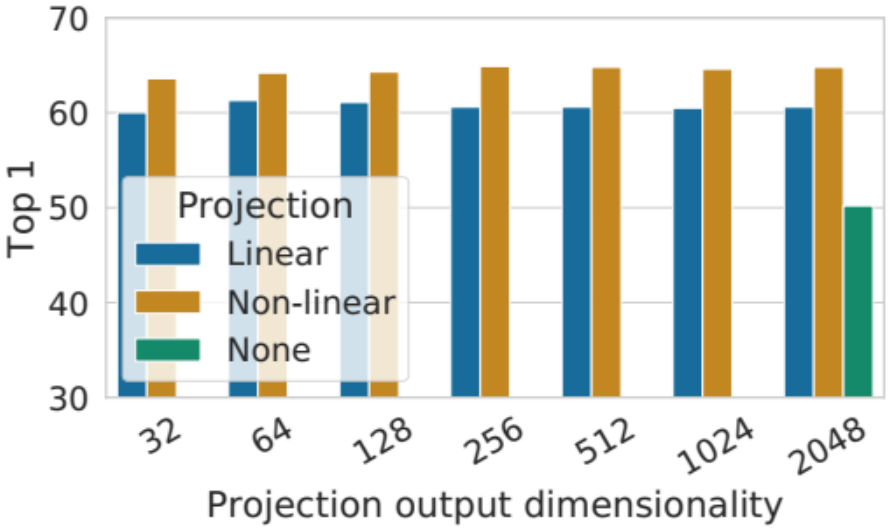
某个数据点上的损失函数(称之为InfoNCE Loss)可以写为：

$$L = -\log \frac{\exp(\sin(\boldsymbol{z}_i, \boldsymbol{z}_i^+)/\tau)}{\sum_{j \neq i} \exp(\sin(\boldsymbol{z}_i, \boldsymbol{z}_j)/\tau) + \sum_{j \neq i} \exp(\sin(\boldsymbol{z}_i, \boldsymbol{z}_j^+)/\tau) + \exp(\sin(\boldsymbol{z}$$

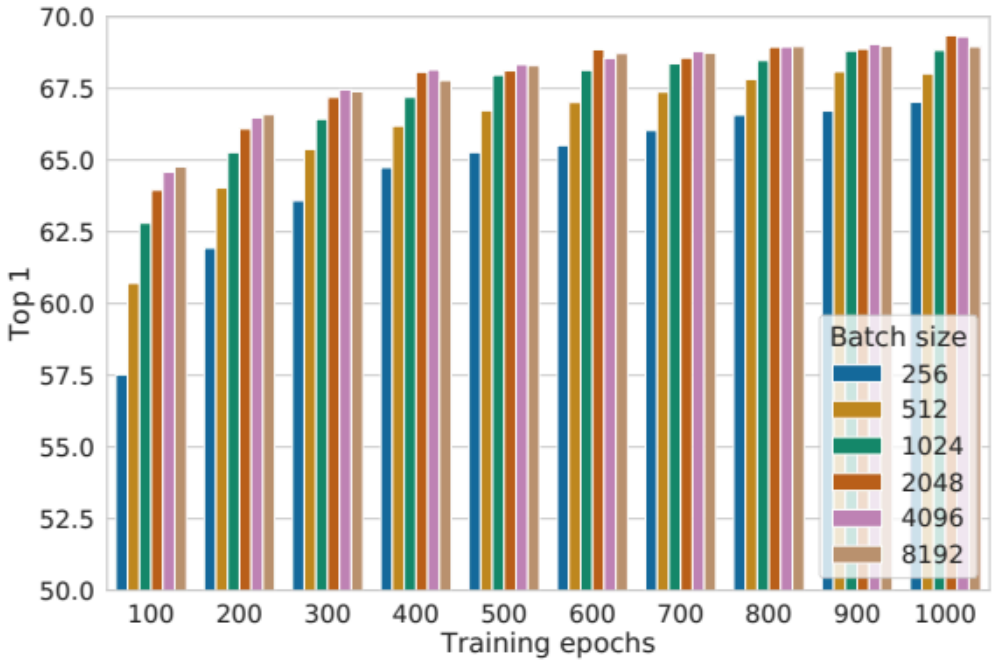
其中$\mathrm{sim}(z_i, z_j) = z_i^T z_j / \|z_i\| \|z_j\|$

由这个函数可以看出，最小化这个损失，就是在尽可能将z_i和z_i^+拉近，同时将z_i和其他图片的表示拉远。当在所有数据点上同时做这件事时，不仅每个点及其增广倾向于接近，相同类别的点也会更接近。举例来说的话，一张狗的图片及其变形会尽量接近，同时不同的狗的图片之间也会倾向于接近。

首先利用 SimCLR 在某个无标签的大数据集上训练一个特征提取器$f(\cdot)$,然后冻结特征提取器(不更新其中的参
数),对比着少量带标签的数据训练一个线性分类器$g(\cdot)$,得到整个模型。
其中前半部分可以看作是在训练一个一般的特征提取器，后半部分可以看作针对任务进行的微调。
后半部分训练的投影头可以显著提升整个模型的性能：



同时使用大批量的数据进行训练也可以显著提升 SimCLR 的性能，但是也会对硬件有更高的要求：



对于两个服从联合分布$p(x,y)$的随机变量X,Y,它们的互信息(Mutualınformation)定义为:

$$I(X,Y)=\int p(x,y)\log\frac{p(x,y)}{p(x)p(y)}dxdy$$

可以从不同的角度理解:

• $I(x,y)$ =KL$(p(x,y)\|p(x)p(y))$:这两个分布之间的 KL 散度("距离")
· $I(x,y)=H(x)-H(x|y)=H(y)-H(y|x)$:已知一个变量给另一个变量带来的信息量
直观来说，且信息可以埋解为两者之间的相关性，共享的信息量。

可以证明，互信息的一个下界是:

$$I(X,Y)\geqslant\log K+\mathbb{E}_{p(x,y)}\prod_{k=1}^{K}p(y_k)\log\frac{e^{s(x,y)}}{e^{s(x,y)}+\sum_{k=1}^{K-1}e^{s(x,y_k)}}$$

其中$p(y)=\int p(x,y)dx$是边缘分布。$s(x,y)$是一个任意形式的分布，实际中通过训练它，使得下界最大 (更接近
互信息)。

期望可以通过采样的方式估计。从联合分布中抽取一个样本(x, y),然后从边缘分布中独立抽取$K-1$个样本y_k,就
可以得到上面不等式右侧的一个下界的估计。于是一个下界的估计可以写为:

$$I(X, Y) \geqslant L \approx \log K+\log \frac{e^{s(x, y)}}{e^{s(x, y)}+\sum_{k=1}^{K-1} e^{s(x, y_k)}}$$

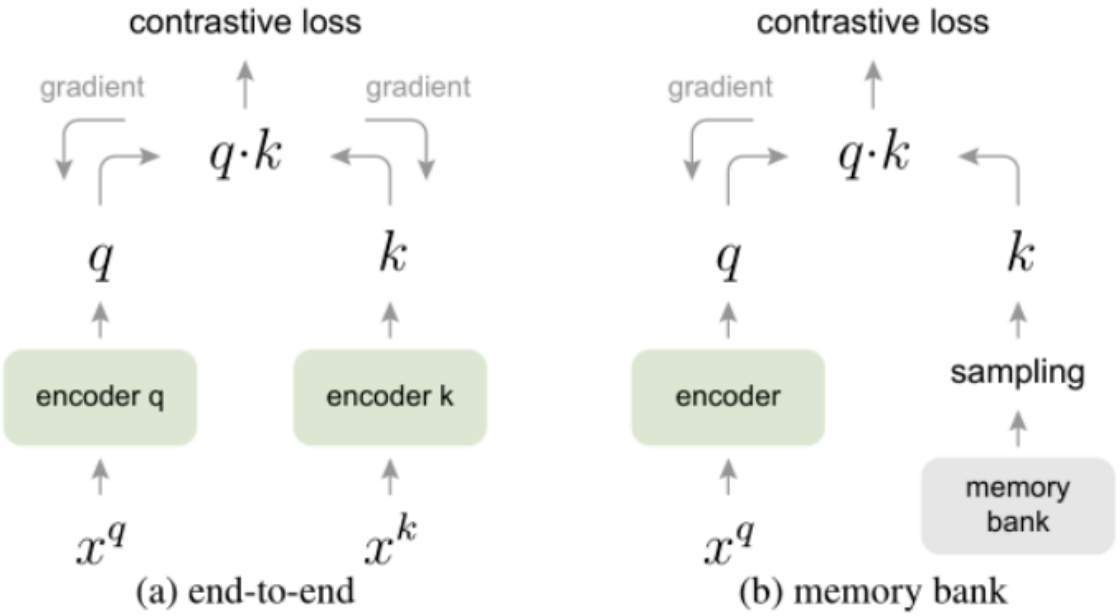
MoCo

名字来源于 Momentum Contrast, 是另一种对比学习框架。

首先引入一些概念。在 SimCLR 中，对原始数据x做增广得到x^+,我们分别称为锚点ι基准点(Anchor)和正样本相应的，其他的数据和它们的增广称为负样本。SimCLR 就是在训练一个编码器，使得由锚点和正样本编码出来的特征在表示空间中尽可能接近，而与负样本尽可能远。
对比学习可以看成是一个字典查询任务。构建一个字典，字典的键就是各个样本，值就是编码后的特征(表示)。我们称锚点的编码为 query (或者简记为 q,对应的原始数据为x^q),正样本和负样本的编码为 key (或者简记为 k, 对应的原始数据为x^k) 。对于一个 query,我们希望在字典中找到与之最接近的 key。注意，这里的 key 并不是 (字典的)键的意思，只是一个称呼，其实际对应是字典的值。

SimCLR 中，实际是对锚点x^q和正负样本x^k使用相同的编码器得到 q 和 k,然后计算相似度，使得 q 和正样本的 k 的相似度尽可能大，而 q 和负样本的 k 相似度尽可能小，以此更新编码器参数。SimCLR 中，每个批次数据以及编码出的特征构成字典。所以字典大小与批次大小相关联，容易受到硬件瓶颈限制。
后来引入了 Memory Bank 的概念，所有样本的编码特征都存储在 Memory Bank 中。每次训练，正常获得 q,然后从 Memory Bank 中随机抽取部分 k 以及对应的x^k构成字典，然后计算相似度，更新编码器参数。最后用新的编码器对x^k 重新编码，更新 Memony Bank。这种方法下，字典大小与批次大小解耦了，一定程度上解决了硬件瓶颈问题

但是这种方法下，由于每次只更新 Memory Bank 的部分编码，所以会导致 Memory Bank 中的编码特征的一致性很差（不是同一个编码器编码的），最终导致 q 和 k 的相似度计算不准确。



MoCo 在 Memory Bank 的基础上进一步改进，它给 Memory Bank 引入了一个更新缓慢的动量编码器。动量编码器的参数更新不如主编码器那么频繁，所以 Memory Bank 中的编码特征相对比较一致。

另外，Memory Bank 换用队列结构。队列中的每个元素都是一个字典。对一个批次训练，使用队头的字典。主编码器、动量编码器的参数更新后，对使用到的字典用动量编码器进行更新，然后将这个字典放入队列的尾部，同时将队列头部的字典移除。

通过以上改进，不仅实现了字典大小与批次大小的解耦，还解决了 Memory Bank 中编码特征的一致性问题。

MoCo v2 结合了 SimCLR 中非线性投影头，以及 MoCo 中的 Memory Bank 和动量编码器，进一步提升了性能。

文本表示

独热编码、BOW 和 TFIDF

最简单的表示文本数据的方法，是将词表中的每一个词都用一个独热向量来表示。但显然，当句子非常长时，这种方
法就非常的笨重。

另一种表示方法——词袋模型 (Bag of Words, BOW),丢起来句子内词的顺序信息，只考虑文本中每个词的数
量，也即每个词被认为有同等的重要性。但是，不同的词所含的信息量是不同的(例如介词和名词，两者所含的信息
相差甚远),所以这种方法也有局限性。

TFIDF, Term Frequency-Inverse Document Frequency,这种方法认为每个词有不同的重要性。

一个词的 TFIDF 计算公式为：

$$\mathrm{TFIDF}(w, d, \mathcal{D}) = \mathrm{TF}(w, d) \cdot \mathrm{IDF}(w, \mathcal{D})$$

其中w代表一个词，d代表第d篇文章，$\mathcal{D}$代表整个语料库。
TF(w, d),也即名字中的的 TF——词频，表示文章d中，w出现的频率TF$(w, d) = n_w / n$
IDF$(w, \mathcal{D})$,也即名字中的 IDF——文档频率倒数，实际表示出现w的文章的频率倒数的对数IDF$(w, \mathcal{D}) =$
$\log(N / N_{\text{contain w}})$
IDF 衡量了w包含信息的多少。w的IDF越大，说明其包含了越多信息，在语料的重要性更大。

以上三种方法均不能反映“近义词关系”，或者说语义相似性（Semantic Similarities），例如“soccer”和“football”在表示空间中可能很不一样。

而且用这三种方法进行表示，维度都比较高（尤其是独热编码），而且比较稀疏。
BOW 和 TDIDF 还不包含词的顺序信息，而众所周知，同一个词在不同上下文的含义可能是完全相反的。

word2vec

基于上面三种方法的局限，我们希望设计出这样的一种文本表示方法：

·表示是比较紧的(表示空间尽可能低维)

·能够反映近义词(词之间的语义相似性)

一个朴素的观察是，如果两个词经常在句子中同时出现(或相互替换),就说明它们有某种程度的语义相似性，也就
是说词的语义相似性隐含在句子中。我们可以通过学习大量的句子，来捕捉到这种相似性

有两种基本的方法，Skip-grams 和 CBOW (Continuous Bag-of-Words) 。前者是根据一个中心词来预测出前后的
词，然后与原文比较，进行训练；后者是根据前后词预测出中心词，然后与原文比较，进行训练。
首先介绍 Skip-grams。一开始，每一个词都用一个随机的实值向量来表示$w \in \mathbb{R}^m$,也称为这个词的(在表示空间
中的)嵌入(Embedding)。

句子$w_1 w_1 w_2 \cdots w_T$,遍历每个词作为中心词为w_t,则目标是最小化下面这个函数:

$$J(\boldsymbol{\theta}) = -\frac{1}{T} \sum_{t=1}^{T} \sum_{|j| \leqslant m, j \neq 0} \log P(\boldsymbol{w}_{t+j} | \boldsymbol{w}_t; \boldsymbol{\theta})$$

其中预测词出现的概率$P(\boldsymbol{w}_{t+j} | \boldsymbol{w}_t; \boldsymbol{\theta})$：

$$P(\boldsymbol{w}_{t+j} | \boldsymbol{w}_t; \boldsymbol{\theta}) = \frac{\exp(\boldsymbol{w}_t^T \boldsymbol{w}_{t+j})}{\sum_{\boldsymbol{w} \in \mathcal{W}} \exp(\boldsymbol{w}_t^T \boldsymbol{w})}$$

其中$\mathcal{W}$代表整个词典。

用大量语料训练后，具有语义相似性越大的两个词的嵌入(两个向量),其sim
$(w_i, w_j) = \cos\theta$大。
CBOW 则反之，是类似的，这里不再赘述。

句子的嵌入可以通过句子中的词嵌入的平均得到$\mathbb{R}^m$,或者拼接为一个矩阵$\mathbb{R}^{T \times m}$,或者用 CNN/ISTM 进行提取(用
下游任务训练句嵌入)。

ELMo

Word2Vex 一个缺点是，每个词都只有一个固定的嵌入。但是一个词可能有很多意思，不同的语义下应该用不同的嵌入方式。

因此，考虑根据输入的句子作为上下文，来具体计算每个词的表征，提出了 ELMo（Embeddings from Language Model）。其基本思想，是用训练语言模型的套路，把语言模型中间隐含层的输出提取出来，作为这个词在当前上下文情境下的表征。（这与我们在图像表示中，用 CNN 最后一个卷积层作为表示的思路类似）

具体而言，其流程如下：

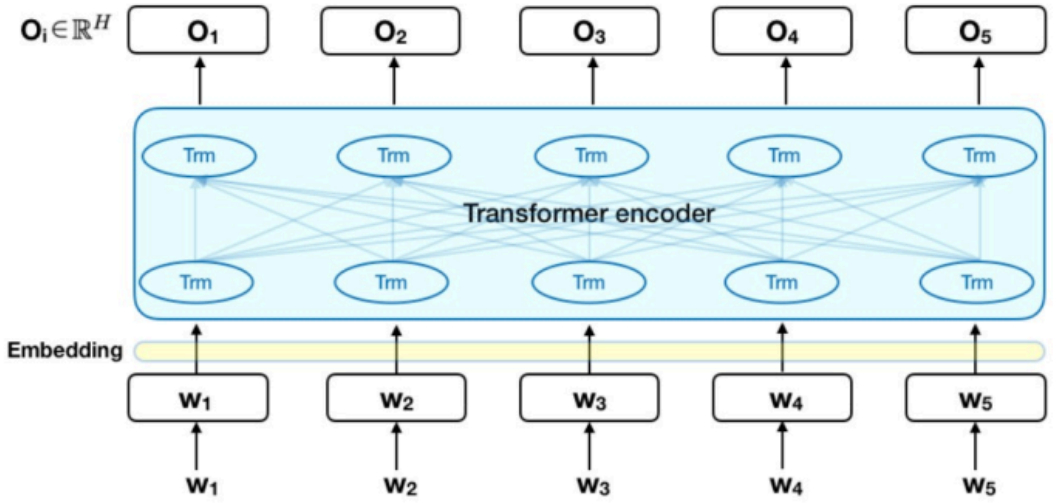
1. 在大语料库上训练一个双向 RNN
2. 将感兴趣的句子正向/反向输入这个预训练的 RNN
3. 最后的词的嵌入由两种情况下 RNN 的隐藏层拼接起来构成

BERT

ELMo 嵌入也有一些缺陷：

- RNN 记忆能力有限，难以捕获长距离依赖
- RNN 只允许顺序执行操作，难以并行化

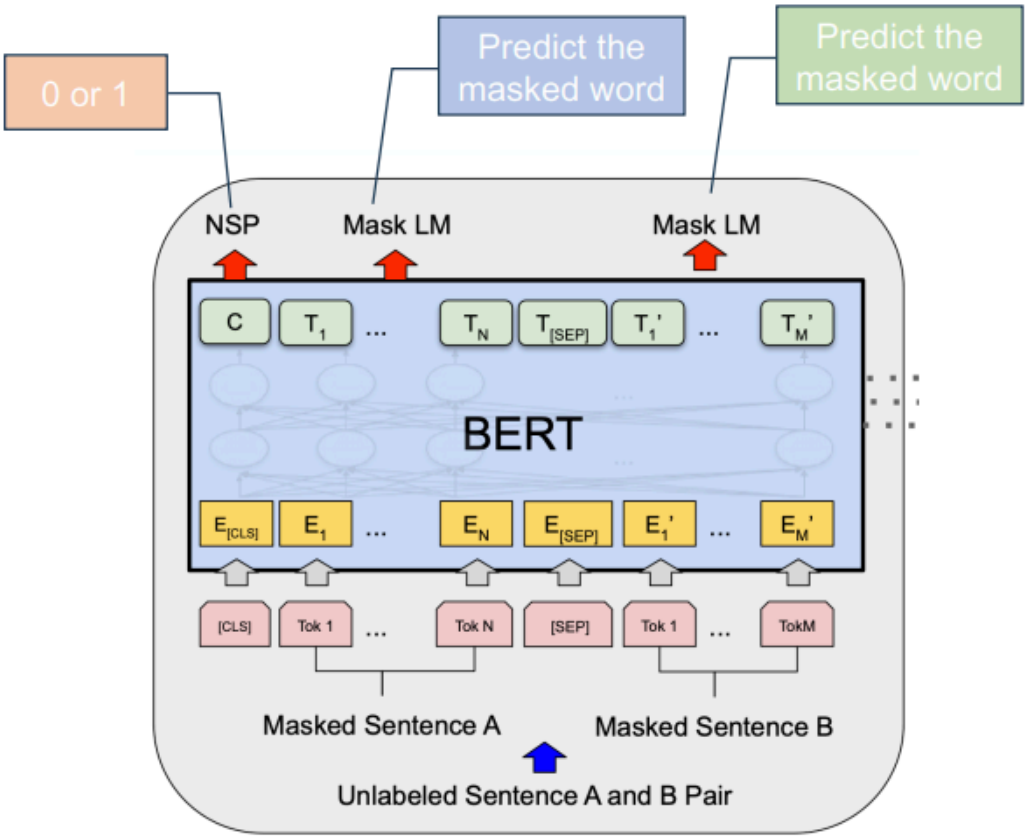
BERT（Bidirectional Encoder Representations from Transformers），其与 RNN 不同，采用的是基于 Transformer 的“全连接”结构。相比于传统的全连接层，Transformer 是一种依赖于自注意力机制的架构，能够有效处理序列数据。



BERT 模型的预训练任务可以分为两个部分：

- Masked Language Model（MLM）：在输入的句子中随机 Mask （掩盖、隐藏）掉一些词，然后训练模型去预测这些 mask 掉的词
- Next Sentence Prediction（NSP）：在输入的句子对中，训练模型去预测第二个句子是否是第一个句子的下一个句子

每次输入两个句子，用特殊的符号区分这两个句子，然后将两个句子拼接起来输入模型，同时进行 MLM 和 NSP 任务：



BERT 模型最终得到的嵌入有三个部分的和构成：

· Token Embeddings: 输入的词的嵌入。

· Segment Embeddings: 区分输入中的不同句子。例如，第一个句子的嵌入为0，第二个句子的嵌入为1。· Position Embeddings: 位置嵌入，用于区分不同位置的词。其公式为：$\mathrm{PE}_{(pos,i)} = \sin(pos/10000^{2i/d})$

推荐系统

1、引入

对推荐问题进行建模，记X为用户集合，S为物品集合，R为用户对物品的效用矩阵(Utility Matrix),R_{ij}可以理解
为用户i对物品j的喜爱程度。效用函数$u：X \times S \rightarrow \mathbb{R}$.

在推荐问题中，有以下几个关键问题：

- 收集可以获得的数据，填充效用矩阵
- 外推无法观测/未知的数据，尤其是关注高的效用值
- 评估外推方法的好坏

对于第一个问题，可以去直接询问（Explit Feedback）用户对物品的评分，也可以通过用户的行为（Implicit Feedback）来推断用户对物品的喜好。例如用户下单购买某商品，相比只是浏览，反映着较高的喜爱。

关于外推其他效用值。首先，由于用户不可能购买过所有商品，所以效用矩阵是稀疏的。也就是说，用户购买商品的记录十分有限，要面临冷启动问题。其次，对于新用户或者是新商品，根本没有任何历史记录，很难做外推。

2、基于内容的推荐

Content-Based Recommendation，这是一种不依赖于效用矩阵的推荐方法。其基本思想是根据该用户打出高分的物品，推荐与之相似的物品，例如推荐相同类型/演员/风格的电影。

为了判断两个物品的相似性，首先我们要为物品创建一个档案（Profile），也即代表物品的特征向量。对于电影来说，档案可以代表导演/标题/演员等；对于文本来说，可以是关键词等。我们需要挑选出最能代表物品的特征，例如对于文本，可以用我们之前提到的 TFIDF 方法。

然后两个物品之间的相似度，可以用档案的余弦相似度来衡量，也即两个向量夹角的余弦。

优点：

- 无需其他用户的数据，没有稀疏以及冷启动的问题。
- 每个用户有一个独立的推荐状态，不易受到其他用户的影响。

- 有可解释性

缺点：

- 找到合适的特征比较困难
- 新用户无历史记录，难以推荐
- 过度专一化，无法跟上用户的新兴趣，也无法利用其它用户的评价来辅助判断。

3、协同过滤

Collaborative Filtering

3.1、用户-用户协同过滤

其灵感是，如果有一批人N和x一样都买了某件商品 A,且N中大部分人还会买 B,可以推测x也会对 B 感兴

趣。

记x对所有商品的评分为$\boldsymbol{r}_x$,$I_{x,y}$是x和y都打过分的上坪的集合，可以引入皮尔逊相关系数(Pearson correlation
coefficient) :

$$\sin(x,y)=\frac{\sum_{i\in I_{xy}}(r_{xi}-\bar{r}_x)(r_{yi}-\bar{r}_y)}{\sqrt{\sum_{i\in I_{xy}}(r_{xi}-\bar{r}_x)^2}\sqrt{\sum_{i\in I_{xy}}(r_{yi}-\bar{r}_y)^2}}$$

这里减去平均值是为了避免将没评分的结果(0)视为最负面的结果，所以减去平均值后，正面的评分为正，负面的
评分为负，更能够反映用户的喜好。

于是可以这样做外推：

$$\hat{r}_{xii}=\bar{r}_x+\frac{\sum_{y\in N}\sin(x,y)(r_{yi}-\bar{r}_y)}{\sum_{y\in N}\sin(x,y)}$$

3.2、物品-物品协同过滤

这种方法的灵感来源于，如果用户x喜欢物品 A,那么他也可能喜欢和 A 相似的物品 B。
其与上面那种方法是类似的，只是这里是计算物品之间的相似度，而不是用户之间的相似度。

$$r_{xi} = b_{xi} + \frac{\sum_{j \in I_x} \sin(i,j)(r_{xj} - b_{xj})}{\sum_{j \in I_x} \sin(i,j)}$$

这里I_x代表用户x评分过的物品集合。其中$b_x i = \mu + b_x + b_i$,μ是所有评分的平均值，b_x是用户x的评分偏差(个
人与大众的偏差),b_i是物品i的评分偏差(大众在物品i上与其他物品的偏差)。
实际中，物品-物品联合推荐表现要比用户-用户联合推荐好，因为物品的特征和相似性更容易捕获，且相对单一。而
用户的品味和爱好是多样的。

3.3、分析

协同过滤的优点：对于各种物品都适用，无需去挑选特征。

缺点：冷启动问题，对于新用户和新物品（First rater），无法推荐。且对于稀疏数据，效果不佳。而且倾向于推荐热门物品，缺乏多样性。

实践中，一般都结合使用上述方法。

4、基于矩阵分解的推荐

考虑效用矩阵$U \in \mathbb{R}^{m \times n}$,其中$m$是用户数，$n$是物品数。我们希期找到两个矩阵
$P^T \in \mathbb{R}^{m \times k}$和$Q \in \mathbb{R}^{n \times k}$,使
得$U \approx P^T Q$.

我们希望近似的结果尽可能接近，并加上一些正则项，所以可以目标是最小化：

$$\min_{P,Q} \sum_{i,j} (r_{ij} - \boldsymbol{p}_i^T \boldsymbol{q}_j)^2 + \lambda(\|\boldsymbol{p}_i\|^2 + \|\boldsymbol{q}_j\|^2)$$

仅考虑有评分的项，通过梯度下降来进行训练。最终可以利用$p^T q$来预测未知的评分。

第十七讲 异常检测

Anomaly Detection，有时也称为离群点检测或新颖性检测。

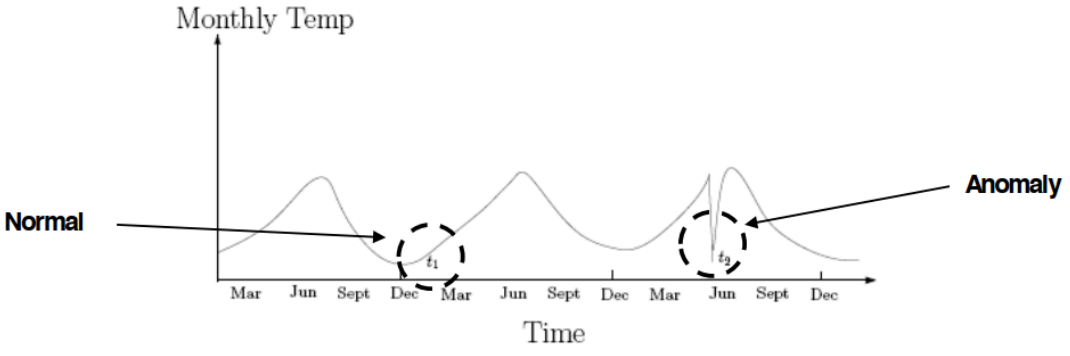
1、引入

一般来说，在一个数据集中，绝大部分的样本都是“相同的”，我们称这些样本为**正常样本**。剩下的样本与正常样本们往往有很大的不同，我们称这些样本为**异常样本**。

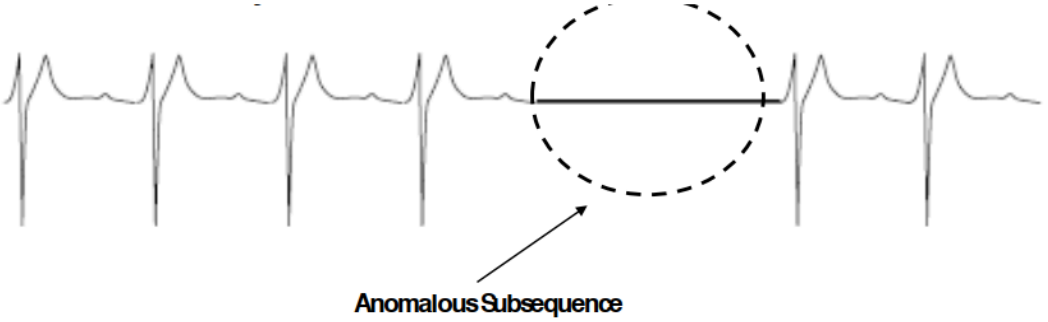
异常样本与正常样本之间差异也是多种多样的，仅通过一个两个显式的标准很难判断某个样本是否异常，这也是异常检测的难点。

网络入侵检测、信用卡欺诈检测、罕见疾病诊断、工业损伤检测等问题，都可以看作是异常检测问题。原来解决这些问题的方法，往往都是依靠已有的或者已出现的异常：如果新的样本和以前出现过的异常样本相似，那么就可以判断这个样本也是异常的。这种方法一个最大的问题是，对于一种全新的异常，是检测不出来的。这也是以前的异常检测方法的一个最大的缺陷。

异常的种类包括：

- **点异常**：只通过样本本身就能判断的异常（相对接下来的上下文异常或者整体异常而言）
- **上下文异常**：样本本身看起来正常，只有放到一个上下文中才能看出的异常。比如一个波正常应该是波峰的区域，出现了波谷：
- 

- **整体异常**：单独看或者在上下文中看都是正常的，但是整体来看是异常的。比如心电图中的某一段变为直线，这段直线在周围正常的波形中就显得异常：



2、基于距离的异常检测

异常点也被称为离群点，也就是距离“大部队”比较“远”的点。基于距离的异常检测方法，就是通过计算样本点与其他样本点的距离，更具体的说，是离第kk个最邻近邻居的距离，来判断样本点是否异常。这个距离越大，说明越有可能是异常。

优点：

- 简单、容易实现
- 可解释性强，符合直觉

缺点：

- 需要知道样本点两两之间的距离，复杂度高达$O(n^2)$
- 对kk的选择敏感
- 对于高维数据，难以找到一种合适的距离度量

3、基于密度的异常检测

另一角度看，离群点旁边的点总是比较少，或者说异常点总是落在低密度的区域。

基于密度的异常检测，首先根据大量的正常样本，估计出一个概率密度分布$\hat{p}(x)$,然后对于一个新的样本x,计算其
概率密度$p(x)$,如果$p(x)$小于某个阈值，就认为这个样本是异常的。

核密度估计 (Kernel Density Estimation) 是一种常用的密度估计方法。这种方法中，一组样本x_i的概率密度函数可
以写为：

$$\hat{p}(\boldsymbol{x}) = \frac{1}{Nh} \sum_{i=1}^{n} K(\frac{\boldsymbol{x} - \boldsymbol{x}_i}{h})$$

其中$K(\cdot)$是某个核函数，h是一个参数，用于控制分布的平滑程度 (带宽)。
KDE 的缺点是，对h的选择敏感，且在高维数据上的表现不佳。

另一种方法是，用已知的分布去拟合数据，类似于之前的高斯混合模型。概率密度分布可写为

$$\hat{p}(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

或者是：

$$\hat{p}(\boldsymbol{x}) = \sum_{i=1}^{k} \pi_i \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$$

其中参数θ通过最大似然估计得到。

这种方法好坏受到选择分布本身的表达能力的限制，且同样在高维数据上表现不佳。

针对在高维数据上表现不佳的问题，提出了两阶段法：

1. 通过某种方式（PCA、自编码器等）对数据进行降维/学习到一个低维的表示
2. 利用 KDE 或者已知分布拟合，对降维后的数据进行密度估计

用大量正常样本训练深度生成模型（如 VAE、GAN）等，也可以得到一个概率密度分布。然后用这个分布来判断新样本是否异常。

4、基于重建的异常检测

在大量正常样本上训练一个编码——解码模型。编码器将输入的样本映射到一个低维的表示，训练解码器能够将这个低维表示重建回原始的样本。

由于是在正常样本上训练的，所以对于正常样本，重建出来的样本应该和原始样本相似；而对于异常样本，重建出来的样本应该和原始样本有较大的差异。依据这个原理，我们可以用重建误差来判断样本是否异常。

其实，这种方法的本质，也是在寻找正常样本的某种特征或者说低维表示（训练编码器），只是利用特征/低维表示的方式有所不同。

5、单类分类器

由于大量的数据都比较“靠近”，可以想象画一个圈，将大部分的数据包含在里面，圈外的数据就是异常的。也就是说，只划分一个类出来，所以这种方法叫做单类分类器。

首先考虑找到一个最小的超球，能将所有的样本点包含进去。设超球的半径为r,球心为c,则这个问题可以写为：

$$\min_{c,r} r^2, \mathrm{s.t.} \|\phi(\boldsymbol{x}_i) - \boldsymbol{c}\|^2 \leqslant r^2$$

其中$\phi(\cdot)$是任意一种非线性映射。

显然，这样的超球很容易受到离群点的影响。为了将离群点包含讲去，不得不增大超球的半径。

首先考虑找到一个最小的超球，能将所有的样本点包含进去。设超球的半径为rr，球心为cc，则这个问题可以写为：

minc,rr2, s.t. ‖ϕ(xi)−c‖2⩽r2c,rminr2, s.t. ‖ϕ(xi)−c‖2⩽r2

其中ϕ(⋅)ϕ(⋅)是任意一种非线性映射。

显然，这样的超球很容易受到离群点的影响。为了将离群点包含进去，不得不增大超球的半径。

接下来，考虑松弛，将这些离群点"放出去”。引入一个松弛变量ξ_i,则问题可以写为：

$$\min_{c,r,\xi} r^2 + \frac{1}{\nu n}\sum_{i=1}^{n}\xi_i,\ \text{s.t.}\|\phi(\boldsymbol{x}_i) - \boldsymbol{c}\|^2 \leqslant r^2 + \xi_i, \xi_i \geqslant 0$$

其中ν是一个超参数，用于控制松弛的程度。当ν趋于 0时，这个问题就退化为前面的问题。(很类似于软间隔 SVM

的形式)

通过这样的超球，就可以按照一开始的想法，将超球内的样本划分为正常样本，超球外的样本划分为异常样本。

$\phi(\cdot)$可以是某个核函数，这样优化问题可以通过解其对偶形式解决。这就完全类似于 SVM 的形式了。
$\phi(\cdot)$还可以是一个神经网络，可以通过用 SGD 来优化以下无约束的损失函数：

$$L = r^2 + \frac{1}{\nu n}\sum_{i=1}^{n}\max(0, \|\phi(\boldsymbol{x}_i) - \boldsymbol{c}\|^2 - r^2)$$

6、评估指标

如何评估异常检测的好坏?

根据样本实际的正常与否以及预测结果，可以得出下面这个表格：

		Actual	
		Positive	Negative
Predicted	Positive	True Positive	False Positive
	Negative	False Negative	True Negative

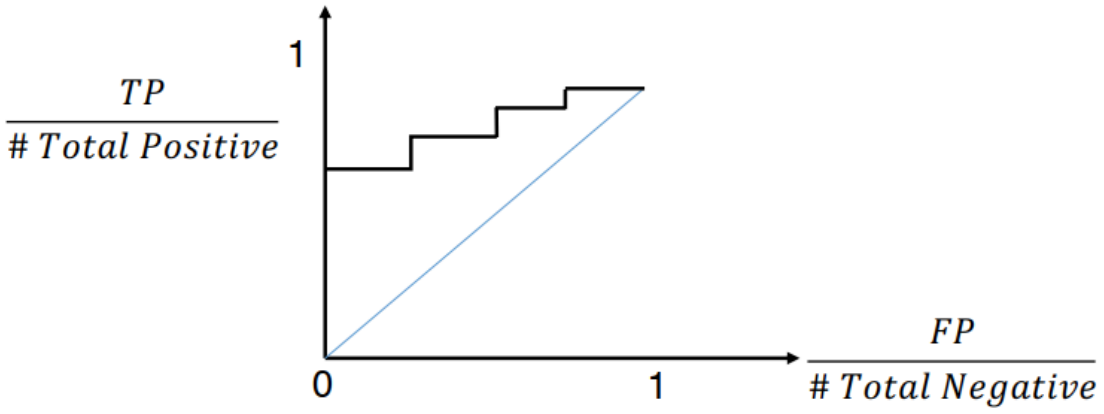
四类样本的数量，用其名字的首字母表示，例如 TP 代表 True Positive。
一种直接的想法是用预测的准确率来评估：

$$\mathrm{AC} = \frac{TP + TN}{TP + TN + FP + FN}$$

但是，这种指标对于异常检测来说并不合适，因为异常样本往往是少数，而正常样本是多数。粗暴地全部认为是正确
样本，就能得到一个很高的准确率。

6.1、ROC 曲线

考虑将正常样本预测为正常的比例（True Positive Rate，也称为召回率）TPR=TP/(TP+FN)，以及将异常样本预测为正常的比例（False Positive Rate）FPR=FP/(FP+TN)。考虑采用不同的分类阈值，得到多组 TPR 和 FPR。然后以 TPR 为纵轴，FPR 为横轴，做出曲线图，这个曲线就称为**ROC 曲线**（Receiver Operating Characteristic Curve）：



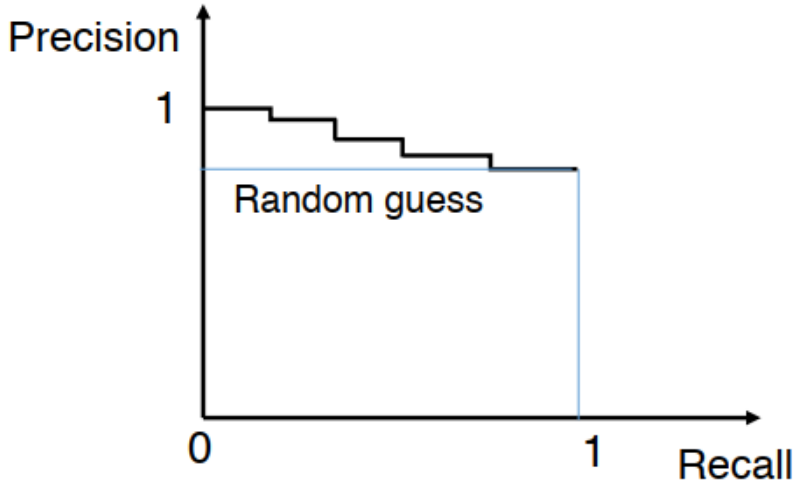
可以想象，某个分类阈值对应的点越靠近左上角，说明其能在正确识别出绝大部分正常样本的同时，只将很少的异常样本误判为正常样本，也就说明这个分类阈值更好。

而对于整个异常检测方法而言，可以用 ROC 曲线下的面积（Area Under ROC Curve，AUROC）来评估。AUROC 越大，说明曲线越靠近左上角，也就说明每个分类阈值下表现都很好，也就说明这个异常检测方法越好。

随机预测的情况下，TPR 始终为 0.5，对应 AUROC 为 0.5。所以可以 0.5 作为基准，来评估异常检测方法的好坏。

6.2、PR 曲线

考虑召回率Recall=TP/(TP+FN)和精确率Precision=TP/(TP+FP)。同样，考虑不同的分类阈值，得到多组召回率和精确率。以召回率为横轴，精确率为纵轴，做出曲线图，这个曲线就称为**PR 曲线**（Precision-Recall Curve）：



某个分类阈值对应的点越靠近右上角，说明不但能够召回大部分正常样本（横坐标大，召回率高），还能较好的剔除异常样本（纵坐标大，精确率高），也就说明这个分类阈值更好。

对于整个异常检测方法而言，可以用 PR 曲线下的面积（Area Under PR Curve，AUPR）来评估。AUPR 越大，说明曲线越靠近右上角，也就说明每个分类阈值下表现都很好，也就说明这个异常检测方法越好。

随机检测的 AUPR 不一定为 0.5，可以直接将用了异常检测方法的 AUPR 与随机检测的 AUPR 进行比较，如果更大，说明这个异常检测方法是有效的。

6.3、F1 值

这是一个形式和使用更加简单的指标。F1 值是精确率和召回率的调和平均：

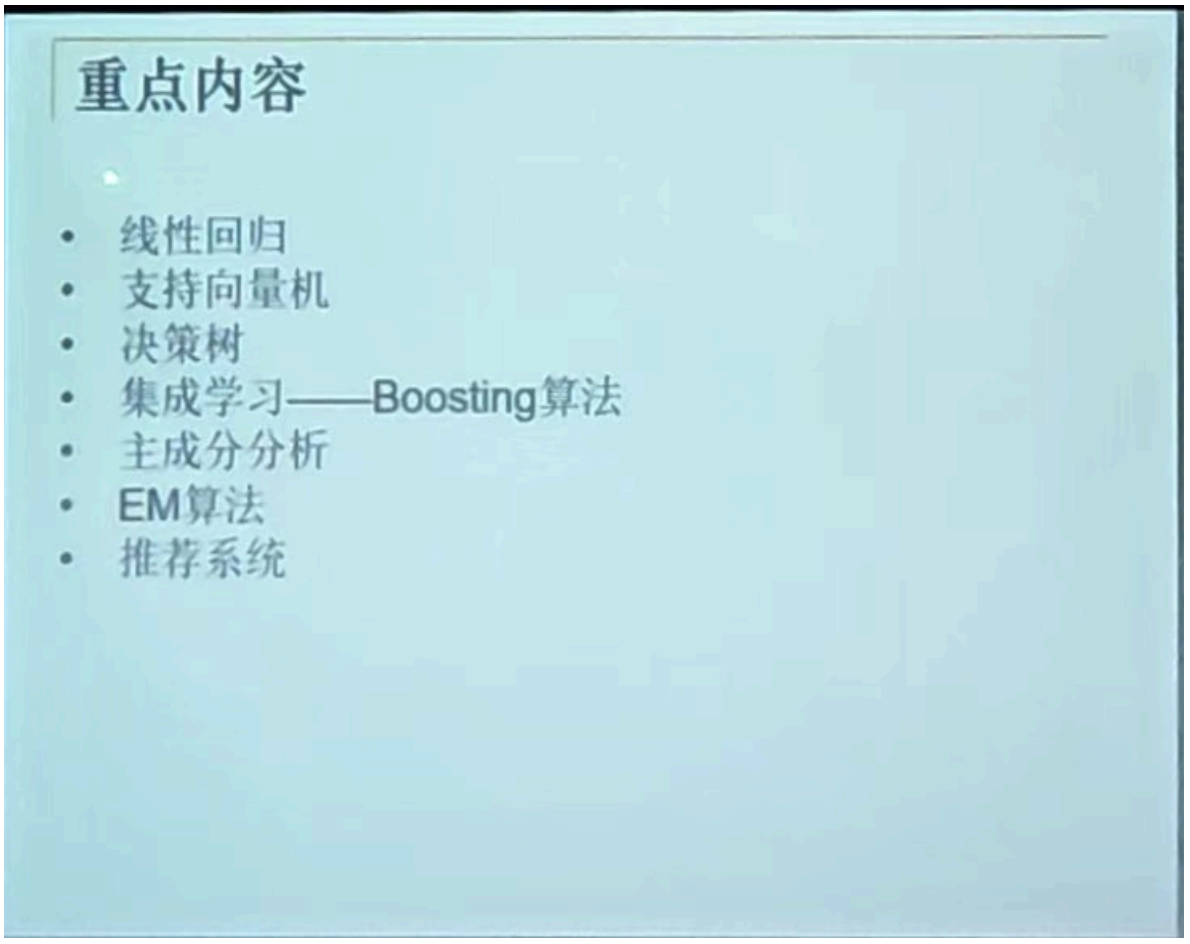
$$F1 = \frac{2}{\mathrm{Precision}^{-1}+\mathrm{Recall}^{-1}} = 2 \cdot \frac{\mathrm{Precision} \times \mathrm{Recall}}{\mathrm{Precision}+\mathrm{Recall}}$$

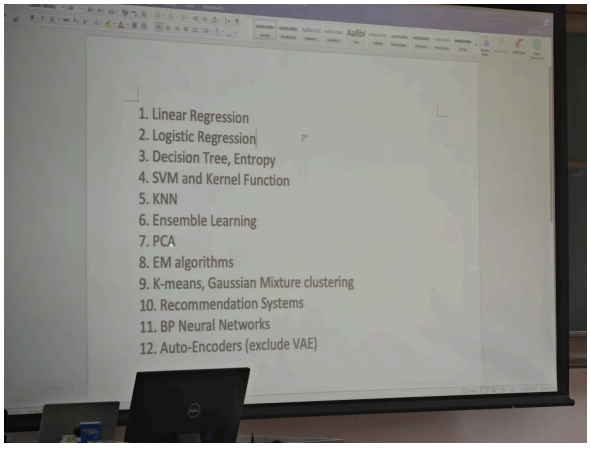
F1 值越大，说明精确率和召回率都很高，也就说明这个异常检测方法越好。

期末大纲

6道大题三道监督三道非监督

深度生成模型基本不考


重点内容
线性回归
支持向量机
决策树
集成学习——Boosting算法
主成分分析
EM算法
推荐系统


1. Linear Regression
2. Logistic Regression
3. Decision Tree, Entropy
4. SVM and Kernel Function
5. KNN
6. Ensemble Learning
7. PCA
8. EM algorithms
9. K-means, Gaussian Mixture clustering
10. Recommendation Systems
11. BP Neural Networks
12. Auto-Encoders (exclude VAE)

SVM最后的证明：

min $\frac{1}{2}\|w\|$ ①

S.t. i...

→ w, w 略#.

Decision Tree ③

Conditional entropy.

知道怎么构成树.

Bagging & Boosting

↓

boose the loss of wrong instance

#SGD

Adam ②

dropout ☆

regularization ☆

normolization → batch
→ layer
→ insance

6大块 { 3 surperised.
3 unsurperised.

PCA → 简答题 公式 × 证明 ④

重点.

+协方差矩阵 . SVD

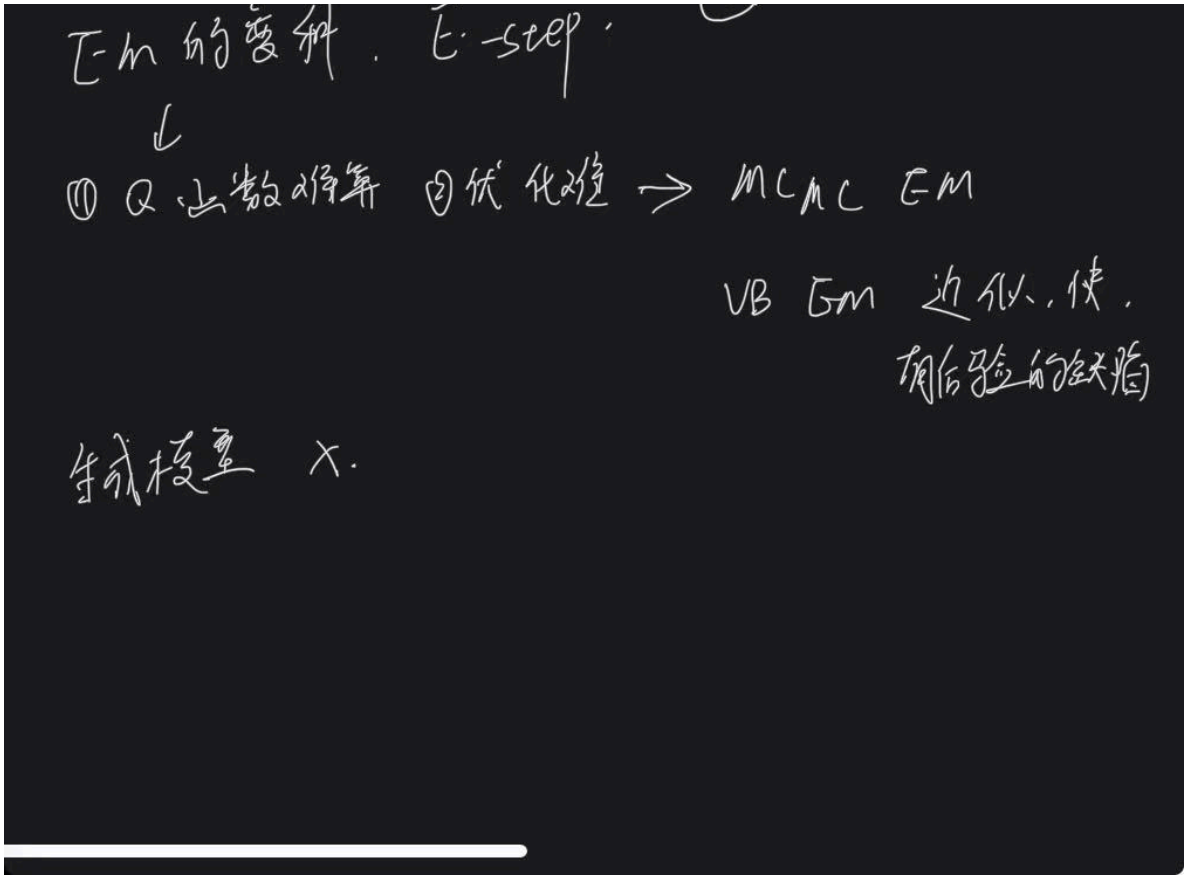
K-MEANS (A差 or B差 选一)

GMM Gaussian Mixture model (先看他) ⑤.

强调 n 遍

EM. : 好.了收敛证明 ×

⑥


EM的变种. E-step.
① Q函数的算 ② 优化难 → MCMC EM
VB EM 近似. 快.
有后验的缺陷
生成模型 X.

1. 线性与非线性模型

- **线性模型**：理解线性分类器和回归模型的基本原理。
- **非线性模型**：重点掌握SVM（支持向量机）和神经网络如何通过核函数（kernel function）将输入空间映射到高维空间，实现非线性分类。
- **核函数**：理解核函数的作用，特别是如何通过核函数将输入数据映射到高维空间，从而实现非线性分类。

2. 过拟合问题

- **过拟合的定义**：模型在训练数据上表现很好，但在测试数据上表现不佳。
- **防止过拟合的方法**：
  - **正则化**：L1正则化（Lasso）和L2正则化（Ridge）的作用及其区别。L1正则化倾向于产生稀疏解，L2正则化则倾向于让权重值较小。
  - **Dropout**：在神经网络中随机丢弃部分神经元，防止过拟合。
  - **早停法**：在验证集性能下降时停止训练。
  - **数据增强**：通过增加训练数据的多样性来防止过拟合。

3. 支持向量机（SVM）

- **最大间隔分类器**：理解SVM的目标是最大化分类边界与最近数据点（支持向量）之间的距离。
- **软间隔分类器**：引入松弛变量（slack variables）处理线性不可分的情况。
- **核技巧**：通过核函数将数据映射到高维空间，实现非线性分类。
- **支持向量**：理解支持向量的定义及其在SVM中的重要性。

4. 神经网络

- **基本结构**：理解神经网络的基本组成（输入层、隐藏层、输出层）及其工作原理。
- **反向传播（Backpropagation）**：理解反向传播算法的基本原理，但不要求详细推导。
- **优化方法**：
  - **SGD（随机梯度下降）**：理解其基本原理及其在神经网络训练中的应用。
  - **Adam优化器**：理解Adam优化器的基本原理及其优势。
- **正则化技术**：
  - **Dropout**：随机丢弃神经元，防止过拟合。
  - **Batch Normalization**：通过标准化每层的输

入，加速训练并提高模型稳定性。

- **Layer Normalization**：在Transformer等模型中常用的归一化方法。

5. 决策树

- **信息增益**：理解信息增益的概念及其在决策树构建中的作用。
- **决策树的构建**：掌握如何通过信息增益来选择特征进行节点分裂。
- **过拟合问题**：理解决策树容易过拟合的原因及其解决方法（如剪枝）。

6. 集成学习

- **Bagging**：理解Bagging的基本原理，特别是随机森林（Random Forest）的构建过程。
- **Boosting**：重点掌握AdaBoost的工作原理，理解如何通过调整样本权重来逐步提升模型性能。
- **集成学习的优势**：理解集成学习如何通过组合多个弱分类器来提高模型的泛化能力。

7. 模型评估与调参

- **训练集、验证集、测试集**：理解三者之间的区别及其在模型评估中的作用。
- **过拟合的检测**：通过验证集性能的变化来判断模型

是否过拟合。

- **超参数调优**：理解如何通过交叉验证等方法来选择最优的超参数。

8. 优化方法

- **SGD（随机梯度下降）**：理解其基本原理及其在神经网络训练中的应用。
- **Adam优化器**：理解Adam优化器的基本原理及其优势。

9. 正则化技术

- **L1和L2正则化**：理解L1和L2正则化的区别及其在防止过拟合中的作用。
- **Dropout**：理解Dropout的工作原理及其在神经网络中的应用。
- **Batch Normalization**：理解Batch Normalization的作用及其在加速训练和提高模型稳定性方面的优势。

10. 其他重要概念

- **信息增益与互信息**：理解信息增益和互信息在决策树构建中的作用。
- **模型复杂度与泛化能力**：理解模型复杂度与泛化能力之间的关系，掌握如何通过正则化等方法控制模型复杂度。

考试形式：

- **推导题**：可能会涉及一些简单的推导，如SVM的最大间隔分类器的推导，但不要求复杂的数学证明。
- **概念理解题**：重点考察对核心概念的理解，如过拟合、正则化、核函数、支持向量等。
- **应用题**：可能会给出具体的数据集或模型，要求分析模型的性能、过拟合情况，并提出改进措施。

总结：

考试重点集中在**线性与非线性模型**、**过拟合问题**、**SVM**、**神经网络**、**决策树**、**集成学习**以及**模型评估与调参**等方面。理解这些核心概念及其应用是考试成功的关键。

三、逻辑回归

目标函数的定义

为什么叫做逻辑回归？和线性回归的区别？

求偏导的方式

如何更新参数

为什么要进行正则化？

四、决策树

给定一个数据集，如何构造决策树

决策树解决的任务（分类）

决策树的基本思路（总结即可）

信息增益怎么计算

基尼值和基尼指数是什么 怎么算

信息熵和条件熵是怎么定义的

增益率是怎么算的

为什么要提出增益率

预剪枝和后剪枝是什么（思想）

五、支持向量机

支持向量机的目标

支持向量机的基本思想

如何计算出来点到给定线的距离

支持向量机的目标函数

知道如何求解目标函数

等高线和向量场的意义是什么

如何用拉格朗日求解目标函数（拉格朗日乘数）

理解什么叫作支持向量

支持向量的作用

支持向量机的优点和缺点

核函数的功能

核函数的种类及其功能（如高斯核函数）

引入核函数的原因

六、KNN

KNN是用来做什么任务的

KNN的优缺点

KNN的敏感度

七、集成学习

集成学习的动机是什么

集成学习的经典算法

Bagging算法是什么?

Boosting算法是什么?

随机森林不需要掌握

无监督聚类的集成学习的基本思想

Soft Correspondence的目标函数的求解

八、 ⭐PCA

一定要了解PCA!

PCA的目的是什么?

PCA的假设是什么?

PCA的目标是什么? （对应的最优化问题的理解）

PCA的推导一定要了解清楚!

最小化误差的推导方式

最大化方差的推导方式

（奇异值分解可以不用管它）

它的一些应用和缺陷

九、EM算法

EM算法是什么

EM算法的参数（观察变量 隐变量 模型参数）

E步和M步分别做什么

EM算法的推导过程

（EM算法的隐变量用在哪里）

EM算法不能获得全局最优的原因

EM算法的应用（如何结合高斯混合聚类）

十、聚类

C-means聚类

K-means聚类是什么

K-means的目标函数和原理

K-means的步骤

K-means的终止条件

层级聚类的目标函数

层级聚类的思想

层级聚类的步骤

高斯混合聚类的步骤

高斯混合聚类的均值和方差是怎么更新的

十一、推荐系统

推荐系统的目的

推荐系统的应用

推荐系统的步骤

如何基于内容进行分解

冷启动是什么

协同过滤的优缺点

如何求解相似用户（相似度）

如何计算相似商品的打分

如何评估商品的好坏

推荐系统的评判指标

（基于检索的评判指标 或者是前述提到的评判指标）

十二、反向传播神经网络

激活函数及其种类

激活函数的意义

目标函数是什么

模型的参数是怎么更新的

输入信号是怎么传播的

误差是怎么传回的

梯度的更新是如何更新的

动量法的含义（为什么要加一个动量）

反向传播神经网络的应用

（公式不需要去记忆，但需要知道公式的含义）

十三、自动编码器

无监督的方法

目标函数是怎么定义的?

它的网络结构是如何定义的?

损失函数是怎么定义的?

自动编码器的两种类型

PCA和自动编码器的关系

自动编码器的变种算法及其功能原理

用自动编码器设计一个分类器

评价指标

一、评判指标

1. 准确度（Accuracy）

准确度是分类模型中最常用的评估指标之一，表示模型预测正确的样本占总样本的比例。

公式：

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

- **TP（True Positive）**：真正例，模型正确预测为正类的样本。
- **TN（True Negative）**：真负例，模型正确预测为负类的样本。
- **FP（False Positive）**：假正例，模型错误预测为正类的样本。
- **FN（False Negative）**：假负例，模型错误预测为负类的样本。

2. 召回率（Recall）

召回率表示模型正确预测的正类样本占所有实际正类样本的比例，衡量模型找出正类样本的能力。

公式：

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

3. 如何根据表格计算准确率和召回率

假设有以下混淆矩阵：

	预测为正类	**预测为负类**
实际为正类	TP	FN
实际为负类	FP	TN

- 精**确率（Precision）**：

  $$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- **召回率（Recall）**：

  $$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

4. F评判指标（F-Score）

F-Score是准确率和召回率的调和平均数，常用于衡量模型的平衡性能，尤其是当正负类样本不平衡时。

公式：

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{(\beta^2 \cdot \text{Precision}) + \text{Recall}}$$

常见的F1-Score是F-Score的特例，其中$\beta = 1$：

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

5. P@K（Precision at K)

P@K表示在前K个预测结果中，正确预测的正类样本的比例。

公式：

$$P@K = \frac{\text{前K个结果中正确预测的正类样本数}}{K}$$

6. MAP（Mean Average Precision）

MAP是多个查询的平均精度（AP）的平均值，常用于信息检索任务。

AP：**平均精度**

$\text{AP} = \sum_{k=1}^{n} (P(k) \times \text{rel}(k))$

其中，$P(k)$ 是前 k 个结果的精度(**Precision**)，$\text{rel}(k)$ 表示第 k 个结果是否相关1or0。

MAP：

$\text{MAP} = \frac{1}{Q} \sum_{q=1}^{Q} \text{AP}(q)$

其中，Q 是查询总数，$\text{AP}(q)$ 是第 q 个查询的 AP。

7. DCG（Discounted Cumulative Gain)

DCG用于衡量排序结果的质量，考虑相关文档的位置，越靠前的相关文档贡献越大。

公式：

$$DCG@K = \sum_{i=1}^{K} \frac{\text{rel}_i}{\log_2(i+1)}$$

其中，rel_i表示第i个结果的相关性得分。

8. NDCG（Normalized Discounted Cumulative Gain）

NDCG是DCG的归一化版本，通过将DCG除以理想排序的DCG（IDCG）来归一化。

公式：

$$NDCG@K = \frac{DCG@K}{IDCG@K}$$

其中，IDCG是理想排序下的DCG。

9. MRR（Mean Reciprocal Rank）平均倒数排名

MRR用于衡量排序任务中第一个相关结果的位置，计算所有查询的**倒数排名的平均值**。

公式：

$$MRR = \frac{1}{Q} \sum_{i=1}^{Q} \frac{1}{\text{rank}_i}$$

其中，rank_i是第i个查询中第一个相关结果的排名，Q是查询总数。

总结

- **准确度**：模型预测正确的比例。
- **召回率**：模型找出正类样本的能力。
- **F-Score**：准确率和召回率的调和平均数。
- **P@K**：前K个结果中正确预测的正类样本比例。

- **MAP**：多个查询的平均精度的平均值。
- **DCG**：考虑相关文档位置的排序质量。
- **NDCG**：归一化的DCG。
- **MRR**：第一个相关结果的倒数排名的平均值。

这些指标广泛应用于分类、排序和信息检索等任务中，帮助评估模型的性能。

线性回归

概念

回归 (Regression) 是基于**几个给定的特征值，预测**与这几个特征**相关的未知值。**
从数学的角度，回归的目标是学习出一个函数 $f(\cdot)$,可以很好地**拟合输入**x和**输出**y之间的关系。
线性回归指**限定**函数族为**线性函数**时的回归：

$$f(\boldsymbol{x}) = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_m x_m$$

线性回归的目标就是找到一组合适的参数$\{w_k\}_{k=1}^m$使得训练集中的数据点通过函数f得到的预测值$\hat{y}$,与对应的真实值y**尽可能的接近。**

单分类

预测函数

$$f(x) = w_0 + w_1 x$$

这条直线在数据平面上与数据点的分布越贴合，说明拟合效果越好。

损失函数

假设有n个数据点，每个数据点记为$(x^{(i)}, y^{(i)})$。我们可以定义损失函数来度量与真实数据的"拟合程度”

目标就是确定能最小化这个函数值的参数。

$$L(w_0, w_1) = \frac{1}{n}\sum_{i=1}^{n}(f(x^{(i)}) - y^{(i)})^2$$

考虑$w_0 = 0$时的情况。此时损失函数变为：

$$L(w_1) = \frac{1}{n}\sum_{i=1}^{n}(w_1 x^{(i)} - y^{(i)})^2 = w_1^2\frac{\sum_{i=1}^{n}(x^{(i)})^2}{n} - 2w_1\frac{\sum_{i=1}^{n}(x^{(i)}y^{(i)})}{n} + \frac{\sum}{}$$

如果将输入写成向量的形式，可以简记为：$L(w_1) = w_1^2\overline{x^2} - 2w_1\overline{xy} + \overline{y^2}$,其中上划线是平均值的意思
这个函数是一个凸函数($w_0 \neq 0$时，这个函数也是一个凸函数),令其偏导为 0:

$$\frac{\partial L}{\partial w_0} = 0, \frac{\partial L}{\partial w_1} = 0$$

即可求出最优的参数：

$$w_0 = \overline{y} - w_1\overline{x}, w_1 = \frac{\overline{xy} - \overline{x}\cdot\overline{y}}{\overline{x^2} - (\overline{x})^2}$$

$$f(x) = w_0 + w_1 x$$

多分类器

模型预测函数

输入$(m+1)\times 1$列向量$\boldsymbol{x} = [1, x_1, x_2, \ldots, x_m]^T$　$(m+1)\times 1$列向量$\boldsymbol{w} = [w_0, w_1, \ldots, w_m]^T$

预测值通过 softmax 函数计算。

$$f(x) = \mathbf{w^T x} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_m x_m$$

损失函数

$\boldsymbol{x}^{(i)}$ 是第 i 个样本的特征向量，形状为 $(m+1)\times 1$

$$L(\boldsymbol{w})=\frac{1}{n}\sum_{i=1}^{n}\left(\boldsymbol{w}^T\boldsymbol{x}^{(i)}-y^{(i)}\right)^2$$

写为矩阵形式 $X=[x^{(1)},x^{(2)},\ldots,\boldsymbol{x}^{(n)}]^T$ (w,y也是向量)

$$L(\boldsymbol{w})=\frac{1}{n}\|\boldsymbol{X}\boldsymbol{w}-\boldsymbol{y}\|^2$$

这也是一个凸函数，令其导数为 0：

$$\frac{\partial L(\boldsymbol{w})}{\partial \boldsymbol{w}}=\frac{2}{n}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{w}-\boldsymbol{y})=\boldsymbol{0}$$

得到最优参数为$w^*=(X^TX)^{-1}X^Ty$

概率角度看回归

从概率的角度来看回归问题，我们假设输出变量 y 在给定输入变量 x 和参数 w 的条件下服从正态分布：

$$p(y|x;w)=\mathcal{N}(y;xw,\sigma^2)$$

我们取这个分布的均值作为预测值：

$$\hat{y}=xw$$

目标是**最大化对数似然**函数：

$$\max_w \log p(y|x;w)$$

这等价于最小化平方误差：

$$\min_w (y - xw)^2$$

对于 N 个独立同分布的样本，联合条件概率密度函数为：

$$p(y^{(1)}, \cdots, y^{(N)} | x^{(1)}, \cdots, x^{(N)}; w) = \prod_{i=1}^{N} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2} (y^{(i)} - x^{(i)} w)^2 \right)$$

对数似然函数为：

$$\log p(y|X; w) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^{N} (y^{(i)} - x^{(i)} w)^2$$

最大化对数似然等价于**最小化平方和**：

$$\min_w \sum_{i=1}^{N} (y^{(i)} - x^{(i)} w)^2$$

八股文

1. 参数的定义方法

在线性回归模型中，参数通常指模型的权重（weights）和偏置（bias）。假设模型为：

$$y = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

其中：

- $w_1, w_2, \ldots, w_n$ 是权重参数，表示每个特征对输出的影响。
- b 是偏置参数，表示模型的基线输出。

参数的定义方法是通过最小化目标函数（如均方误差）来确定。

---

2. 模型的数值形式

线性回归模型的数值形式为：

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

其中：

- $\hat{y}$ 是模型的预测值。
- $x_1, x_2, \ldots, x_n$ 是输入特征。
- $w_1, w_2, \ldots, w_n$ 是权重。
- b 是偏置。

用向量形式表示为：

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

其中 $\mathbf{w}$ 是权重向量，$\mathbf{x}$ 是特征向量。

---

3. 目标函数的推导

线性回归的目标是最小化预测值与真实值之间的误差，常用均方误差（MSE）作为目标函数：

$$J(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)})^2$$

其中：

- m 是样本数量。
- $\hat{y}^{(i)}$ 是第 i 个样本的预测值。
- $y^{(i)}$ 是第 i 个样本的真实值。
- $\frac{1}{2}$ 是为了方便求导而添加的常数。

---

4. 什么是梯度下降

梯度下降是一种优化算法，用于最小化目标函数。其核心思想是通过迭代调整参数，沿着目标函数的负梯度方向更新参数，逐步逼近最小值。

梯度下降的更新公式为：

$$\mathbf{w} := \mathbf{w} - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial \mathbf{w}}$$

$$b := b - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial b}$$

其中：

- α 是学习率，控制每次更新的步长。
- $\frac{\partial J(\mathbf{w},b)}{\partial \mathbf{w}}$ 和 $\frac{\partial J(\mathbf{w},b)}{\partial b}$ 分别是目标函数对权重和偏置的偏导数。

---

5. 使用梯度下降求解

梯度下降的求解步骤如下：

1. 初始化参数 $\mathbf{w}$ 和 b。
2. 计算目标函数的梯度（偏导数）。
3. 更新参数：

   $$\mathbf{w} := \mathbf{w} - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial \mathbf{w}}$$

   $$b := b - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial b}$$

4. 重复步骤 2 和 3，直到收敛（目标函数值变化很小或达到最大迭代次数）。

---

6. 学习率对于求解的影响

学习率 α 是梯度下降的关键超参数，影响模型的收敛速度和稳定性：

- **学习率过大**：可能导致参数更新过快，无法**收敛**，甚至发散。
- **学习率过小**：收敛速度慢，训练时间**过长。**
- **合适的学习率**：能够快速且稳定地收敛到最优解。

---

7. 为什么减是求最小值?

梯度下降的目标是最小化目标函数。梯度 $\nabla J(\mathbf{w}, b)$ 指向目标函数**增长最快**的方向，因此负梯度 $-\nabla J(\mathbf{w}, b)$ 指向目标函数**下降最快**的方向。通过减去梯度，参数沿着目标函数下降的方向更新，逐步逼近最小值。

---

8. 为什么要进行空间的转换

空间的转换（如特征缩放、标准化、多项式特征生成等）是为了：

1. **加速收敛**：特征尺度不一致时，梯度下降可能**收敛缓慢。**
2. **提高模型性能**：某些特征可能对目标变量的影响更大，转换后可以更好地**捕捉这些关系。**
3. **解决非线性问题**：通过引入多项式特征或核函数，可以将线性模型**扩展到非线性问题。**

线性分类

二分类

在二分类中，用0和1两种结果值表示两个类$y \in \{0, 1\}$

引入 sigmoid 函数 (逻辑函数),来将值域变换到[0,1]之间：

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

将$f(x) = \sigma(xw)$这样的回归称之为**逻辑回归**

要实现分类，我们希望真实的标签值为 1 时，预测值尽量接近 1;真实标签为 0 时，预测值尽量接近 0。
采用$f(x)=\sigma(xw)$作为预测函数，可以定义损失函数：

$$L(\boldsymbol{w})=\begin{cases}-\log(\sigma(\boldsymbol{xw})), & y=1\\ -\log(1-\sigma(\boldsymbol{xw})), & y=0\end{cases}$$

写成等价形式：

$$L(\boldsymbol{w})=-y\log(\sigma(\boldsymbol{xw}))-(1-y)\log(1-\sigma(\boldsymbol{xw}))$$

这种形式的损失函数，我们称之为交叉熵

交叉熵函数是凸的，比非凸的平方误差更容易优化。

交叉熵的梯度

$$\frac{\partial L(\boldsymbol{w})}{\partial \boldsymbol{w}}=\frac{1}{N}\sum_{i=1}^{N}[\sigma(\boldsymbol{x}^{(i)}\boldsymbol{w})-y^{(i)}]\boldsymbol{x}^{(i)T}$$

由于其是凸的，令梯度为 0 即可得到最优的w^*,但是这里其**解析解不存在**，可以用梯度下降法等**数值解法**求得。

最终，利用最优的w^*

这样进行分类：计算$f(x)=\sigma(xw)$,如果其不小于 0.5,预测值为 1;如果小于 0.5,预测值为 0。

多分类器

模型预测函数

引入了 softmax 函数：

$$\operatorname{softmax}_i(\boldsymbol{z})=\frac{e^{z_i}}{\sum_{k=1}^{K}e^{z_k}}$$

一个显然的性质是$\sum_{i=1}^{K}\text{softmax}_i(\boldsymbol{z})=1$

样本x被分到第i类的概率可以写为：

$$f_i(\boldsymbol{x})=\text{softmax}_i(\boldsymbol{xW})=\frac{e^{\boldsymbol{xw}_i}}{\sum_{k=1}^{K}e^{\boldsymbol{xw}_k}}$$

这里$W=[\boldsymbol{w}_1,\boldsymbol{w}_2,\ldots,\boldsymbol{w}_k]$。如果样本$x$的真实标签是第$i$类，就需要让$f_i(x)$尽可能的大。

损失函数（交叉熵损失）

将真实标签y写成独热编码y。目标是**最大化属于样本对应类别的概率**，则损失概率可以写为：

$$L(\boldsymbol{W})=-\frac{1}{N}\sum_{l=1}^{N}\sum_{k=1}^{K}y_k^{(l)}\log[\text{softmax}_k(\boldsymbol{x}^{(l)}\boldsymbol{W})]$$

其中$y_k^{(l)}$是$y^{(l)}$第k个元素。由独热编码的性质，这里每个样本实际上只被取了在正确类别上的概率，则整个损失函数的意义就
是最大化所有样本在正确类别上的平均概率。

损失函数对 $\mathbf{W}$ 求导

关于$\boldsymbol{w}_j$的梯度是：$\frac{\partial L(\boldsymbol{W})}{\partial \boldsymbol{w}_j}=\frac{1}{N}\sum_{l=1}^{N}\left(\text{softmax}_j\left(\boldsymbol{x}^{(l)}\boldsymbol{W}\right)-\boldsymbol{y}_j^{(l)}\right)\boldsymbol{x}^{(l)T}$ 需要同时更新每个w

用矩阵形式

梯度可以写为：$\frac{\partial L(\boldsymbol{W})}{\partial \boldsymbol{W}}=\frac{1}{N}\sum_{l=1}^{N}\boldsymbol{x}^{(l)T}\left(\text{softmax}\left(\boldsymbol{x}^{(l)}\boldsymbol{W}\right)-\boldsymbol{y}^{(l)}\right)$

使用梯度下降法更新权重

$$w_k\leftarrow w_k-\alpha\cdot\frac{\partial L}{\partial w_k}$$

其中，α 是学习率。

概率角度看分类

假设输出变量 y 是一个独热编码的向量，表示属于哪个类别。例如，对于 K 个类别，y 是一个 K 维向量，只有一个元素是 1，其他是 0。

$\boldsymbol{x} \in \mathbb{R}^D$ 是输入特征向量，表示样本的特征。

W 是 $D \times K$ 的权重矩阵，每一列对应一个类别的权重向量 W_k。

$\pi_k = \mathrm{softmax}(Wx)_k$，即：

$$\pi_k = \frac{\exp(x^T W_k)}{\sum_{k'=1}^{K} \exp(x^T W_{k'})}$$

假设 $p(y|x;W)$ 是一个多分类分布，使用 softmax 函数来表示类别概率：

$$p(y|x;W) = \prod_{k=1}^{K} \pi_k^{y_k}$$

为了简化计算，通常对似然函数取对数，得到对数似然函数：

$$\log p(y|x;W) = \sum_{k=1}^{K} y_k \log \pi_k$$

最大化对数似然等价于最小化负对数似然，即**最小化交叉熵损失**：

$$-\log p(y|x;W) = -\sum_{k=1}^{K} y_k \log \pi_k$$

非线性模型、过拟合与正则化

非线性模型

用基函数去非线性化原来的线性模型。

原来的线性回归可写为$f(x)=w_0+w_1x=[1,x]w$。
如果采用更复杂的基函数，则可以写为$f(x)=w_0+w_1x+w_2x^2+w_3x^3=\boldsymbol{\phi}(x)\boldsymbol{w}$

一般地，基函数模型可以写为：$f(x)=\phi(x)\boldsymbol{w}=[\phi_1(\boldsymbol{x}),\phi_2(\boldsymbol{x}),\cdots,\phi_n(\boldsymbol{x})]\boldsymbol{w}$
基函数模型对x不一定是线性的，但是对参数w仍然是线性的。

经过非线性变换后，损失函数可以写为：

$$L(\boldsymbol{w})=\frac{1}{N}\|\phi(\boldsymbol{X})\boldsymbol{w}-\boldsymbol{y}\|^2$$

其中

$$\phi(X)=\begin{bmatrix}\phi(\boldsymbol{x}^{(1)})\\\vdots\\\boldsymbol{\phi}(\boldsymbol{x}^{(N)})\end{bmatrix}$$

最小化损失函数(求驻点=0)，最优参数为：

$$w^*=(\phi^T\phi)^{-1}\phi^Ty$$

这就是将原来线性回归中的X替换为$\phi=\phi(X)$的结果。

分类问题中同理。交叉熵变为：

$$L(\boldsymbol{W})=-\frac{1}{N}\sum_{l=1}^{N}\sum_{k=1}^{K}y_k^{(l)}\log[\mathrm{softmax}_k(\boldsymbol{\phi}(\boldsymbol{x}^{(l)})\boldsymbol{W})]$$

梯度变为：

$$\frac{\partial L(\boldsymbol{W})}{\partial\boldsymbol{w}_j}=\frac{1}{N}\sum_{l=1}^{N}\left(\mathrm{softmax}_j(\boldsymbol{\phi}^{(l)}\boldsymbol{W})-\boldsymbol{y}^{(l)}\right)\boldsymbol{\phi}^{(l)T}$$

同样是将原来的$x^{(l)}$替换为$\phi^{(l)}=\phi(x^{(l)})$的结果

过拟合

基函数的维数越高，就能更好的拟合训练数据。但是相对的，在测试集上的表现就会越来越差。

模型在**模式数据上**的表现称为这个模型的**泛化能力**。

> 模式数据是模型在训练过程中未见过的、但与训练数据具有**相似分布**的数据

定义

当验证集上的错误率随模型**复杂度提升而下降**时，称模型此时是**欠拟合**的；
当验证集上的错误率随模型**复杂度提升而提升**时，称模型此时是**过拟合**的。

K-fold 交叉验证

训练数据通常是**比较少**的，如果还划分一部分用于验证集，则没有足够的数据样本用于训练，此时可以采取K-fold 交叉验证。

也即把训练集分为 K 份，第 i 轮训练用第 i 份作为验证集，其他 K-1 份用于训练。

信息准则 (Information Criteria)

用于评估模型的**复杂度**。

Akaike Information Criteria (AIC阿卡克信息准则) :

$$\mathrm{AIC} = 2M - 2\log(L)$$

其中M是模型的**参数量**，L是该模型下的**最大似然**。一般，AIC越小，模型越好。

Bayesian Information Criteria (BIC,贝叶斯信息准则)

$$\mathrm{BIC} = M\log N - 2\log(L)$$

其中N是训**练数据数量**。

这两个标准一般只能用于概率模型中，因为需要用到对数似然。

> 似然函数表示在给定模型参数的情况下，观测数据出现的概率。假设我们有一个概率模型 $P(X|\theta)$，其中：
>
> - X 是观测数据，
> - θ 是模型的参数。
>
> 似然函数定义为：
>
> $$L(\theta) = P(X|\theta)$$
>
> 对于独立同分布的数据，似然函数可以写成：
>
> $$L(\theta) = \prod_{i=1}^{n} P(x_i|\theta)$$
>
> 其中 x_i 是第 i 个观测数据点。
> 由于似然函数通常是连乘的形式，直接计算可能会遇到数值下溢的问题（即结果太小，计算机无法表示）。因此，通常对似然函数取对数，将连乘转化为累加：
>
> $$\ln L(\theta) = \sum_{i=1}^{n} \ln P(x_i|\theta)$$
>
> 对数似然函数不仅简化了计算，还保持了似然函数的单调性，因此最大化对数似然等价于最大化似然函数。

正则化

过拟合一般是由高阶项对应的参数过大导致的。引入正则项来平衡这些影响

限制了备选空间范围,来降低模型复杂度,减少过拟合

> 正则化是机器学习中一种用于**防止模型过拟合,减少泛化误差**的技术

L_2正则化:

$$\tilde{L}(\boldsymbol{w}) = L(\boldsymbol{w}) + \lambda \|\boldsymbol{w}\|_2^2$$

其中λ是超参数，用于控制正则项的影响。它将参数**趋向于 0**。
正则化后得到的**模型曲线更平滑**。

L_1正则化：

$$\tilde{L}(\boldsymbol{w}) = L(\boldsymbol{w}) + \lambda \|\boldsymbol{w}\|_1$$

这种正则化可以让**参数变为 0** (而不仅仅是趋向 0),所以经常趋向于得到松弛解。

> 正则化通过添加惩罚项间接限制了模型参数的范围，类似于凸优化中的可行域限制

PCA

八股文

目的&应用

PCA（主成分分析）的主要目的是降维，通过将高维数据转换到低维空间，保留数据中的主要变化信息。具体目标包括：

1. **降维**：减少数据维度，降低**计算复杂度**。
2. **去噪**：通过**保留主要**成分，去除**噪声**和不重要的信息。
3. **可视化**：将高维数据降至2D或3D，便于可视化。
4. **特征提取**：生成**新的特征**，用于后续的机器学习任务。

假设

1. **线性关系**：数据的**主**成分是**原**始特征的**线性组合**。
2. **高方差包含重要信息**：方差大的方向包含更多有用信息。
3. **正交性**：**主成分**之间**相互正交**，确保成分之间无相关性。
4. **数据标准化**：通常需要**标准化数据**，使各**特征**具有**相同**的**尺度**。

目标（对应的最优化问题的理解）

PCA的核心目标是找到一组正交基（主成分），使得数据在这些基上的投影方差最大化。具体来说：

1. **第一主成分**：找到使投影方差最大的方向。
2. **后续主成分**：在已找到的主成分正交的条件下，继续寻找使剩余方差最大的方向。

PCA的目标可以形式化为一个最优化问题，具体步骤如下：

1. **数据准备**：
   - 假设数据矩阵为 X，其中每一行是一个样本，每一列是一个特征。
   - 假设数据已经中心化（即每个特征的均值为0）。
2. **目标函数**：
   - 对于第一主成分，目标是找到一个单位向量 $\mathbf{w}$（即 $\|\mathbf{w}\|_2 = 1$），使得投影后的方差最大化。
   - 投影后的数据为 $\mathbf{X}\mathbf{w}$，其方差为 $\text{Var}(\mathbf{X}\mathbf{w}) = \mathbf{w}^\top \mathbf{X}^\top \mathbf{X}\mathbf{w}$。
   - 因此，目标函数可以写成：

$$\max_{\mathbf{w}} \mathbf{w}^\top \mathbf{X}^\top \mathbf{X}\mathbf{w} \quad \text{subject to} \quad \|\mathbf{w}\|_2 = 1$$

3. **求解方法**：
   - 这个问题可以通过特征值分解或奇异值分解（SVD）来求解。
   - 具体来说，$\mathbf{X}^\top \mathbf{X}$ 是数据的协方差矩阵，对其进行特征值分解，得到特征值和特征向量。
   - 最大特征值对应的特征向量就是第一主成分 $\mathbf{w}_1$。
   - 第二主成分 $\mathbf{w}_2$ 是与 $\mathbf{w}_1$ 正交且使剩余方差最大的方向，依此类推。
4. **降维**：
   - 选择前 k 个主成分（即前 k 个特征向量），将数据投影到这 k 个主成分上，得到降维后的数据。

缺陷

1. **线性假设**：
   - PCA假设数据是线性相关的，如果数据具有**非线性**结构，PCA可能无法有效地捕捉数据的内在结构。

2. **信息损失**：
    - 虽然PCA保留了数据的大部分 variance，但在降维过程中仍有可能**丢失一些重要的信息**，特别是那些在较小主成分中包含的信息。
3. **解释性差**：
    - PCA生成的主成分是原始特征的线性组合，这使得它们的解释性较差，难以直接对应到**原始特征的含义**。
4. **计算复杂度**：
    - 对于大规模数据集，计算**协方差矩阵**和进行特征值分解的**计算成本较高**，可能不适用于实时或在线应用。
5. **对异常值敏感**：
    - PCA对异常值非常敏感，因为**协方差矩阵受异常值**的影响较大，这可能导致主成分的方向受到异常值的严重影响。

角度一——最小化重建误差

由于抛弃了一些维度，误差是客观存在的，但是希望重建后数据与原始数据的误差尽可能小。

如果给定一组**正交基**，即一组向量$u_1, u_2, \cdots, u_M$满足：

$$\boldsymbol{u}_i^T \boldsymbol{u}_j = \delta_{i,j} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$

在这组正交基张成的**子空间**中，某个向量x的**最佳近似**是：

$$\tilde{\boldsymbol{x}} = \sum_{i=1}^{M} \alpha_i \boldsymbol{u}_i = \sum_{i=1}^{M} (\boldsymbol{u}_i^T \boldsymbol{x}) \boldsymbol{u}_i$$

其中$\alpha_i = \boldsymbol{u}_i^T \boldsymbol{x}$是标量,表示x在ut方向的投影长度

> 因为是子空间,$\tilde{\boldsymbol{x}}$是x在子空间的正交**投影**

证明方法如下

$$\min \|\boldsymbol{x}-\tilde{\boldsymbol{x}}\|^2=\min \|\boldsymbol{x}-\sum_{i=1}^M \alpha_i \boldsymbol{u}_i\|^2=\min (\|\boldsymbol{x}\|^2-\sum_{i=1}^M \alpha_i \boldsymbol{u}_i^T \boldsymbol{x}+\sum_{i=1}^M \alpha_i^2)$$

这是一个凸优化问题，其解就是$\alpha_i=\boldsymbol{u}_i^T \boldsymbol{x}$.

> 最小化 $\|\mathbf{x}-\tilde{\mathbf{x}}\|^2$ 的过程实际上是在求正交投影，因为在欧几里得空间中，最近的点就是正交投影

接下来就是确定一组正交基，能够最小化这个最小的误差。
一般，我们处理$x-\overline{x}$,而不是直接处理x,其中$\bar{x}$是整个数据集的均值。

> 目的:平移中心点到原点,方便计算

$\widetilde{\boldsymbol{x}}^{(n)}=\sum_{i=1}^M \alpha_i^{(n)} \boldsymbol{u}_i$

$\alpha_i^{(n)}=\boldsymbol{u}_i^T\left(x^{(n)}-\overline{\boldsymbol{x}}\right)$

目标:**最小化**整个数据集的平均**重建误差**:

$$E=\frac{1}{N} \sum_{n=1}^N\|\left(x^{(n)}-\overline{\boldsymbol{x}}\right)-\widetilde{\boldsymbol{x}}^{(n)}\|^2=\frac{1}{N} \sum_{n=1}^N\|\left(x^{(n)}-\overline{\boldsymbol{x}}\right)-\sum_{i=1}^M \alpha_i^{(n)} u_i\|^2$$

展开二次项,之后代入$\alpha_i^{(n)}$

$E=$
$\frac{1}{N}\left(\sum_{n=1}^N\left\|\mathbf{x}^{(n)}-\overline{\mathbf{x}}\right\|^2-2 \sum_{n=1}^N \sum_{i=1}^M \alpha_i^{(n)}\left(\mathbf{x}^{(n)}-\overline{\mathbf{x}}\right)^T \mathbf{u}_i+\sum_{n=1}^N \sum_{i=1}^M\left(\alpha\right.\right.$
$=\frac{1}{N} \Sigma_{n=1}^N\|\boldsymbol{x}^{(n)}-\overline{\boldsymbol{x}}\|^2-\sum_{i=1}^M \boldsymbol{u}_i^T \frac{1}{N} \Sigma_{n=1}^N\left(\boldsymbol{x}^{(n)}-\overline{\boldsymbol{x}}\right)\left(\boldsymbol{x}^{(n)}-\overline{\boldsymbol{x}}\right)^T \boldsymbol{u}_i$

注意到协方差矩阵$\mathbf{S}=\frac{1}{N} \mathbf{X_c} \mathbf{X_c}^T$ 把前面写成矩阵$\mathbf{X}=\left[x^{(1)}, x^{(2)}, \ldots, x^{(N)}\right]$ M是特征个数

$$E=\frac{1}{N}\|X-\overline{X}\|_F^2-\frac{1}{N} \sum_{i=1}^M \boldsymbol{u}_i^T \mathbf{S} \boldsymbol{u}_i$$

> $$\|A\|_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n}|a_{ij}|^2}$$
>
> $\|X-\overline{X}\|_F^2$ 表示矩阵 $X-\overline{X}$ 各元素平方和。

可以证明，与其(最小化误差)**等价**的约束问题：

$$\max_{\boldsymbol{u}_i}\sum_{i=1}^{M}\boldsymbol{u}_i^T\boldsymbol{S}\boldsymbol{u}_i \quad \text{s.t.}\boldsymbol{u}_i^T\boldsymbol{u}_j=\delta_{i,j}$$

解是$\boldsymbol{S}$从大到小排序前M个特征向量。由于$S\in\mathbb{R}^{D\times D}$是实对称矩阵，所以其一定存在$D$个相互正交的特征向量。

角度二——最大化投影方差

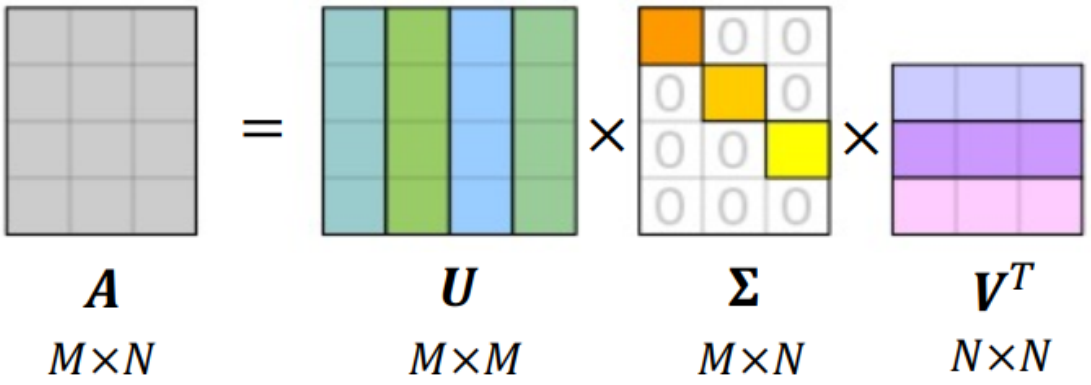
另一个角度是最大化重建数据的方差。可以想象，把面上数据投影到一条线上，如果所有的数据都集中**在线上某个点附近**，则几乎无法区分数据，携带的信息**很少**；如果数据比较**松散地分布在较大的范围内**，容易区分出每个点，携带的信息就比**较多。**

由方差的定义，可以写出在某个方向u_j上的整个数据集方差：

$$\begin{aligned}\text{var}&=\frac{1}{N}\sum_{n=1}^{N}\left(\boldsymbol{u}_j^T(\boldsymbol{x}^{(n)}-\overline{\boldsymbol{x}})\right)^2\\&=\boldsymbol{u}_j^T\frac{1}{N}\sum_{n=1}^{N}(\boldsymbol{x}^{(n)}-\overline{\boldsymbol{x}})(\boldsymbol{x}^{(n)}-\overline{\boldsymbol{x}})^T\boldsymbol{u}_j\\&=\boldsymbol{u}_j^T\boldsymbol{S}\boldsymbol{u}_j\end{aligned}$$

每个方向都是这个表达式,最大化总的

角度三——奇异值分解(不用管)



一个矩阵$A \in \mathbb{R}^M \times N$可以做奇异值分解：

$$A = U\Sigma V^T$$

其中$U = [u_1, u_2, \cdots, u_M] \in \mathbb{R}^{M \times M}$和$V = [v_1, v_2, \cdots, v_N] \in \mathbb{R}^{N \times N}$是正交矩阵。$u_i$和$v_i$分别是$AA^T$和
$A^T A$的特征向量。Σ是一个对角矩阵，**对角线上的元素是奇异值(**奇异值 σ_i 是特征值的平方根**)**。

由于Σ只有对角线上有非零值，原矩阵A可以表达为：

$$A = \sum_{i=1}^{r} \Sigma_{i,i} \boldsymbol{u}_i \boldsymbol{v}_i^T = \boldsymbol{U}'\boldsymbol{\Sigma}'\boldsymbol{V}'$$

矩阵A的SVD分解中的向量u_i是矩阵AA^T关于其第i大特征值的特征向量。

令$A = \widetilde{X}$，其中$\widetilde{X} \triangleq [x^{(1)} - \overline{x}, x^{(2)} - \overline{x}, \cdots, x^{(N)} - \overline{x}]$，我们可以看到

$$\widetilde{X}\widetilde{X}^T = \sum_{n=1}^{N} \left(x^{(n)} - \overline{x}\right)\left(x^{(n)} - \overline{x}\right)^T = N \cdot S$$

> 样本数量N;矩阵S表示的是数据的协方差矩阵$S = \frac{1}{N}\sum_{n=1}^{N}(x^{(n)} - \overline{x})(x^{(n)} - \overline{x})^T$

这意味着$\widetilde{X}\widetilde{X}^T$的特征向量与矩阵$S$的特征向量相同。

通过对$\widetilde{X}$进行SVD分解，可以直接获得数据$\{x^{(n)}\}_{n=1}^{N}$的主方向，这些主方向就是SVD分解中**左矩阵U的列向量。**

SVM

线性分类器的决策边界

之前的线性分类器的基础上加上一个偏置项，交叉熵损失函数变为：

$$L(\boldsymbol{w}, b) = -y \log(\sigma(\boldsymbol{w}^T \boldsymbol{x} + b)) - (1 - y) \log(1 - \sigma(\boldsymbol{w}^T \boldsymbol{x} + b))$$

损失函数最小值为 0,记使其为 0 的最优的w^*, b^*,则$L(\boldsymbol{w}, b) = 0$表示的超平面为：$\{w|w^{*T}x + b^* = 0\}$

这个超平面：

- 垂直于w^*
- 到原点的距离是$-b^*/||w^*||$

对于一个点 x_0，其到超平面 $w^T x + b = 0$ 的距离 D 可以通过以下公式计算：

$$D = \frac{|w^T x_0 + b|}{\|w\|}$$

其中：

- $w^T x_0 + b$ 是点 x_0 到超平面的函数间隔。
- $\|w\|$ 是法向量 w 的范数（即 $\|w\| = \sqrt{w_1^2 + w_2^2 + \cdots + w_n^2}$）。

线性最大间隔分类器

用点到直线(决策超平面)的距离表示间隔

“间隔”取决于**距离超平面最近**的那些数据点 $\mathrm{Margin} = \min_l \frac{y^{(l)}(\boldsymbol{w}^T \boldsymbol{x}^{(l)} + b)}{\|w\|}$

原问题

为了将其转化为凸优化问题，最小化 $\frac{1}{2}\|w\|^2$，并加入约束条件 $y^{(l)}(w^T x^{(l)} + b) \geq 1$：

$$\begin{aligned} &\min_{w,b} \quad \frac{1}{2}\|w\|^2 \\ &\text{s.t.} \quad y^{(l)}(w^T x^{(l)} + b) \geq 1, \quad \forall l = 1, 2, \ldots, n \end{aligned}$$

> 约束条件可以由决策平面上方的y=1,下方的y=-1推出

预测分类结果$\hat{y} = \text{sign}(\boldsymbol{w}^{*T}\boldsymbol{x} + b^*)$

> $$\text{符号函数：}\text{sign}(x) = \begin{cases} -1, & \text{if } x < 0 \\ 0, & \text{if } x = 0 \\ 1, & \text{if } x > 0 \end{cases}$$

对偶问题

> 引入拉格朗日乘数法，构造拉格朗日函数：
>
> $$\mathcal{L}(w, b, \alpha) = \frac{1}{2}\|w\|^2 - \sum_{l=1}^{n} \alpha_l [y^{(l)}(w^T x^{(l)} + b) - 1]$$
>
> 对 w 和 b 求偏导并设偏导为零：
>
> $$\frac{\partial \mathcal{L}}{\partial w} = w - \sum_{l=1}^{n} \alpha_l y^{(l)} x^{(l)} = 0 \quad \Rightarrow \quad w = \sum_{l=1}^{n} \alpha_l y^{(l)} x^{(l)}$$
>
> $$\frac{\partial \mathcal{L}}{\partial b} = -\sum_{l=1}^{n} \alpha_l y^{(l)} = 0 \quad \Rightarrow \quad \sum_{l=1}^{n} \alpha_l y^{(l)} = 0$$
>
> 将 w 的表达式代入拉格朗日函数：

$$\mathcal{L}=\sum_{l=1}^{n}\alpha_l-\frac{1}{2}\sum_{l=1}^{n}\sum_{k=1}^{n}\alpha_l\alpha_k y^{(l)}y^{(k)}x^{(l)}\cdot x^{(k)}$$

对偶问题为：

$$\begin{aligned}\max_{\alpha}\quad & \sum_{l=1}^{n}\alpha_l-\frac{1}{2}\sum_{l=1}^{n}\sum_{k=1}^{n}\alpha_l\alpha_k y^{(l)}y^{(k)}x^{(l)T}\cdot x^{(k)}\\ \text{s.t.}\quad & \alpha_l\geq 0,\quad \forall l=1,2,\dots,n\\ & \sum_{l=1}^{n}\alpha_l y^{(l)}=0\end{aligned}$$

预测分类结果$\hat{y}=\operatorname{sign}\left(\sum_{l=1}^{N}\alpha^{(l)}y^{(l)}\boldsymbol{x}^{(l)T}\cdot\boldsymbol{x}+b^*\right)$

软线性最大间隔分类器

如果数据**不是线性可分**的，就需要引入**软间隔。**

约束变为： $y^{(l)}\left(\boldsymbol{w}^T\boldsymbol{x}^{(l)}+b\right)\geqslant 1-\xi^{(l)}$

其中$\xi^{(l)}\geqslant 0$是一个松弛变量，用于表示最多能容忍的误差。我们希望同时最小化间隔和误差，所以优化目标变为：

$$\min_{\boldsymbol{w},b,\boldsymbol{\xi}}\left(\frac{1}{2}\|\boldsymbol{w}\|^2+\lambda\sum_{l=1}^{N}\xi^{(l)}\right)$$

其中λ是惩罚因子，它表示了误差的权重。λ取无穷时，就退化到上一个线性最大间隔分类器
同样可以利用拉格朗日乘子法求解。最终的分类器的形式仍然是：

$$\hat{y}=\operatorname{sign}\left(\sum_{l=1}^{N}\alpha^{(l)}y^{(l)}\boldsymbol{x}^{(l)T}\boldsymbol{x}+b^*\right)$$

SVM八股文

等高线&向量场

等高线是连接具有**相同函数值的点的曲线。**在机器学习中，等高线常用于**可视化损失函数或决策函数的形状。**

向量场是空间中**每个点都对应一个向量的场。**在机器学习中，向量场常用于**表示梯度或方向导数。**

优缺点

优点

1. **泛化能力强**：SVM基于**结构风险最小化**原则。在机器学习中，我们不仅要考虑训练误差（经验风险），还要考虑模型在未知数据上的泛化能力（结构风险）。SVM通过最大化间隔来实现结构风险最小化。
2. **灵活处理非线性问题**：通过**核技巧**，SVM能够处理非线性分类问题，常用核函数包括多项式核和径向基函数（RBF）核。
3. **鲁棒性强**：在实际数据中，往往存在一些噪声点，SVM通过引入**软间隔**的概念来处理这种情况。软间隔允许一些数据点违反间隔约束，即可以有少量的误分类。

缺点

1. **计算复杂度高**：训练时间复杂度为 $O(n^2)$ 到 $O(n^3)$，在大规模数据集上**训练较慢。**
2. **参数选择敏感**：核函数和正则化参数 λ 的选择对模型性能影响较大，调优过程复杂。
3. **难以解释**：特别是使用**非线性核**时，模型决策过程不易解释。

核技巧

上述的分类器仍然是线性的，引入基函数后，可以得到**非线性**的分类器：

$$\hat{y} = \text{sign}\left(\sum_{l=1}^{N} \alpha^{(l)} y^{(l)} \phi(\boldsymbol{x}^{(l)})^T \phi(\boldsymbol{x}) + b^*\right)$$

越高维的空间中，数据**点之间的距离越大**，也就**越容易找到一个可行的"超平面"**,所以我们希望变换后的特征空间$\phi(x)$尽可能高维。但是也不能过高，因为维度越高，计算量也就越大。

整个问题的难点在于计算$\phi(x^{(l)})^T\phi(x)$,这在高维空间中的计算量是非常大的。

> 核技巧无需将原本的低维向量转为高维再做内积,而是可以**直接算出两个低维向量在高维的内积**

核函数是一个包含两个变量的函数$K(x,x')$,可以表示为两个向量的内积:

$$K(\boldsymbol{x},\boldsymbol{x}')=\phi(\boldsymbol{x})^T\phi(\boldsymbol{x}')$$

Mercer 定理

如果一个函数$K(x,x')$是对称正定的，也即

$$\iint K(\boldsymbol{x},\boldsymbol{x}')f(\boldsymbol{x})f(\boldsymbol{x}')d\boldsymbol{x}d\boldsymbol{x}'\geqslant 0,\forall f(\cdot)\in L^2$$

则一定存在一个特征空间$\phi(x)$,使得$K(\boldsymbol{x},\boldsymbol{x}')=\boldsymbol{\phi}(\boldsymbol{x})^T\boldsymbol{\phi}(\boldsymbol{x}')$

核函数的功能

核函数的核心功能是**隐式地将数据映射到高维空间**，而无需显式地计算高维空间中的特征表示。具体来说，核函数通过计算两个样本在原始特征空间中的相似性（内积），间接实现了高维空间中的计算

核函数种类&特点

1. 线性核函数（Linear Kernel)

- **公式**:

  $$K(x,y)=x^Ty$$

- **功能**:

- 适用于线性可分的数据。
- 不进行任何非线性变换，直接在**原始特征空间**中寻找最优超平面。
- 计算简单，速度快。

2. 多项式核函数（Polynomial Kernel）

- **公式**：

$$K(x, y) = (x^T y + c)^d$$

  - 其中，c 是常数，d 是多项式的**阶数**。
- **功能**：
  - 适用于**非线性可分**的数据。
  - 通过多项式变换将数据映射到**高维空间**。
  - 阶数 d 的选择影响模型的复杂度和拟合能力。

3. 高斯核函数（Gaussian Kernel，也称为RBF核函数 径向基函数核）

- **公式**：

$$K(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right)$$

  - 其中，σ 是高斯核的带宽参数。
- **功能**：
  - 适用于高度非线性的数据。
  - 通过高斯变换将数据映射到**无限维**空间。
  - 具有强大的拟合能力，但需要谨慎选择 σ 以避免过拟合。

引入核函数的原因

传统的线性分类器（如线性 SVM）只能处理线性可分的数据，即数据可以通过一个超平面完全分开。

在实际问题中，许多数据集是**非线性可分的**，即无法通过一个超平面将不同类别的样本完全分开。

核函数通过将数据从**原始特征**空间映射到一个**更高维**的空间，使得在原始空间中非线性可分的数据**在高维空间中变得线性可分**。

优化算法与训练技术

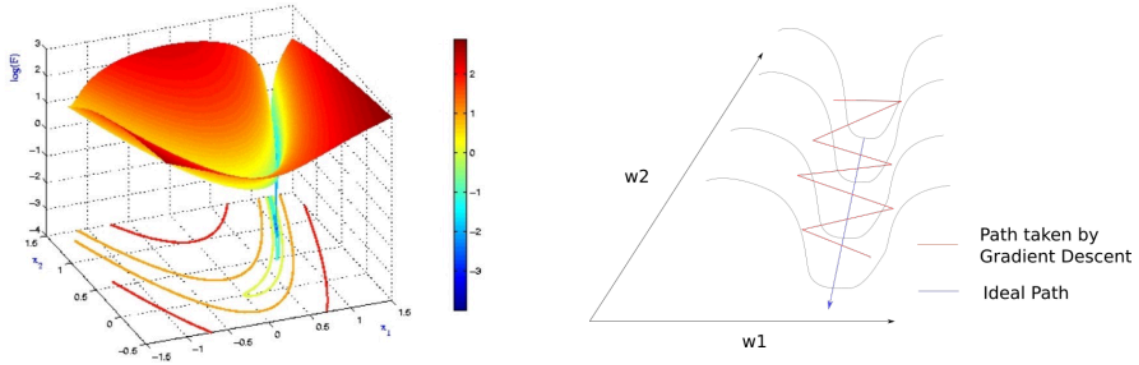
随机梯度下降(SGD)

使用部分样本计算损失，然后继续梯度下降

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta \nabla L(\boldsymbol{x}, y; \boldsymbol{w}_t)$$

其中η是学习率,$\nabla L(\boldsymbol{x}, y; \boldsymbol{w}_t)$是**部分样本**的平均损失。

SGD 的问题是，非常容易**陷入局部最优**，且解空间中一些**病态弯曲**（pathological curvature）造成**收敛缓慢**，如下图：



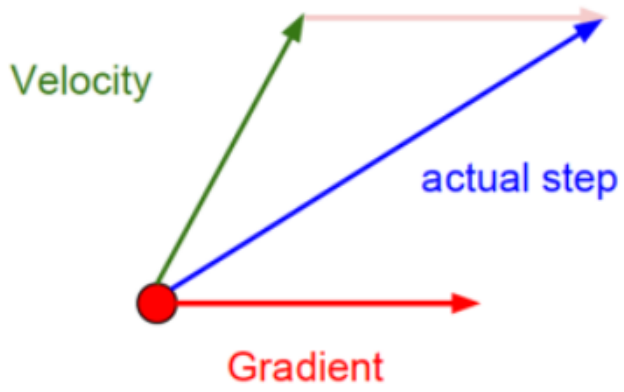
SGD 的路线会是图中的折线，而不是我们期望的直线。

SGD+Momentum

SGD+Momentum 是在 SGD 的基础上引入了动量项，其更新公式为：

$$\boldsymbol{v}_t = \rho \boldsymbol{v}_{t-1} + \nabla L(\boldsymbol{x}, y; \boldsymbol{w}_t)$$
$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta \boldsymbol{v}_t$$

其中$\rho \in (0, 1)$是动量因子，常选择的值是 0.9、0.95、0.99。



这里的动量项与物理学中的动量类似。可以想象，一个小球从坡上滚下，首先由于有动量，下坡速度越来越快；其次如果遇到一个上坡，积累的速度有益于克服上坡的阻力帮其脱出。也就是说，引入动量项可以**加速收敛，避免陷入局部最优。**

RMSProp

Root Mean Square Propagation,均方根传播。

RMSProp 的核心思想是对**梯度**进行**指数加权移动平均**，并根据历史梯度的大小动态调整每个参数的学习率

其更新公式为：

$$\boldsymbol{s}_t = \rho \boldsymbol{s}_{t-1} + (1-\rho)(\nabla L(\boldsymbol{x}, y; \boldsymbol{w}_t))^2$$
$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta \nabla L(\boldsymbol{x}, y; \boldsymbol{w}_t) \oslash \sqrt{\boldsymbol{s}_t}$$

其中$\oslash$表示两个相同形状的矩阵**对应元素相除**，s_t是之前**梯度的平方的加权。**

$\oslash \sqrt{s_t}$这个操作相当于将加权梯度进行了归一化，拉到相同的水平，这样可以避免病态弯曲(两边不会那么陡峭)。

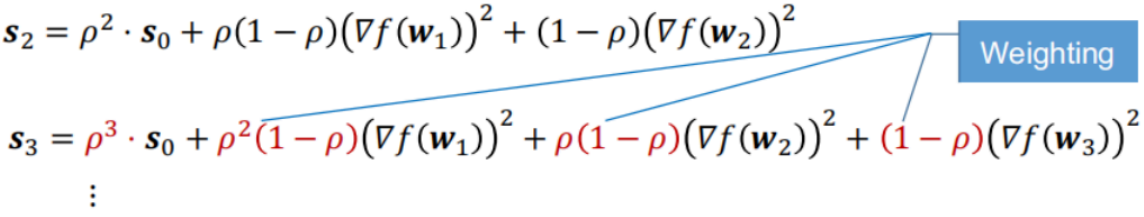
将更新式子展开后即可发现，在衰减因子的作用下，**时间越早的梯度对更新的影响越小。**

$$s_1 = \rho \cdot s_0 + (1-\rho)\left(\nabla f(w_1)\right)^2$$

$$s_2 = \rho^2 \cdot s_0 + \rho(1-\rho)\left(\nabla f(w_1)\right)^2 + (1-\rho)\left(\nabla f(w_2)\right)^2$$

$$s_3 = \rho^3 \cdot s_0 + \rho^2(1-\rho)\left(\nabla f(w_1)\right)^2 + \rho(1-\rho)\left(\nabla f(w_2)\right)^2 + (1-\rho)\left(\nabla f(w_3)\right)^2$$

$$\vdots$$



RMSProp 的优点

1. **自适应学习率**：每个参数的学习率根据梯度的大小**动态调整**，适合处理**非平稳**目标函数。
2. **缓解梯度震荡**：在**梯度方向变化较大**的情况下，RMSProp 可以**平滑更新**，减少震荡。
3. **适合稀疏梯度**：在处理**稀疏数据**时（如自然语言处理），RMSProp 表现良好。

RMSProp 的缺点

1. **依赖超参数**：衰减率 ρ 和初始学习率 η 需要**手动调参。**
2. **可能陷入局部最优**：在某些情况下，RMSProp 可能会过早收敛到局部最优。

Adam

Adaptive Moment Estimation 的缩写，是一种结合了 Momentum 和 RMSProp 的优化算法。其更新公式为：

$$\begin{aligned}
\boldsymbol{m}_t &= \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1)\nabla L(\boldsymbol{x}, y; \boldsymbol{w}_t) \\
\boldsymbol{s}_t &= \beta_2 \boldsymbol{s}_{t-1} + (1-\beta_2)\left(\nabla L(\boldsymbol{x}, y; \boldsymbol{w}_t)\right)^2 \\
\boldsymbol{w}_{t+1} &= \boldsymbol{w}_t - \eta \hat{\boldsymbol{m}}_t \oslash \sqrt{\boldsymbol{s}_t}
\end{aligned}$$

其中m_t是过**去梯度的移动平均。**当$\beta_1 = 0$时，Adam 就退化为了 RMSProp。

参数的一组常见设置是$\beta_1 = 0.9, \beta_2 = 0.999, \eta = 0.001$

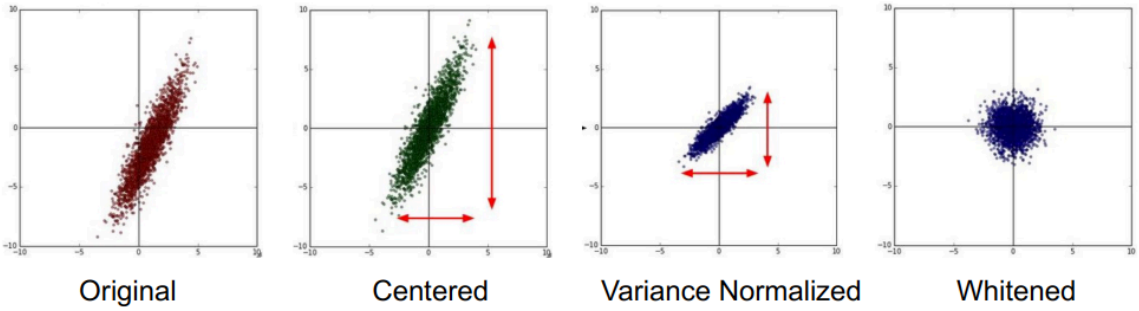
1. **自适应学习率**：每个参数的学习率根据梯度的一阶矩和二阶矩动态调整，适合处理**非平稳**目标函数。
2. **动量加速**：结合动量法，Adam 在梯度方向上**加速收敛，减少震荡。**

3. **偏差修正**：初始时刻的偏差修正机制使得**更新更加稳定。**
4. **高效且鲁棒**：在大多数深度学习任务中表现良好，几乎不需要调参。

训练技术

一些经常使用的数据处理方法包括：

- 中心化：例如同时**减去**均值
- 归一化：每个维度的数据都**除以**标准差
- 白化（Whitening）数据:消除数据中的冗余信息（如特征之间的相关性），并使数据分布更加均匀



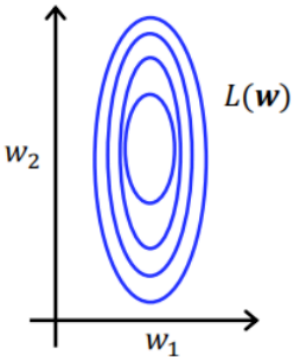
可能的问题

- 当特征值的尺度差异较大时，损失函数 $L(w)$ 的等高线在参数空间中会呈现“拉长的椭圆”形状。
    - 这是因为不同维度的梯度（导数）变化速率不同（例如某些维度的梯度较大，另一些较小）。
- 这种情况下，梯度下降（gradient descent）优化算法在靠近全局最优点时，**路径会“震荡”且收敛缓慢。**

解决方法

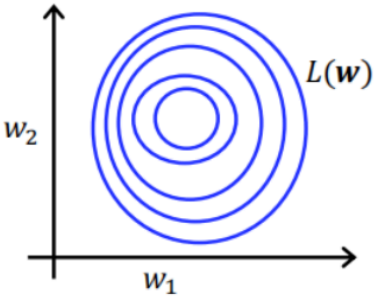
- 通过预处理，将所有特征值**缩放到相似的尺度**（通常通过**中心化和归一化**）。
- 当特征值的尺度相似时，损失函数 $L(w)$ 的等高线会变得更接近于“圆形”。
- 这样一来，梯度下降法的每次迭代都会更稳定，优化路径更直，收敛速度更快。

Obviously, the cost function $L(\boldsymbol{w})$ decreases at different rates for different dimensions



This results in the *slow convergence* of gradient descent. *Why??*

If the features are preprocessed to similar scales, then the loss function looks like



初始化

从**固定高斯分布随机取样**来初始化神经网络中各层的**权重**，尤其是在较浅的神经网络(层数小于8)时，会取得较好
的效果。

一个常用的高斯分布是：

$$w \sim N(0, 0.01^2)$$

如果激活函数使用的是ReLU，则可以使用截断的高斯分布，其是通过将标准高斯分布中**超出特定范围的值的概率设
为零**，并对剩余部分进行归一化处理得到的。

$$w \sim N_T(0, 0.01^2)$$

如果神经网络较深，可以通过对以下高斯分布进行采样，来对计算出第i层节点值要用到的参数初始化

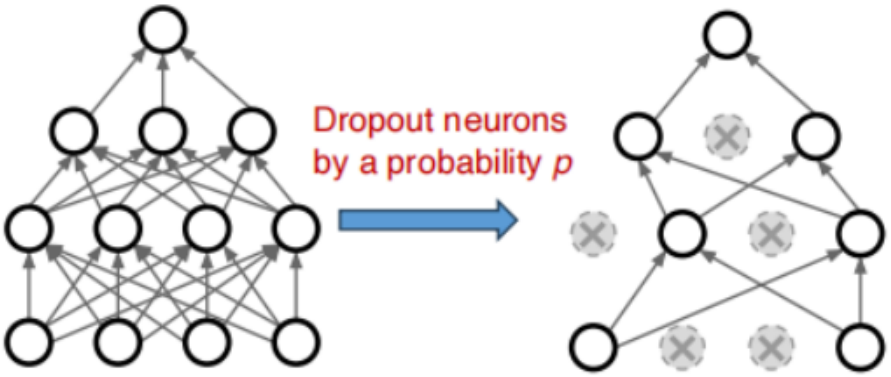
$$w \sim N(0, \sigma_i^2)$$

其中σ_i是第$i-1$层的神经元**个数的倒数的平方根。**

Dropout

一种用于**防止神经网络过拟合**的**正则化技术**

它的核心思想是在训练过程中**随机“丢弃”**（即暂时移除）神经网络中的一部分神经元，从而**减少**神经元之间的**复杂共适应性**（co-adaptation），使**网络更加鲁棒。**



训练阶段的 Dropout

- 核心思想：
  - 在每次**前向**传播中，随机将一部分神经元的**输出设置为 0。**
  - 被丢弃的神经元在**本次**训练迭代中**不参与**计算。
- 丢弃概率 p：
  - 丢弃神经元的概率 p 是一个超参数，通常设置为 0.5（即每次训练时，大约 50% 的神经元会被丢弃）。
  - 例如，如果 $p = 0.5$，则每个神经元有 50% 的概率被丢弃。
- 作用：
  - Dropout 可以有效缓解深度神经网络中的过拟合问题。
  - 通过随机丢弃神经元，Dropout 迫使网络**不依赖于某些特定的神经元**，从而提高模型的**泛化能力。**

测试阶段的 Dropout

- 核心思想：
  - 在测试阶段，模型的输出应该是确定的（即对于相同的输入，输出应该是固定的）。
  - 因此，测试阶段不使用 Dropout，所有神经元都参与计算。
- 缩放操作：
  - 为了保持训练和测试时的一致性，测试阶段需要对每个神经元的输出进行**缩放**。
  - 每层的**输入**也要加上 Dropout 的概率

$$h_{l+1} = \sigma(p\boldsymbol{W}_l\boldsymbol{h}_l)$$

- - 假设某个神经元的输出为 h，则测试时的**输出**为：

$$h_{\text{test}} = h \cdot (1-p)$$

  - 这种缩放操作确保了**训练和测试**时每个神经元的**输入范围一致**。

批量归一化BN

Batch Normalization, BN。

对**每一层的输入**进行**标准化处理**，使其均值为 0，方差为 1。这样可以**减少内部协变量偏移**（Internal Covariate Shift），即网络层输入分布的变化，从而**加速训练并提高模型的稳定性**。

一种解决方法就是引入 BN 层，将某一个隐藏层的输出$\{\boldsymbol{h}_l^{(i)}\}_{i=1}^N$变为标准正态分布：

$$\hat{\boldsymbol{h}}_l^{(i)} = (\boldsymbol{h}_l^{(i)} - \boldsymbol{\mu}_l) \oslash \boldsymbol{\sigma}_l$$

其中μ_l, σ_l^2分别是$h_l^{(i)}$的均值和标准差。进一步地，可以引入两个可学习的参数γ_l, β_l，来调整归一化后的值：

$$\tilde{\boldsymbol{h}}_l^{(i)} = \gamma_l \odot \hat{\boldsymbol{h}}_l^{(i)} + \beta_l$$

其中⊙是对应元素相乘。

使用所有样本计算均值和标准差代价太大，一般用某**一个小批次的均值和方差来近似**.

在训练阶段，使用当前 mini-batch 的均值和方差来标准化输入。然而，在测试阶段，这种方法的两个主要问题是：

- 输出依赖于 mini-batch：如果测试时仍然使用当前 mini-batch 的均值和方差，模型的输出会**依赖于具体的** mini-batch，导致不一致的结果。
- mini-batch 大小可能为 1：在测试时，mini-batch 的大小可能为 1（即单个样本），此时无法准确计算均值和方差。

一般有两个**解决方法**

方法 1：计算精确的均值和方差

- 使用**整个**训练数据集的均值和方差，而不是 mini-batch 的均值和方差。
- 具体公式：

  $$\mu_p = \frac{1}{N} \sum_{i=1}^{N} \hat{R}_p^{(i)}$$

  $$\sigma_p^2 = \frac{1}{N} \sum_{i=1}^{N} (\hat{R}_p^{(i)} - \mu_p)^2$$

  其中：

  - N 是训练数据集的大小。
  - $\hat{R}_p^{(i)}$ 是第 i 个样本在某一层的输出。
- 优点：计算精确，结果稳定。
- 缺点：需要遍历整个训练数据集，计算成本较高。

方法 2：使用**移动平均**（Moving Average）

- 在训练阶段，通过移动平均的方式逐步估计均值和方差，并在测试阶段使用这些估计值。
- 具体步骤：
  1. 在训练时，记录每个 mini-batch 的均值和方差。
  2. 使用指数移动平均（Exponential Moving Average, EMA）更新全局的均值和方差：

$$\mu_{\text{global}} = \alpha \cdot \mu_{\text{global}} + (1 - \alpha) \cdot \mu_{\text{batch}}$$

$$\sigma^2_{\text{global}} = \alpha \cdot \sigma^2_{\text{global}} + (1 - \alpha) \cdot \sigma^2_{\text{batch}}$$

     其中，α 是平滑系数（通常接近 1，如 0.9 或 0.99）。
  3. 在测试阶段，直接使用训练时估计的全局均值和方差。
- 优点：计算高效，适合在线学习或大规模数据集。
- 缺点：估计值可能不如精确计算准确

最后，整个BN层的计算过程可以写为：

$$\text{BN}(\boldsymbol{h}_l) = \gamma_l \odot (\boldsymbol{h}_l - \boldsymbol{\mu}_l) \oslash \boldsymbol{\sigma}_l + \boldsymbol{\beta}_l$$

几种归一化对比

1. Batch Normalization

Batch Normalization 是最常见的一种归一化方法，主要对**批次中**的数据进行归一化。它的作用是加速收敛、减小内部协变量偏移（Internal Covariate Shift），并起到一定的正则化效果。

Batch Normalization 是对 **批次数据** 的归一化，在训练和测试中表现可能有所不同，因为测试时通常使用训练中的移动平均均值和方差。

2. Layer Normalization

Layer Normalization 通常用于 **Recurrent Neural Networks (RNNs)** 或 Transformer 中，与 Batch Normalization 的主要区别在于它对 **样本数据的每一层** 进行归一化。

优势：

- Layer Normalization 对批次大小不敏感，适合于小批次训练或序列任务（如 RNN）。
- 在训练和测试阶段行为一致。

---

3. Instance Normalization

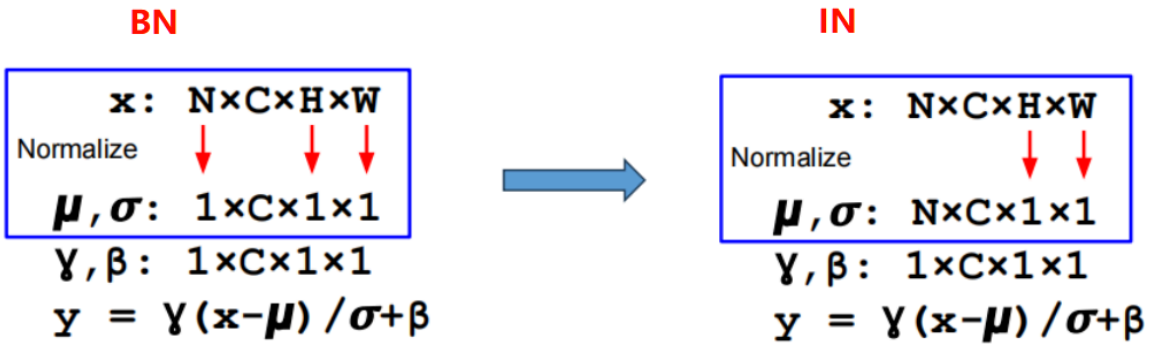
Instance Normalization 常用于风格迁移（Style Transfer）等任务，注重每个样本的独立性。

优势：

- Instance Normalization 的行为与测试和训练一致，非常适合处理每个样本的独立性要求（如图像风格）。

N是样本数量,D是特征维度,**C**: 通道数，H 和 W 分别表示卷积神经网络中特征图（Feature Map）的高度和宽度



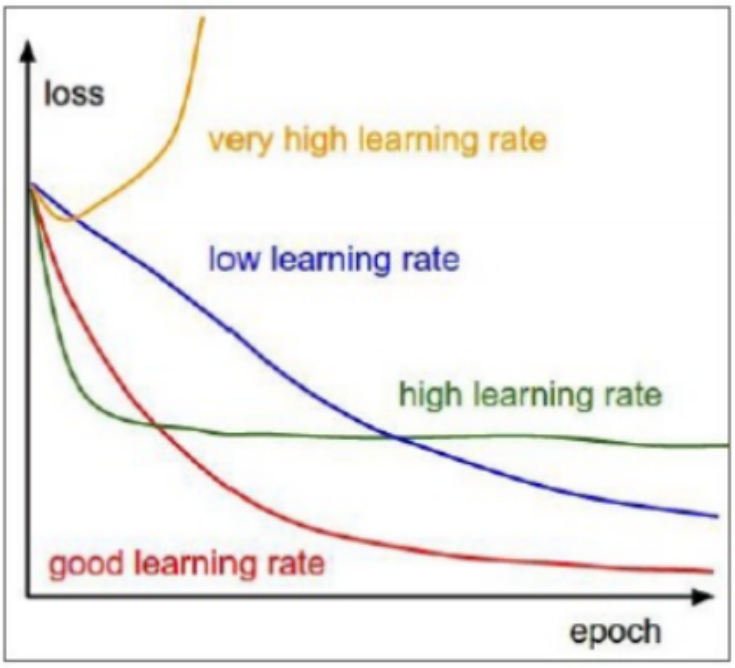


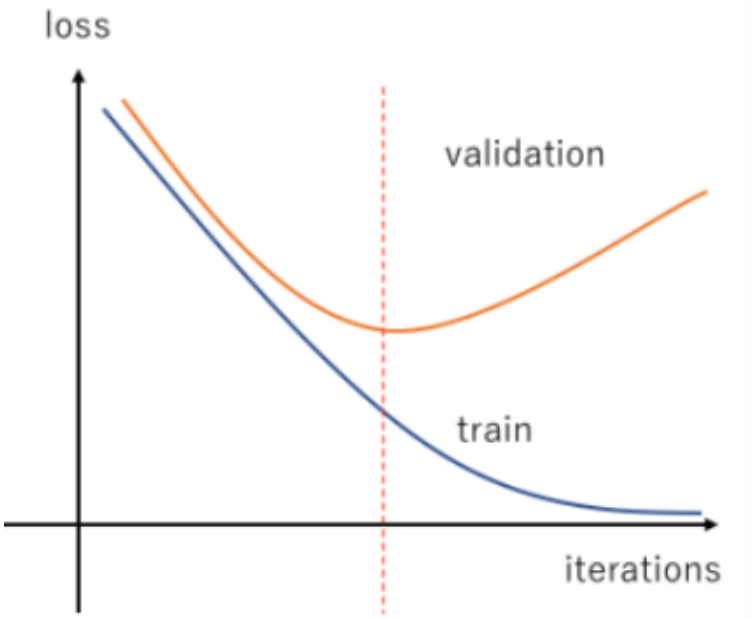
超参数调整

学习率等超参数的调整也不是盲目的，有一些一般性的方法：

1. 首先尝试**在许多问题上表现优异**的超参数**(经验)**
2. 根据**训练结果**逐渐来调整

训练损失曲线暗示了良好的学习率的可能范围　　在训练集和验证集上的性能之间的关系表明，该模型过拟合





决策树

基本概念

决策树中：

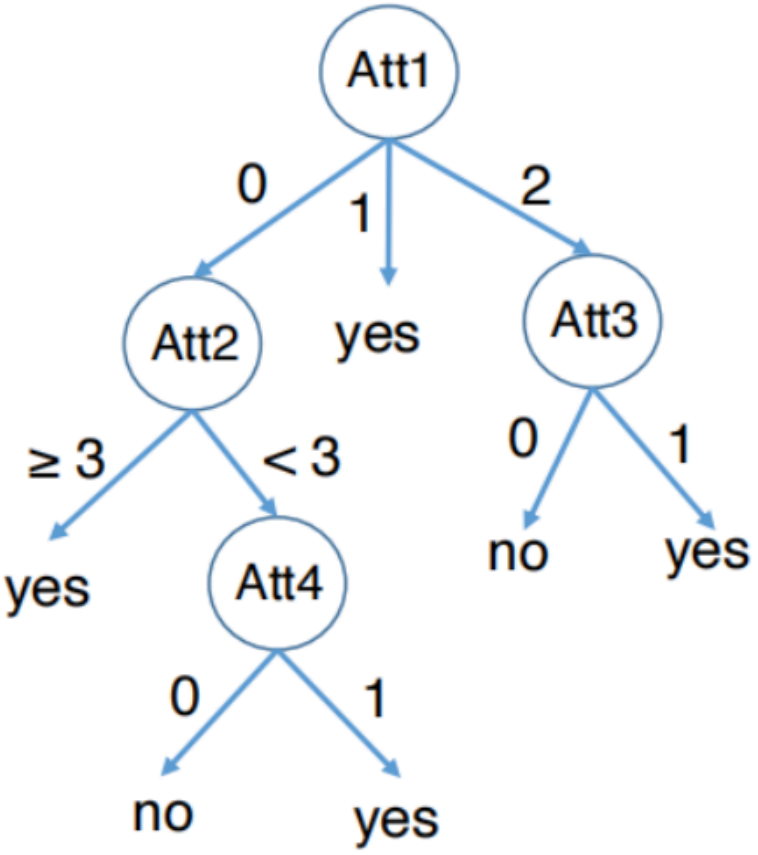
1. 根节点和内部节点表示一个**属性**（特征）

2. 叶子节点对应一个（最终被划分为的）**类别**
3. 边对应某个**属性值**

利用决策树进行分类，就是从根节点开始，根据属性值逐步向下，直到到达叶子节点。**叶子节点的类别**就是最终的**分类结果。**

学习出一个好的决策树，要考虑：

- 每个节点上用**哪个属性**进行划分（可以重复使用属性）
- 从某个节点如何引入边（如何**划分属性值**）
- **何时停止**拓展决策树



选择属性标准

从最明显的特征入手，或者说从“包含最多信息”的特征入手，

熵 (Entropy)

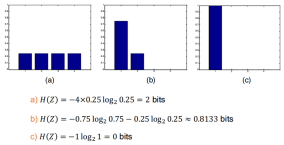
对随机变量的不确定性的度量。

给定一个随机变量$Z \sim p(z)$,则其熵是

$$H(Z) = -\sum_{z \in \mathcal{Z}} p(z) \log p(z)$$

其中$\mathcal{Z}$是随机变量Z可能取值的集合。

熵越大，变量取值的不确定性越大，分布越平坦（均匀）



条件熵

在给定一个随机变量Y的情况下，另一个随机变量Z的熵：

$H(Z|Y=y) = -\sum_{z \in \mathcal{Z}} p(z|y) \log p(z|y)$

$$H(Z|Y) = \sum_{y \in \mathcal{Y}} p(Y=y) H(Z|Y=y)$$

我们总有$H(Z) \geqslant H(Z|Y)$,可以这样直观的理解：获取额外信息 (Y的取值)后，把握一般都会变大，对Z的不
确定性会减小。

信息增益

Y带来的信息增益$IG(Y)$,是知道随机变量Y取值，使另一个随机变量Z的**熵减少的程度**：

$$IG(Y) = H(Z) - H(Z|Y)$$

分裂信息

分裂信息衡量的是特征 A 对数据集 D 的**划分程度**。对于一个特征 A，将数据集 D 划分为 k 个子集 $D_1, D_2, \ldots, D_k$

- 用于计算增益率，避免信息增益对**多值特征的偏好**。

分裂信息定义为：

$$\text{Split_Info}(D, A) = -\sum_{j=1}^{k} \frac{|D_j|}{|D|} \cdot \log_2 \left(\frac{|D_j|}{|D|} \right)$$

其中：

- $|D_j|$ 是子集 D_j 的样本数量；
- $|D|$ 是数据集 D 的总样本数量。

增益率

增益率是信息增益与分裂信息的比值：

$$\text{Gain_Ratio}(D, A) = \frac{IG(A)}{\text{Split_Info}(D, A)}$$

其中：

- $IG(A)$ 是特征 A 的信息增益；
- $\text{Split_Info}(D, A)$ 是特征 A 的分裂信息。

为什么要提出增益率?

信息增益在选择特征时有一个明显的**缺点**：它倾向于选择取**值较多的特征**。

这是因为取值较多的特征通常能够将数据集**划分成更多的子集**，从而使得每个子集的纯度更高，信息增益更大。

然而，这种划分可能会导致**过拟合**，尤其是在特征取值较多但每个取值对应的样本数量较少的情况下。

基尼值/系数

基尼值衡量一个数据集中**样本类别**的**纯度**。基尼值**越小**，数据集**越纯(类别均匀)**。

对于一个包含 C 个类别的数据集 D，基尼值定义为：

$$\text{Gini}(D) = 1 - \sum_{i=1}^{C} p_i^2$$

其中：

- p_i 是数据集中第 i 类样本的比例。

反映了D中随机抽取两个样本,标签不同的概率

基尼指数

基尼指数用于衡量某个**特征**对数据集的**分割效果。**基尼**指数越小**，分割后的子集**越纯。**

对于一个特征 A，将数据集 D 划分为 k 个子集 $D_1, D_2, \ldots, D_k$，基尼指数定义为：

$$\text{Gini_Index}(D, A) = \sum_{j=1}^{k} \frac{|D_j|}{|D|} \cdot \text{Gini}(D_j)$$

其中：

- $|D_j|$ 是子集 D_j 的样本数量；
- $|D|$ 是数据集 D 的总样本数量；
- $\text{Gini}(D_j)$ 是子集 D_j 的基尼值。

构建决策树

从根节点开始，计算知道各个属性对确定最后分类结果带来的**信息增益**，选择信息增益**最大的属性作为根节点。**

接下来，根据根节点各个属性的取值，将数据集**划分为若干个子集，** 对每个子集递归地**重复**上述过程，用其他属性来构建子树。

停止拓展条件:

- 当某个节点的**所有样本都属于同一类别**时

- 所有属性**都已使用过**
- 树达到**最大深度**
- 信息增益值**小于**某个**阈值**

树过大或者过复杂时，容易过拟合，可以通过**剪枝**来解决这个问题。

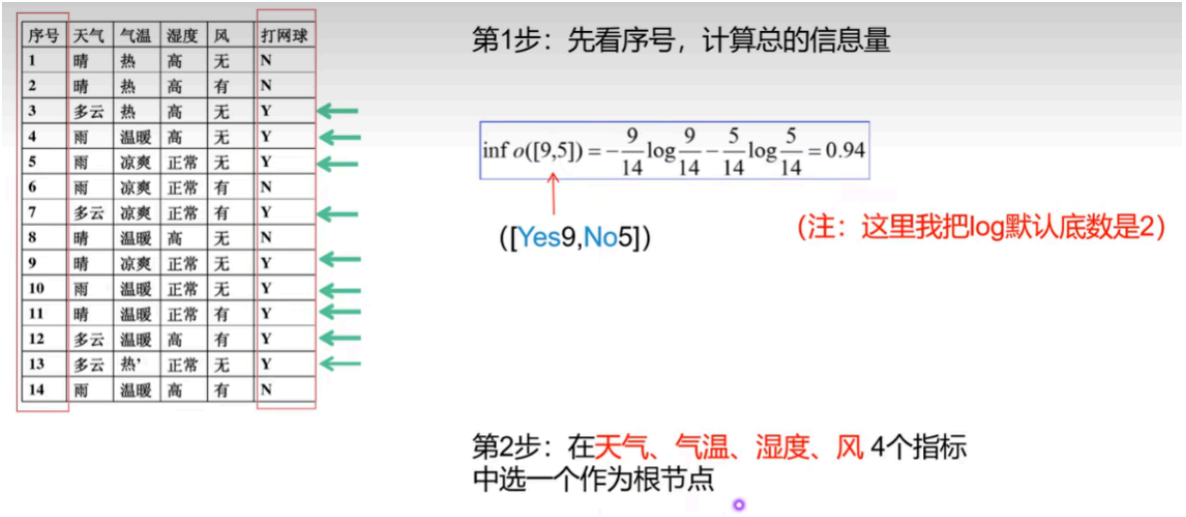
利用验证集，可以在构建过程中或者构建结束后使用下面的策略：

- 减去某个节点不影响验证集的准确率
- 减去对验证集准确率提升最小的节点
- 验证集上的准确率开始下降即停止扩展

计算例子

https://www.bilibili.com/video/BV1Cq4y1S7k1

这里的info相当于H(Z)



序号	天气	气温	湿度	风	打网球
1	晴	热	高	无	N
2	晴	热	高	有	N
3	多云	热	高	无	Y
4	雨	温暖	高	无	Y
5	雨	凉爽	正常	无	Y
6	雨	凉爽	正常	有	N
7	多云	凉爽	正常	有	Y
8	晴	温暖	高	无	N
9	晴	凉爽	正常	无	Y
10	雨	温暖	正常	无	Y
11	晴	温暖	正常	有	Y
12	多云	温暖	高	有	Y
13	多云	热'	正常	无	Y
14	雨	温暖	高	有	N

这里的E(天气)相当于H(Z|Y),里面的xx/14就是不同的P(Y)

序号	天气	气温	湿度	风	打网球
1	晴	热	高	无	N
2	晴	热	高	有	N
3	多云	热	高	无	Y
4	雨	温暖	高	无	Y
5	雨	凉爽	正常	无	Y
6	雨	凉爽	正常	有	N
7	多云	凉爽	正常	有	Y
8	晴	温暖	高	无	N
9	晴	凉爽	正常	无	Y
10	雨	温暖	正常	无	Y
11	晴	温暖	正常	有	Y
12	多云	温暖	高	有	Y
13	多云	热’	正常	无	Y
14	雨	温暖	高	有	N

第2步：在天气、气温、湿度、风 4个指标中选一个作为根节点

天气-晴：

inf o（[2,3]）= -2/5log2/5 - 3/5log3/5=0.97

天气-雨：

$$\inf o([3,2]) = -\frac{3}{5}\log\frac{3}{5} - \frac{2}{5}\log\frac{2}{5} = 0.97$$

天气-云：

$$\inf o([4,0]) = 0.00$$

天气各种情况加权和：

E（天气）= $$\inf o([2,3],[4,0],[3,2]) = \frac{5}{14}\inf o[2,3] + \frac{4}{14}\inf o[4,0] + \frac{5}{14}\inf o[3,2] = 0.69$$

inf o:信息量
E：信息熵
gain：信息增益

基于天气决定是否打网球的信息增益为：

$$gain(天气) = \inf o([9,5]) - \inf o([2,3],[4,0],[3,2]) = 0.25$$

第一步算的总的信息量=0.94

第2步天气的三种情况加权之和=0.69

继续算下一个的信息增益

序号	天气	气温	湿度	风	打网球
1	晴	热	高	无	N
2	晴	热	高	有	N
3	多云	热	高	无	Y
4	雨	温暖	高	无	Y
5	雨	凉爽	正常	无	Y
6	雨	凉爽	正常	有	N
7	多云	凉爽	正常	有	Y
8	晴	温暖	高	无	N
9	晴	凉爽	正常	无	Y
10	雨	温暖	正常	无	Y
11	晴	温暖	正常	有	Y
12	多云	温暖	高	有	Y
13	多云	热’	正常	无	Y
14	雨	温暖	高	有	N

第2步：在天气、气温、湿度、风 4个指标中选一个作为根节点

气温-热：

inf o（[2,2]）= -2/4log2/4 - 2/4log2/4=

气温-温暖：

inf o（[5,1]）= -5/6log5/6 - 1/6log1/6=

气温-凉爽：

inf o（[3,1]）= -3/4log3/4 - 1/4log1/4=

天气各种情况加权和：

inf o（[2,2],[5,1],[3,1]）=
4/14inf o（[2,2]）+6/14inf o（[5,1]）+4/14inf o（[3,1]）=

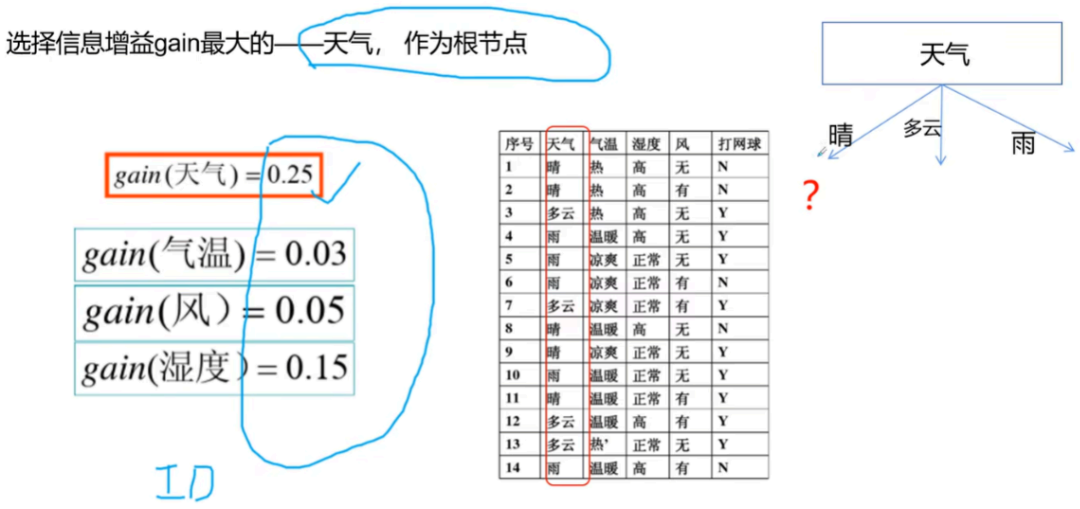
基于气温决定是否打网球的信息增益为：

gain(气温)=inf o([9,5])-inf o([2,2],[5,1],[3,1])=0.03

第一步算的总的信息量=0.94

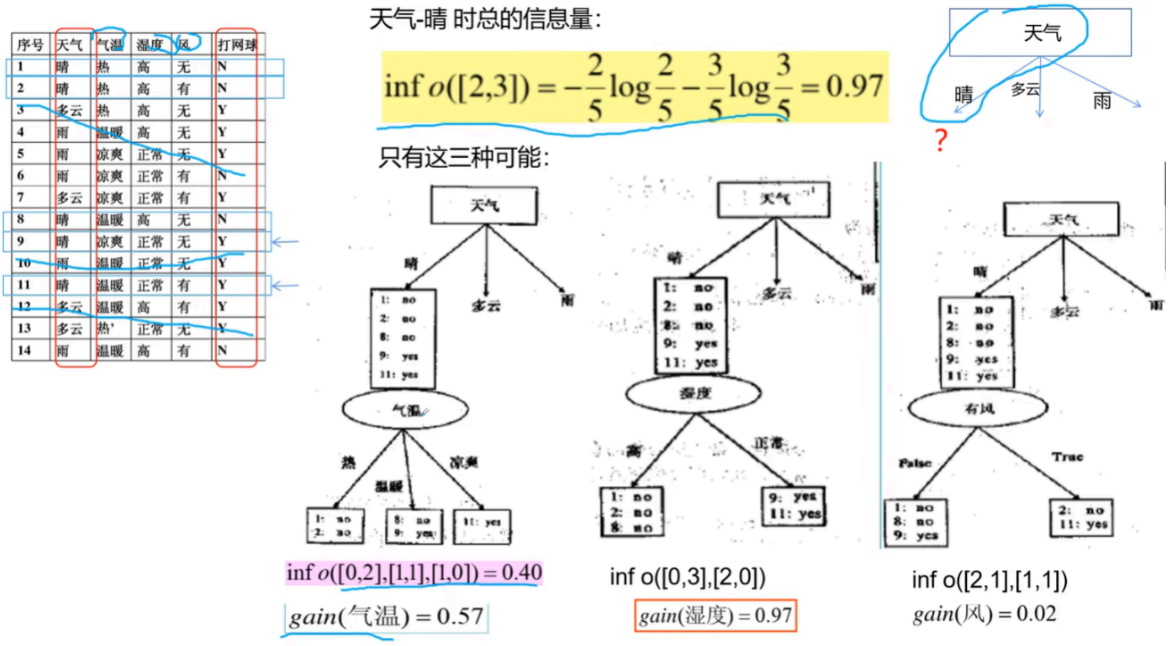
第2步天气的三种情况加权之和=0.797

假设全部算完开始构建

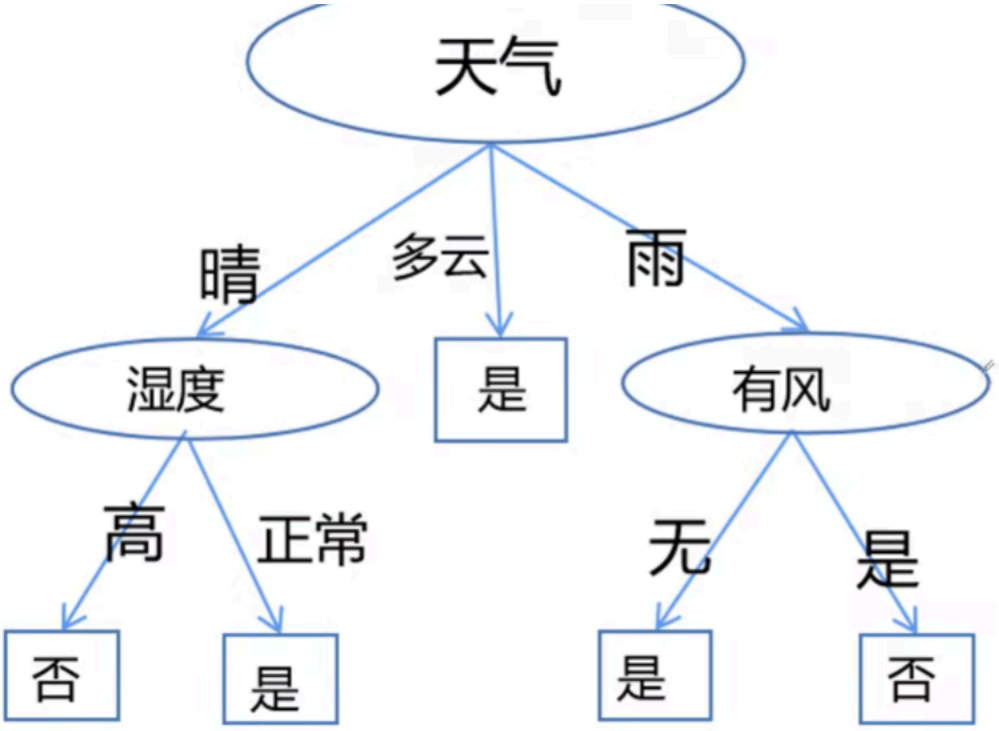


序号	天气	气温	湿度	风	打网球
1	晴	热	高	无	N
2	晴	热	高	有	N
3	多云	热	高	无	Y
4	雨	温暖	高	无	Y
5	雨	凉爽	正常	无	Y
6	雨	凉爽	正常	有	N
7	多云	凉爽	正常	有	Y
8	晴	温暖	高	无	N
9	晴	凉爽	正常	无	Y
10	雨	温暖	正常	无	Y
11	晴	温暖	正常	有	Y
12	多云	温暖	高	有	Y
13	多云	热'	正常	无	Y
14	雨	温暖	高	有	N

看每一个分支,就只需要**考虑满足分支这个条**件的这些行,算该条件每个**剩余属性**的**信息增益**



结果



剪枝

预剪枝

预剪枝是在决策树**生成**过程中**提前停止树的生长**，防止树变得过于复杂。具体方法是在树的构建过程中，对每个节点的划分进行评估，如果划分不能带来显著的性能提升，则停止划分并将该节点标记为叶节点。

预剪枝的常用方法：

- **最大深度**：限制树的最大深度，防止树过深。
- **最小样本数**：如果一个节点的**样本数少于某个阈值**，则停止划分。
- **信息增益的阈值**：如果划分后的信息增益**小于某个阈值**，则停止划分。

优点：

- 简单高效，能够显著**减少训练时间。**
- 避免生成过于复杂的树，降低**过拟合**风险。

缺点：

- 可能过早停止树的生长，导致**欠拟合**。
- 难以**确定合适**的停止条件（如阈值或最大深度）。

---

2. 后剪枝

后剪枝是在决策树**完全生成后**，**从底部向上**对树进行修剪，**移除**不必要的子树，并用叶节点替换。后剪枝通常基于验证集的性能来决定是否剪枝。

后剪枝的常用方法：

- **错误率降低剪枝 (Reduced Error Pruning)**：
  - 使用验证集评估每个**子树替换为叶节点**后的性能。
  - 如果**替换后**验证集的错误率**降低或不变**，则**剪枝**。
- **代价复杂度剪枝 (Cost Complexity Pruning)**：
  - 引入**复杂度惩罚项**，平衡树的**复杂度和拟合能力**。
  - 计算每个子树的**代价复杂度**，选择最优的剪枝方案。

优点：

- 生成完整的树后再剪枝，保留了**更多的划分信息**。
- 通常比预剪枝**效果更好**，能够更好地平衡欠拟合和过拟合。

缺点：

- 计算**开销较大**，需要**额外的验证集**。
- 剪枝过程**较为复杂**，实现**难度较高**。

预剪枝 vs 后剪枝

特性	预剪枝	后剪枝
剪枝时机	在树生成过程中剪枝	在树完全生成后剪枝
计算开销	较小	较大
过拟合风险	较低	较低

特性	预剪枝	后剪枝
欠拟合风险	较高（可能过早停止生长）	较低
实现难度	简单	复杂
常用方法	最大深度、最小样本数、阈值控制	错误率降低剪枝、代价复杂度剪枝

集成学习

集成学习是一种机器学习策略，通过**结合多个弱分类器**（Weak Classifier）的预测来提高整体模型的性能。

基本思想是，虽然**单个弱**分类器在**整体**数据集上的表现可能**一般**，但它们可能在数据集的不同**子集上表现优异**。

通过**合理地组合**这些弱分类器，可以得到一个性能更强大的**强分类器**（Strong Classifier）。

Bagging

Bootstrap Aggregating, 自举汇聚法。

抽样构建方法

1. 创建训练数据的**子集**

- Bootstrap**采样**：从原始训练数据集中通过**有放回地随机抽样**生成多个子集。假设有N个样本，每次**随机抽取N'个样本**（通常N' = N），生成一个子集。这个过程**重复K次**，生成K个不同的子集S1, S2, ..., SK。
  样本被抽中后会放回原数据集，因此同一个样本可能在同一个子集中被多次抽中，也可能有些样本在某个子集中没有被抽中。

2. **训练**多个**基模型**

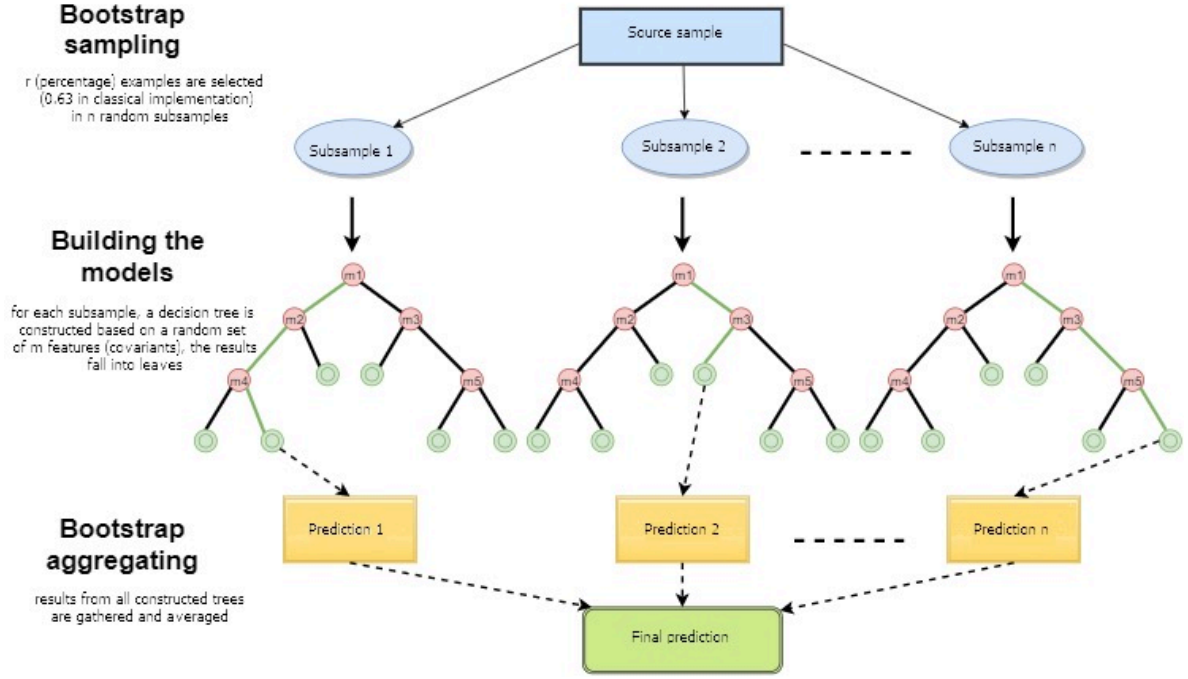
- 训练基模型：在每个生成的子集SK上训练一个独立的决策树模型。因此，最终会**得到K个决策树模型**。

3. **结合**多个模型的预测

- 结合预测结果：

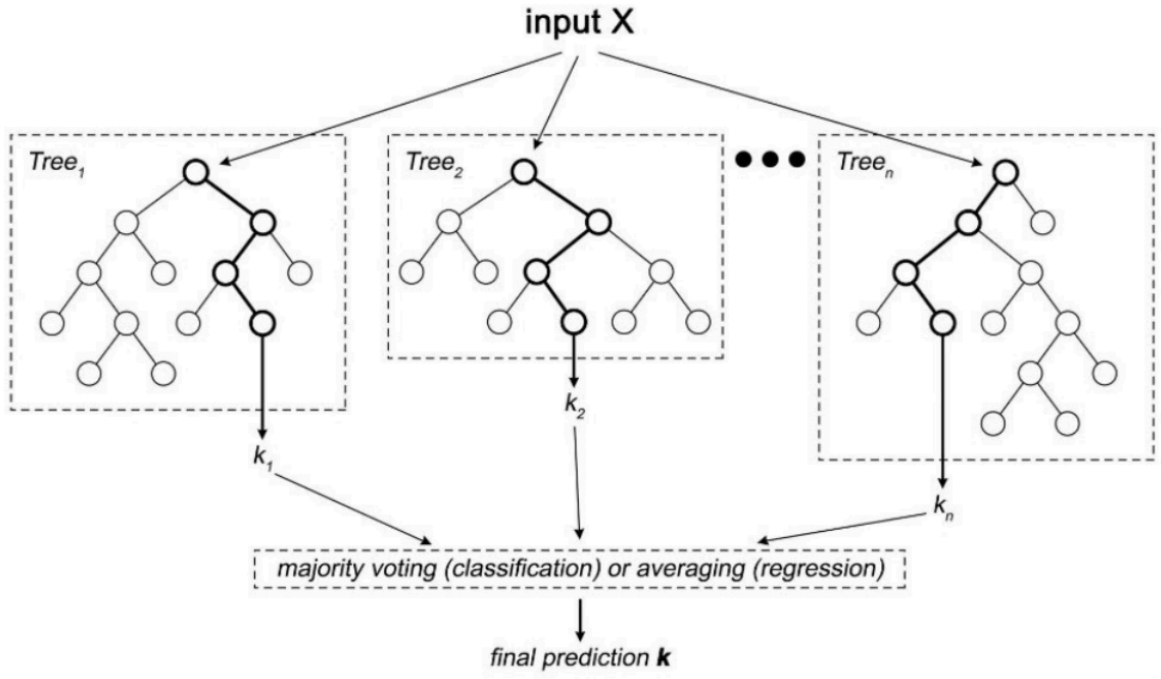
  对于分类任务，通过**多数投票**（majority voting）来确定最终的预测类别；

  对于回归任务，则通常取所有基模型预测值的**平均值**作为最终预测结果。



可见 Bagging 是一种**无权**的组合方式。

这种方式下，各个弱分类器的误差可能还是有较大的相关性，我们可以进一步增加随机性。例如，如果弱分类器采用的是决策树，我们可以在构建决策树是，只使用**随机地抽取的部分属性**。这样每个决策树的形状都不尽相同，是一片随机森林(Random Forest) 。



Boosting

每次训练一个弱分类器，对于**被误分类的样本**，给予其**更大的权重**，然后投入到**下一个弱分类器**的训练中。最后将所有弱分类器**带权组合**起来。

AdaBoost (Adaptive Boosting自适应增强) 算法：

下标表示第几个样本,上标表示第几轮迭代;一共N个样本,K个迭代,每次迭代训练一个h

1.**初始化**样本权重$w_i^{(0)} = \frac{1}{N}, i = 1, 2, \cdots, N$

2.对于第k轮**迭代**，训练一个分类器$h_k(\boldsymbol{x})$,用带权$\boldsymbol{w}^{(k-1)}$的样本

3.训练结束后，计算**错误率**：

$$\epsilon_k = \frac{\sum_{i=1}^{N} w_i^{(k-1)} I(h_k(\boldsymbol{x}_i) \neq y_i)}{\sum_{i=1}^{N} w_i^{(k-1)}}$$

也即是被**误分类的样本**的权重之和占**所有**样本权重之和的比例

> I是指示函数,()里面条件为真的时候值是1,否则是0

4.该**分类器权重**为

$$\alpha_k = \frac{1}{2} \ln \frac{1 - \epsilon_k}{\epsilon_k}$$

> 错的越多,ϵ_k越大,**分类器权重**α_k越小

5.更新**样本权重**，用于下一轮训练：

$$w_i^{(k)} = w_i^{(k-1)} \exp(-\alpha_k y_i h_k(\boldsymbol{x}_i))$$

> y_i 是样本的真实标签，$h_k(\boldsymbol{x}_i)$ 是第 k 个分类器的预测结果
>
> 对α_k取负号,被误分类的**样本权重**会增加，使得后续的分类器更加关注这些样本。

由上述训练过程可见，误分类的样本权重会增大，正确分类的样本权重会减小，这样可以使得下一个弱分类器更关注于误分类的样本，这是一个**增强**的过程。同时还可以发现，最后的组合是带权的，与弱分类器的表现有关。

$H(x) = \text{sign}\left(\sum_{i=1}^{K} \alpha_i \cdot h_i(x)\right)$

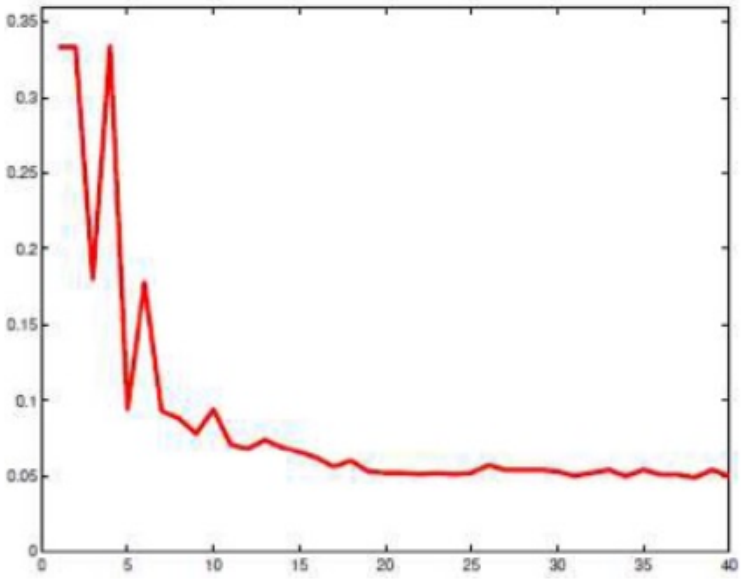
考虑定义一个指数损失函数：

$$L(\boldsymbol{w}) = \sum_{i=1}^{N} \exp(-y_i H(\boldsymbol{x}_i))$$

可以证明，AdaBoost 算法等价于在最小化指数损失函数。

典型的弱分类器 决策树;逻辑回归;神经网络

一个典型的误分类率曲线如下图所示：



集成学习的经典算法

- **随机森林（Random Forest）**：由**多棵决策树**组成，每棵树**在随机选择的特征子集**上训练，最终通过**投票**决定分类结果或平均回归结果。
- **AdaBoost**：调整**错误样本的权重**，使后续学习器更**关注难分类**的样本。

无监督集成学习的基本思想

无监督集成学习的基本思想是通过结合多个**无监督学习模型**的结果，提升整体性能。其核心在于利用不同模型的**多样性和互补性**，以更准确地捕捉数据的内在结构。以下是其主要思想：

1. **多样性**：使用**多种**无监督学习**算法**或**同一算法的不同参数**设置，生成多个基础模型。这些模型在数据的不同方面表现出色，增加了结果的多样性。
2. **互补性**：不同模型可能**关注**数据的**不同特征**，集成这些模型可以**弥补单一模型的不足**，提供更全面的数据描述。
3. **集成策略**：通过**投票、加权平均、聚类**等方法，将多个模型的结果结合起来，形成更稳定、更准确的最终结果。
4. **鲁棒性**：集成学习能**降低单一**模型对**噪声或异常值**的**敏感性**，提高**整体鲁棒性。**
5. **自动化**：无监督集成学习通常**不需要标签数据**，适用于缺乏标注的场景，能够自动发现数据中的潜在模式。

常见方法

- **聚类集成**：结合多个聚类结果，生成更一致的聚类。
- **降维集成**：融合多个降维结果，获得更有效的低维表示。
- **异常检测集成**：集成多个异常检测模型，提高检测准确性。

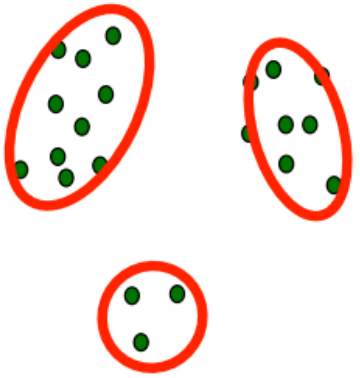
应用场景

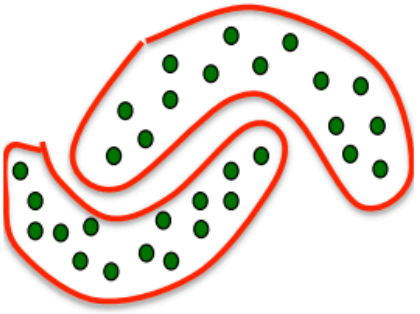
- **数据挖掘**：发现数据中的隐藏模式。
- **图像分割**：结合多种分割方法，提升分割效果。
- **生物信息学**：分析基因表达数据，识别功能模块。

K-means

给定一组数据$\{x_i\}$,聚类就是将数据分成若干个簇：

簇内的数据点**尽可能相似**，簇间的数据点**尽可能不同。**





流程

K-means 是一种常见的聚类算法，流程如下：

1.假如要将数据分成K个簇，首先生成K个**初始中心点** μ_k,每个中心点代表一个簇
2.对于每一个数据点，计算到所有中心点距离,之后**分配**到**最近**的中心点所代表的簇。
若用$r_{i,k} \in \{0,1\}$表示x_i**是否属于**第k个簇，则有：

$$r_{i,k} = \begin{cases} 1, & \text{if } k = \arg\min_j \|\boldsymbol{x}_i - \boldsymbol{\mu}_j\|^2 \\ 0, & \text{otherwise} \end{cases}$$

3.更新中心点，使得每个簇的**中心点**是簇内所有数据点的平均值：

$$\mu_k = \frac{\sum_{i=1}^{N} r_{i,k} \cdot \boldsymbol{x}_i}{\sum_{i=1}^{N} r_{i,k}}$$

- **分子** $\sum_{i=1}^{N} r_{i,k} \cdot \boldsymbol{x}_i$:
  - 将所有属于第 k 个簇的数据点的特征向量相加，得到这些数据点的总和。
- **分母** $\sum_{i=1}^{N} r_{i,k}$:
  - 计算第 k 个簇中**数据点的数量。**

终止条件

1. 中心点变化小于**阈值。**
2. 达到**最大迭代**次数。

3. 簇的**分配不再变化。**

假设有一个簇 k，包含以下 3 个数据点：

$$\boldsymbol{x}_1 = [1, 2], \quad \boldsymbol{x}_2 = [2, 3], \quad \boldsymbol{x}_3 = [3, 4]$$

- 计算新的中心点 μ_k：
  - 分子：

    $$\sum_{i=1}^{3} r_{i,k} \cdot \boldsymbol{x}_i = [1, 2] + [2, 3] + [3, 4] = [6, 9]$$

  - 分母：

    $$\sum_{i=1}^{3} r_{i,k} = 1 + 1 + 1 = 3$$

  - 新的中心点：

    $$\mu_k = \frac{[6, 9]}{3} = [2, 3]$$

判断K-means 是否收敛

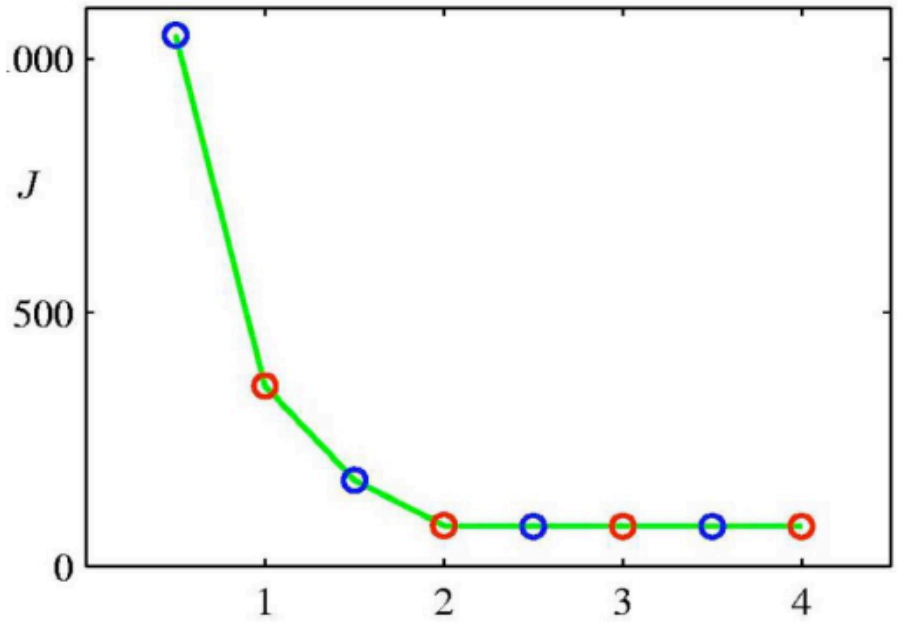
可以定义如下目标函数：

$$SSE = \sum_{i=1}^{N} \sum_{k=1}^{K} r_{i,k} \|\boldsymbol{x}_i - \boldsymbol{\mu}_k\|^2$$

这个目标函数的含义是，每个数据点到其所属簇中心的距离之和。K-means 就相当于在最小化这个目标函数。易知
SSE是单调递减的，因为每次迭代都会使得SSE减小。所以 K-means 一定会收敛。

易知，随着K增大，J总是减小的。一般，可以用**肘部法则**来确定最佳的K值：

可以通过寻找曲线的“肘部点”来确定 K 的最佳值。在这个例子中，$K = 2$ 是肘部点，即在这个点之后，J 的减少速度明显减缓，增加聚类数对结果的改进有限。



选择 K 的两种方法：

- 方法1（肘部法则）：通过观察肘部点确定 K，如图所示 $K = 2$ 是肘部点。
- 方法2：根据**下游任务的性能**来决定最佳的 K。比如，使用不同的 K 值进行聚类后，测试其对后续任务的影响，从而选择一个在应用中效果最优的 K。

K-means 聚类的效果还与**初始中心点高度相关**。

几种初始化方法：

- **随机**初始化。这种方法可能导致**初始生成邻近**的中心点，使得最终聚类效果**不好**

- **基于距离**的初始化。第一个中心点随机生成，接下来选择**离现存中心点最远**的数据点作为下一个中心点。这种
  方法的问题是**可能选择到一些离群点。**
- **随机+基于距离**的初始化。先随机生成一个中心点，然后基于距离选择下一个中心点，剩下的距离离目前的中
  心点**越远**，被选中的**概率越大。**

隐变量&GMM

引入

在第三讲中我们提到，分类和回归这种有监督学习都可以看成是在学习某个条件概率分布$p(y|x;w)$。

无监督学习也可以看成是在学习某个概率分布$p(\boldsymbol{x};\boldsymbol{w})$。

由于没有标签y,对x建模要困难许多。

一种最简单的想法是将x的分布限定为某个高斯分布$\mathcal{N}(\mu,\boldsymbol{\Sigma})$

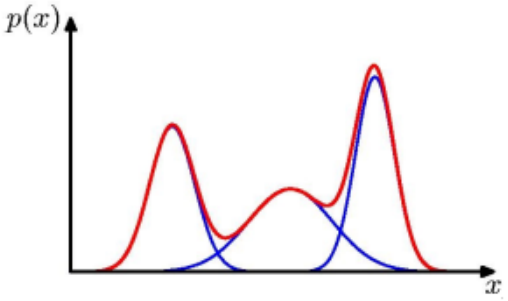
- μ 是数据的均值，表示数据的中心位置。
- Σ 是数据的协方差矩阵，表示数据的分布形状和方向。

通过优化 μ 和 Σ，使得高斯分布能够最好地拟合数据点 $\left\{x^{(n)}\right\}_{n=1}^{N}$。

设想引入一个简单的分类分布$p(z)=Cat(K,\pi)$,满足$p(z_i)=\pi_i,i=1,2,\cdots,K$。我们将其与一个高斯分布组合起来则我们可以获得一个更复杂的分布：

> **Cat(K, π)** 是一种描述离散随机变量的分布，其中 K 是类别数，π 是每个类别的概率向量。
>
> 例子,有一个骰子，$K=6$（骰子有 6 个面），且骰子是公平的，则 $\pi=\left(\frac{1}{6},\frac{1}{6},\frac{1}{6},\frac{1}{6},\frac{1}{6},\frac{1}{6}\right)$。此时，分类分布描述了骰子每个面朝上的概率。

$$p(\boldsymbol{x})=\sum_{i=1}^{K}p(\boldsymbol{x}|\boldsymbol{z}_i)p(\boldsymbol{z}_i)=\sum_{i=1}^{K}\boldsymbol{\pi}_i\mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_i,\boldsymbol{\Sigma}_i)$$



这里的$p(z)$可以代表数据隐藏的结构。一般地，隐变量模型是带有隐变量的概率模型：

$$p(\boldsymbol{x}, z)$$

其中z就称为隐变量。隐变量也可以有多个，如$p(\boldsymbol{x}, \boldsymbol{z}_1, \boldsymbol{z}_2, \cdots, \boldsymbol{z}_K)$。

相应的，关于x的概率$p(x)$是一个边缘概率，可以写为：

$$p(\boldsymbol{x}) = \int_z p(\boldsymbol{x}, \boldsymbol{z}) d\boldsymbol{z}$$

高斯隐变量模型

若隐变量的分布是一个高斯分布：

$$p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{0}, I)$$

引入参数W写出条件概率$p(x|z)$,假定其也是一个高斯分布：

$$p(\boldsymbol{x}|\boldsymbol{z}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{W}\boldsymbol{z} + \boldsymbol{\mu}, \sigma^2 \boldsymbol{I})$$

我们可以通过最大化其(对数)似然概率：

$$\log p(\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_N) = \log \prod_{i=1}^{N} p(\boldsymbol{x}_i) = \sum_{i=1}^{N} \log p(\boldsymbol{x}_i)$$

来得到能最好拟合真实数据的参数。

> 在两者都是高斯分布的情况下，目标函数可以有一个解析解。首先，由于：
>
> $$p(\boldsymbol{x}_i) = p(\boldsymbol{x}_i|\boldsymbol{z}_i) \cdot p(\boldsymbol{z}_i) = \mathcal{N}(\boldsymbol{x}_i|\boldsymbol{W}\boldsymbol{z}_i + \boldsymbol{\mu}, \sigma^2\boldsymbol{I})\mathcal{N}(\boldsymbol{z}_i|\boldsymbol{0}, \boldsymbol{I})$$
>
> 则：
>
> $$\boldsymbol{x}_i = \boldsymbol{\mu} + W\boldsymbol{z}_i + \boldsymbol{\epsilon}_i$$
>
> 其中$\epsilon_i \sim \mathcal{N}(0, \sigma^2\boldsymbol{I})$。因为高斯分布的线性组合仍然是高斯分布，所以$x_i$也是服从高斯分布的。
>
> 这个多维高斯分布的期望为：
>
> $$E[\boldsymbol{x}_i] = \boldsymbol{\mu} + \boldsymbol{W}\mathbb{E}[\boldsymbol{z}_i] + \mathbb{E}[\boldsymbol{\epsilon}_i] = \boldsymbol{\mu}$$
>
> 协方差为：
>
> $$\mathrm{Cov}[\boldsymbol{x}_i] = \mathbb{E}[(\boldsymbol{x}_i - \boldsymbol{\mu})(\boldsymbol{x}_i - \boldsymbol{\mu})^T] = WW^T + \sigma^2\boldsymbol{I}$$
>
> 所以有：
>
> $$p(\boldsymbol{x}_i) = \mathcal{N}(\boldsymbol{x}_i|\boldsymbol{\mu}, \boldsymbol{W}\boldsymbol{W}^T + \sigma^2\boldsymbol{I})$$

回头看对数似然概率，可以写成：

$$\log p(\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_N) = \log \prod_{i=1}^{N} p(\boldsymbol{x}_i) = \sum_{i=1}^{N} \log \mathcal{N}(\boldsymbol{x}_i|\boldsymbol{\mu}, \boldsymbol{W}\boldsymbol{W}^T + \sigma^2\boldsymbol{I}) =$$

$$-\frac{1}{2}\left[ND\log 2\pi + N\log det(\boldsymbol{W}\boldsymbol{W}^T + \sigma^2\boldsymbol{I}) + \sum_{i=1}^{N}(\boldsymbol{x}_i - \boldsymbol{\mu})^T(\boldsymbol{W}\boldsymbol{W}^T + \sigma^2\boldsymbol{I}\right.$$

记$\boldsymbol{\Sigma} = \boldsymbol{W}\boldsymbol{W}^T + \sigma^2\boldsymbol{I}, \boldsymbol{S} = \sum_{i=1}^{N}(\boldsymbol{x}_i - \boldsymbol{\mu})(\boldsymbol{x}_i - \boldsymbol{\mu})^T$,则：

$$\frac{\partial \log p(\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_N)}{\partial \boldsymbol{\mu}} = 0 \Rightarrow \sum_{i=1}^{N}(\boldsymbol{x}_i - \boldsymbol{\mu}) = 0 \Rightarrow \boldsymbol{\mu} = \frac{1}{N}\sum_{i=1}^{N}\boldsymbol{x}_i$$

$$\begin{aligned}\frac{\partial \log p(\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N)}{\partial \boldsymbol{\Sigma}} &= -\frac{N}{2}\Sigma^{-1} + \frac{1}{2}\sum_{n=1}^{N}\Sigma^{-1}(x_n - \mu_n)(x_n - \mu_n)^T\Sigma^{-1} \\ &= -\frac{N}{2}\boldsymbol{\Sigma}^{-1} + \frac{N}{2}\boldsymbol{\Sigma}^{-1}\boldsymbol{S}\boldsymbol{\Sigma}^{-1} = 0 \\ &\Rightarrow \boldsymbol{\Sigma} = \boldsymbol{S}\end{aligned}$$

综上，在最大化似然概率时，易知$x_i \sim \mathcal{N}(\mu, \Sigma) = \mathcal{N}(\overline{x}, S)$。

进一步可求出$W = U\sqrt{\Lambda - \sigma^2 I}$,其中

- U由S的前M个特征向量组成
- Λ是S的前M个特征值组成的对角矩阵

所以W就可以视作没有经过正则化的数据x的主成分，其中每i个成分被放大了$\sqrt{\lambda_i} - \sigma^2$倍

所以高斯隐变量模型也被称为概率 PCA。

高斯混合模型

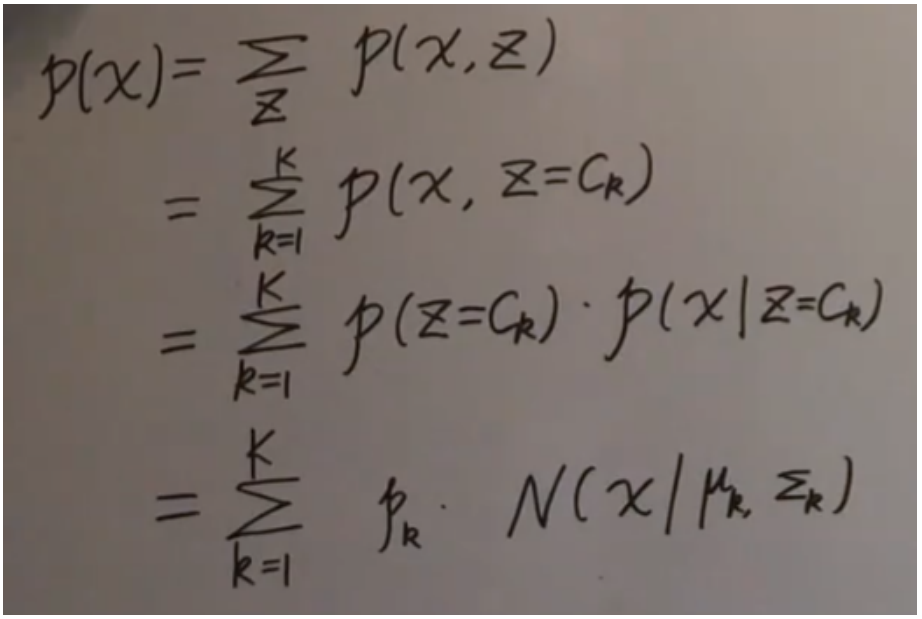
https://www.bilibili.com/video/BV13b411w7Xj

高斯混合模型（Gaussian Mixture Model, GMM） 是一种概率生成模型，用于对复杂数据分布进行建模。它假设数据是由多个高斯分布（正态分布）的加权组合生成的，每个高斯分布代表一个子类（或簇）。GMM 是一种软聚类方法，能够处理数据中的不确定性，并为每个数据点分配属于不同簇的概率。

在高斯混合模型中，我们假设数据是由**多个高斯分布混合而成**的：

$$p(\boldsymbol{x}) = \sum_{i=1}^{K}\pi_i\mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$$

其中π_i是权重，满足$\sum_i^K \pi_i = 1$。



高斯混合模型可以写作隐变量模型的形式。引入隐变量z,且z是一个独热编码，记z_i表示第i个元素为 1,其他元素
为0的向量。

隐变量的分布写为$p(z)$,满足$p(\boldsymbol{z}_i) = \pi_i$。条件概率$p(\boldsymbol{x}|\boldsymbol{z})$写为$p(\boldsymbol{x}|\boldsymbol{z}_i) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$。这就得到了高斯混合
模型的隐变量模型形式。

通过最大化似然概率，可以得到最优参数$\mu_i, \boldsymbol{\Sigma}_i, \pi_i$。

有了x的分布之后，可以**计算后验概率**:

$$p(\boldsymbol{z}_i|\boldsymbol{x}) = \frac{p(\boldsymbol{x}|\boldsymbol{z}_i)p(\boldsymbol{z}_i)}{p(\boldsymbol{x})} = \frac{\pi_i \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

这个后验概率的含义就是，数据点x属于第i个高斯分布的概率，相当于在做聚类。

LVM (隐变量模型)还运用在隐式马尔科夫模型、Sigmoid 信念网络 (SBN)、隐式狄利克雷分布 (LDA)等模型中。

EM算法

https://www.bilibili.com/video/BV1qW411k7ao/

八股文

是什么

EM算法(期望最大化算法)是一种用于**估计**含有**隐**变量的**概率模型 参数**的**迭代**算法。

用于解决下面的问题：

假如有联合分布$p(x,z|\theta)$,我们需要**最大化对数似然概率**(一个边缘概率)$p(\boldsymbol{x};\boldsymbol{\theta})$

$p(x;\theta)=\sum_z p(x,z;\theta)$

$$\theta^*=\arg\max_\theta \log p(\boldsymbol{x};\boldsymbol{\theta})$$

参数

观察变量

数据集中可以直接观测到的变量

用 X 表示，$X=\{x_1,x_2,\ldots,x_N\}$，其中 x_i 是第 i 个观测数据。

隐变量

无法直接观测到的变量，通常用于描述数据生成过程中的隐含结构。

用 Z 表示，$Z=\{z_1,z_2,\ldots,z_N\}$，其中 z_i 是第 i 个观测数据对应的隐变量。

模型参数

定义概率模型的参数，需要通过EM算法进行估计。

用 θ 表示

E步和M步分别做什么

1. E-step: 计算下面这个期望：

$$Q(\boldsymbol{\theta};\boldsymbol{\theta}^{(t)})=\mathbb{E}_{\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)}}[\log p(\boldsymbol{x},\boldsymbol{z};\boldsymbol{\theta})]$$

这实际上是在计算在给定数据x和参数$\theta^{(t)}$的情况下，隐变量z的期望对数似然概率
2.M-step:更新参数是这个期望最大化：

$$\theta^{(t+1)} = \arg\max_{\theta} Q(\boldsymbol{\theta}; \boldsymbol{\theta}^{(t)})$$

隐变量用在哪里

隐变量在EM算法中用于表示数据中**未观测到的部分**，这些隐变量通常是数据所属的**类别**、**潜在的状态**或其他**不可直接观测**的特征。

- **GMM**：隐变量表示每个数据点**属于哪个高斯成分。**
- **隐马尔可夫模型（HMM）**：隐变量表示隐藏的**状态序列。**
- **聚类分析**：隐变量表示数据点所属的**簇。**

不能获得全局最优的原因

- **局部最优**：EM算法是一种**局部优化**算法，它在每次迭代中寻找的是局部最优解，而不是全局最优解。
- **初始值敏感**：EM算法对**初始参数值非常敏感。**不同的初始值可能导致算法**收敛到不同**的局部最优解。
- **目标函数的性质**：在某些情况下，目标函数可能存在**多个局部极值点**，EM算法可能**陷入其中某一个**，而不能找到全局最优解。

应用（如何结合高斯混合聚类）

- **E步（期望步）**：
  - 假设当前参数已知（通常是上一次迭代的参数），计算每个数据点**属于每个**高斯成分（簇）的**概率**（软分配）。
  - 这一步实际上是在估计隐变量的 posterior probability，即数据点属于某个簇的概率。
- **M步（最大化步）**：

> - 根据E步得到的 posterior probability，重新**估计**高斯混合模型的**参数**，包括每个成分的均值、方差和混合系数。
> - 这一步是通过**最大化期望对数似然函数**来更新参数。

理论推导(证明收敛)

将对数似然概率重写。

> x和θ相互独立

假定q(z)是任意的分布,考虑到全部求和=1，有:

$$\log p(x;\theta)=\sum_z q(\boldsymbol{z})\log p(x;\theta)=\sum_Z q(z)\log\frac{p(x,z;\theta)q(z)}{p(z|x;\theta)q(z)}$$
$$=\sum_{\mathbf{z}} q(\mathbf{z})\log\frac{p(\mathbf{x},\mathbf{z};\theta)}{q(\mathbf{z})}+\sum_{\mathbf{z}} q(\mathbf{z})\log\frac{q(\mathbf{z})}{p(\mathbf{z}|\mathbf{x};\theta)}$$
$$=\mathcal{L}(q,\theta)+KL(q||p(z|x;\theta))$$

> 前一项是变分下界$\mathcal{L}(q,\boldsymbol{\theta})=\sum_z q(z)\log\frac{p(\boldsymbol{x},z;\boldsymbol{\theta})}{q(z)}$
> 后一项我们称之为 KL 散度$\mathrm{KL}(q\|p)=\int q(\boldsymbol{z})\log\frac{q(\boldsymbol{z})}{p(\boldsymbol{z})}d\boldsymbol{z}\geqslant 0$
>
> 用于衡量两个分布的相似程度。

$$\log p(x;\theta^{(t)})=\sum_{\mathbf{z}} q(\mathbf{z})\log\frac{p\left(x,\mathbf{z};\boldsymbol{\theta}^{(t)}\right)}{q(\mathbf{z})}+\sum_{\mathbf{z}} q(\mathbf{z})\log\frac{q(\mathbf{z})}{p(\mathbf{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)})}$$

> 若取$q(\boldsymbol{z})=p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)})$,则$KL(q||p(\mathbf{z}|\mathbf{x};\boldsymbol{\theta}^{(t)}))=0$;公式后面一项为0,然后替换第一项的q(z)

$$\log p(\boldsymbol{x};\boldsymbol{\theta}^{(t)})=\mathcal{L}(p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)}),\boldsymbol{\theta}^{(t)})=\sum_z p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)})\log\frac{p(\boldsymbol{x},\boldsymbol{z};\boldsymbol{\theta}^{(t)})}{p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)})}$$

如果更新参数:

$$\boldsymbol{\theta}^{(t+1)}=\arg\max_{\theta}\mathcal{L}(p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)}),\boldsymbol{\theta})$$

因为θ是最大化$\mathcal{L}$得到的参数,则一定有**更新后的$\mathcal{L}$更大**:

$$\mathcal{L}(p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)}),\boldsymbol{\theta}^{(t+1)})\geqslant\mathcal{L}(p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)}),\boldsymbol{\theta}^{(t)})=\log p(\boldsymbol{x};\boldsymbol{\theta}^{(t)})$$

我们再取$q(\boldsymbol{z}) = p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)})$,计算参数为$\boldsymbol{\theta}^{(t+1)}$时的对数似然概率：

$$\log p(\boldsymbol{x};\boldsymbol{\theta}^{(t+1)}) = \mathcal{L}(p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)}),\boldsymbol{\theta}^{(t+1)}) + \mathrm{KL}(p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)})\|p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t+1)})) \geqslant$$

所以按照上面更新参数的方法，总有：

$$\log p(\boldsymbol{x};\boldsymbol{\theta}^{(t+1)}) \geqslant \log p(\boldsymbol{x};\boldsymbol{\theta}^{(t)})$$

$\arg\max_\theta \mathcal{L}(p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)}),\boldsymbol{\theta})$能保证每一步的似然性都能增加

接下来，只需证明 EM 算法和上述过程等价。因为：

$$\mathcal{L}(q,\theta) = \sum_z q(z)\log\frac{p(\boldsymbol{x},\boldsymbol{z};\boldsymbol{\theta})}{q(\boldsymbol{z})} = \sum_z q(\boldsymbol{z})\log p(\boldsymbol{x},\boldsymbol{z};\boldsymbol{\theta}) - \sum_z q(\boldsymbol{z})\log q(z$$

后半部分其实就是$q(z)$的熵，可以忽略。而前半部分正是 E-step 中的期望。所以 EM 算法和上述过程等价。

> “上述过程”指的是在 EM 算法中通过选择 $q(\mathbf{z}) = p(\mathbf{z}|\mathbf{x};\boldsymbol{\theta}^{(t)})$ 并最大化 $\mathcal{L}(q,\theta)$ 来更新参数 $\boldsymbol{\theta}$ 的过程。具体来说，该过程包括以下步骤：
>
> 1. **选择** $q(\mathbf{z}) = p(\mathbf{z}|\mathbf{x};\boldsymbol{\theta}^{(t)})$：在第 t 轮迭代中，选择隐变量 $\mathbf{z}$ 的分布为当前参数 $\boldsymbol{\theta}^{(t)}$ 下的后验分布。
> 2. **最大化** $\mathcal{L}(q,\theta)$ **来更新参数**：通过最大化期望对数似然函数 $\mathcal{L}(q,\theta)$ 来得到新的参数 $\boldsymbol{\theta}^{(t+1)}$。
>
> 这个过程确保了每一步对数似然概率 $\log p(\mathbf{x};\boldsymbol{\theta})$ 都会增加，并且与 EM 算法的 E 步和 M 步相对应：
>
> - **E 步**：计算期望 $\mathcal{L}(p(\mathbf{z}|\mathbf{x};\boldsymbol{\theta}^{(t)}),\boldsymbol{\theta})$。
> - **M 步**：最大化这个期望来得到新的参数 $\boldsymbol{\theta}^{(t+1)}$。

例子-训练GMM

https://www.bilibili.com/video/BV13b411w7Xj

GMM 可以利用 EM 算法来训练。MLE无法求解析解

这里用视频中的公式体系

p_k定义为选择第k个高斯分布的概率

$$
\begin{aligned}
P(x|z) &= N(x|\mu_z, \Sigma_z) \\
P(z) &= p_z \\
P(x,z) &= P(z) \cdot P(x|z) = p_z \cdot N(x|\mu_z, \Sigma_z) \\
P(x) &= \sum_{k=1}^{K} P(z=k) \cdot P(x|z=k) = \sum_{k=1}^{K} p_k \cdot N(x|\mu_k, \Sigma_k) \\
P(z|x) &= \frac{P(x,z)}{P(x)} = \frac{p_z \cdot N(x|\mu_z, \Sigma_z)}{\sum_{k=1}^{K} p_k \cdot N(x|\mu_k, \Sigma_k)}
\end{aligned}
$$

E-step

根据期望定义求Q

$$
\begin{aligned}
Q(\theta, \theta^{(t)}) &= \int_Z \log P(X, Z|\theta) \cdot P(Z|X, \theta^{(t)}) \\
&= \sum_Z \log \prod_{i=1}^{N} P(x_i, z_i|\theta) \cdot \prod_{i=1}^{N} P(z_i|x_i, \theta^{(t)}) \\
&= \sum_Z \sum_{i=1}^{N} \log P(x_i, z_i|\theta) \cdot \prod_{i=1}^{N} P(z_i|x_i, \theta^{(t)}) \\
&= \sum_Z [\log P(x_1, z_1|\theta) + \log P(x_2, z_2|\theta) + \cdots + \log P(x_N, z_N|\theta)] \cdot
\end{aligned}
$$

单独取每一个**[前面括号的log]**和后面**[连乘项]**相乘可以发现规律,后面的会只剩下对于z_i的

$$
\begin{aligned}
Q(\theta, \theta^{(t)}) &= \sum_{z_1} \log P(x_1, z_1|\theta) \cdot P(z_1|x_1, \theta^{(t)}) + \cdots + \sum_{z_N} \log P(x_N, z_N|\theta) \cdot \\
&= \sum_{i=1}^{N} \sum_{z_i} \log P(x_i, z_i|\theta) \cdot P(z_i|x_i, \theta^{(t)}) \quad (\text{求和项合并}) \\
&= \sum_{i=1}^{N} \sum_{z_i} \log[p_{z_i} \cdot N(x_i|\mu_{z_i}, \Sigma_{z_i})] \cdot \frac{p_{z_i} \cdot N(x_i|\mu_{z_i}^{(t)}, \Sigma_{z_i}^{(t)})}{\sum_{j=1}^{K} p_{z_j}^{(t)} \cdot N(x_i|\mu_{z_j}^{(t)}, \Sigma_{z_j}^{(t)})} \quad \text{化}
\end{aligned}
$$

M-step

对Q前面一项log展开

$Q(\theta, \theta^{(t)}) = \sum_{k=1}^{K} \sum_{i=1}^{N} [\log p_k + \log N(x_i|\mu_k, \Sigma_k)] \cdot P(z_i = C_k|x_i, \theta^{(t)})$

此时求解最优θ: $\theta^{(t+1)} = arg\max_{\theta} Q(\theta, \theta^{(t)})$

以下以p_k 为例,进行求解:

$$
p^{(t+1)} = (p_1^{(t+1)}, p_2^{(t+1)}, \cdots, p_K^{(t+1)})
$$

$$
\begin{cases}
\max_p \sum_{k=1}^{K} \sum_{i=1}^{N} \log p_k \cdot P(z_i = C_k|x_i, \theta^{(t)}) \\
s.t. \quad \sum_{k=1}^{K} p_k = 1
\end{cases}
$$

> 上式由于$\log N(x_i|\mu_k, \Sigma_k)$与$p$ 无关,故优化时可消去

用拉格朗日乘子法,令偏导数为0得到结果

$p_k^{(t+1)} = \frac{1}{N} \sum_{i=1}^{N} P(z_i = C_k|x_i, \theta^{(t)})$

同理可以容易求出 μ_k, Σ_k

$$
\mu_k^{(t+1)} = \frac{\sum_{i=1}^{N} [P(z_i = C_k|x_i, \theta^{(t)}) \cdot x_i]}{\sum_{i=1}^{N} P(z_i = C_k|x_i, \theta^{(t)})}
$$

$$\Sigma_k^{(t+1)} = \frac{\sum_{i=1}^{N}[P(z_i = C_k|x_i, \theta^{(t)}) \cdot (x_i - \mu_k^{(t+1)})(x_i - \mu_k^{(t+1)})^T]}{\sum_{i=1}^{N} P(z_i = C_k|x_i, \theta^{(t)})}$$

EM变体

上面我们讨论了 EM 在训练 GMM 中的应用，实际问题中的后验分布不总是高斯分布，期望和参数更新都不一定有闭式解。

EM+SGD

推导 EM 算法的过程中，我们发现最大化期望，只是在保证似然概率不减。事实上，保证似然概率不减，不一定要最大化期望。

对期望利用梯度上升法来更新参数：

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} + \gamma \cdot \frac{\partial Q\left(\boldsymbol{\theta}; \boldsymbol{\theta}^{(t)}\right)}{\partial \boldsymbol{\theta}}|_{\boldsymbol{\theta}=\boldsymbol{\theta}^{(t)}}$$

显然有：

$$Q(\boldsymbol{\theta}^{(t+1)}; \boldsymbol{\theta}^{(t)}) \geq Q(\boldsymbol{\theta}^{(t)}; \boldsymbol{\theta}^{(t)})$$

由于$p(z)$的熵可以认为是不变的，所以根据 EM 算法的推导过程可以反向推导出，仍然有：

$$\log p(\boldsymbol{x}; \boldsymbol{\theta}^{(t+1)}) \geqslant \log p(\boldsymbol{x}; \boldsymbol{\theta}^{(t)})$$

MCMC EM

Markov Chain Monte Carlo EM。马尔可夫链蒙特卡洛

使用 **MCMC（马尔可夫链蒙特卡洛）** 方法从**后验分布** $p(\boldsymbol{z} \mid \boldsymbol{x}; \boldsymbol{\theta}^{(t)})$ 中采样，用这些采样值来近似期望

期望可以用这些采样值来近似：

$$Q(\boldsymbol{\theta};\boldsymbol{\theta}^{(t)}) \approx \frac{1}{S}\sum_{s=1}^{S}\log p(\boldsymbol{x},\boldsymbol{z}_s;\boldsymbol{\theta})$$

再结合 SGD,就得到了 MCMC EM, 这是常用于训练隐变量模型的方法。

采样模拟p,修改Q公式,M最大化theta更新参数

VB EM

Variational Bayes EM,变分贝叶斯 EM。

直接在高维空间中**采样**出z_s**代价比较大。**

VBEM用一个**简单的分布**$q(z,\phi)$来**近似**$p(z|x;\theta)$,然后在这个简单的分布上进行 EM 算法。

这个简单的分布，可以是每个维度用一个高斯分布的组合：

$$z_i \sim \mathcal{N}(\mu_i,\sigma_i^2)$$

然后找到能最小化与原后验分布 KL 散度的参数ϕ。

最小化 KL 散度进行转化：

$$\min_{\phi}\mathrm{KL}(q(\boldsymbol{z};\boldsymbol{\phi})\|p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta})) \Leftrightarrow \max_{\phi}\int q(\boldsymbol{z};\boldsymbol{\phi})\log\frac{p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta})}{q(\boldsymbol{z};\boldsymbol{\phi})}d\boldsymbol{z} \Leftrightarrow \max_{\phi}\int q($$

第二步舍去了$\log p(\boldsymbol{x};\boldsymbol{\theta})$,因为它与$\phi$无关。

最大化期望可以写为：

$$\max_{\theta}\mathbb{E}_{q(\boldsymbol{z};\boldsymbol{\phi})}[\log p(\boldsymbol{x},\boldsymbol{z};\boldsymbol{\theta})] = \max_{\theta}\int q(\boldsymbol{z};\boldsymbol{\phi})\log p(\boldsymbol{x},\boldsymbol{z}|\boldsymbol{\theta})d\boldsymbol{z} \Leftrightarrow \max_{\theta}\int q(\boldsymbol{z};$$

第二步添加了与θ无关的项 $log(q(\boldsymbol{z};\boldsymbol{\phi}))$,不影响最大化。
可以发现**最小化散度和最大化期望可以转化为相同的形式。**所以以上两个问题统一写为：

$$\max_{\theta,\phi} \int q(\boldsymbol{z};\boldsymbol{\phi}) \log \frac{p(\boldsymbol{x},\boldsymbol{z};\boldsymbol{\theta})}{q(\boldsymbol{z};\boldsymbol{\phi})} d\boldsymbol{z} = \max_{\boldsymbol{\theta},\phi} \int \mathcal{L}(\boldsymbol{x};\boldsymbol{\theta},\boldsymbol{\phi}) d\boldsymbol{z}$$

利用**梯度下降法**，同时更新θ, ϕ,这就是 VB EM。

可以证明，$\mathcal{L}(\boldsymbol{x};\boldsymbol{\theta},\boldsymbol{\phi})$是任意参数的对数似然概率的下界。VB EM 在最大化这个下界，也就是在最大化对数似然概率。

- Steps to update the model parameter $\boldsymbol{\theta}$

1) Finding the best approximate $q(\boldsymbol{z};\boldsymbol{\phi})$ such that 先找到最好的Φ

$$\boldsymbol{\phi}^{(t)} = \arg\min_{\boldsymbol{\phi}} KL\left(q(\boldsymbol{z};\boldsymbol{\phi}) \| p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)})\right)$$

2) Using $q(\boldsymbol{z};\boldsymbol{\phi}^{(t)})$ to compute expectation $\mathbb{E}_{p(\boldsymbol{z}|\boldsymbol{x};\boldsymbol{\theta}^{(t)})}[\log p(\boldsymbol{x},\boldsymbol{z};\boldsymbol{\theta})]$ approximately as 得到近似Q函数，“E”

$$\tilde{Q}(\boldsymbol{\theta};\boldsymbol{\phi}^{(t)}) = \mathbb{E}_{q(\boldsymbol{z};\boldsymbol{\phi}^{(t)})}[\log p(\boldsymbol{x},\boldsymbol{z};\boldsymbol{\theta})]$$

3) Obtaining the new value $\boldsymbol{\theta}^{(t+1)}$ as “M”

$$\boldsymbol{\theta}^{(t+1)} = \arg\max_{\boldsymbol{\theta}} \tilde{Q}(\boldsymbol{\theta};\boldsymbol{\phi}^{(t)})$$

EM对比

特性	EM	MCMC-EM	VBEM
全称	Expectation-Maximization	Markov Chain Monte Carlo EM	Variational Bayesian EM
核心思想	通过E步和M步**迭代优化参数点估计**	在E步中使用**MCMC采样 近似后验分布**	使用**变分推断 近似后验分布**
隐变量处理	**计算**隐变量的后验期望	通过MCMC**采样**隐变量的后验分布	通过**变分分布近似**隐变量的后验分布
参数处理	参数作为点估计（固定值）	参数通过MCMC采样得到后验分布	参数作为随机变量，通过变分分布近似
不确定性建模	不直接建模参数的不确定性	通过采样建模参数和隐变量的不确定性	通过变分分布建模参数和隐变量的不确定性
计算复杂度	较低	较高（MCMC采样计算量大）	中等（变分推断比MCMC高效）
收敛性	可能陷入局部最优	全局最优，但收敛慢	可能陷入局部最优，但比EM更稳定
适用场景	参数点估计足够的情况	需要精确后验分布的场景	需要高效近似后验分布的场景
优点	简单、计算高效	精确建模不确定性	高效、能建模不确定性
缺点	无法建模参数**不确定性**，易陷入局部最优	计算量大，收敛慢	近似精度可能不如MCMC

推荐系统

引入

对推荐问题进行建模，记X为用户集合，S为物品集合，R为用户对物品的**效用矩阵**(Utility Matrix),R_{ij}可以理解
为用户i对物品j的喜爱程度。

效用函数$u:X\times S\to\mathbb{R}$.

在推荐问题中，有以下几个关键问题：

- **收集**可以获得的**数据**，填充**效用矩阵**
- 外推无法观测/未知的数据，尤其是关注高的效用值
- 评估外推方法的好坏

对于第一个问题，可以去**直接询问**（Explit Feedback）用户对物品的评分，也可以通过用户的**行为**（Implicit Feedback）来**推断**用户对物品的喜好。例如用户**下单**购买某商品，相比只是浏览，**反映**着**较高**的喜爱。

关于外推其他效用值。首先，由于用户不可能购买过所有商品，所以效用矩阵是**稀疏**的。也就是说，用户购买商品的记录十分有限，要面临**冷启动**问题。其次，对于**新用户或者是新商品**，根本没有任何历史记录，**很难做外推**。

基于内容的推荐

Content-Based Recommendation，这是一种不依赖于效用矩阵的推荐方法。其基本思想是**根据该用户打出高分**的物品，推荐与之**相似**的物品，例如推荐相同类型/演员/风格的电影。

基于内容的推荐通过分析用户**过去喜欢**的物品**特征**，推荐**具有相似特征的新**物品。例如，如果用户喜欢某些电影，系统会推荐风格、演员或导演相似的其他电影。

优点：

- **无需其他用户**的数据，没有稀疏以及冷启动的问题。

- 每个用户有一个独立的推荐状态，不易受到其他用户的影响。
- 有**可解释性**

缺点：

- 找到**合适**的**特征**比较**困难**
- **新**用户**无**历史记录，**难**以推荐
- 过度专一化，无法跟上用户的新兴趣，也无法利用其它用户的评价来辅助判断。

协同过滤

Collaborative Filtering

用户-用户协同过滤

基于相似用户的行为推荐物品。

其灵感是，如果有一批人N和x一样都买了某件商品 A,且N中大部分人还会买 B,可以推测x也会对 B 感兴趣。

记x对所有商品的评分为$\boldsymbol{r}_x$,$I_{x,y}$是x和y共同评分的物品集合

余弦相似度

$\sin(x,y)=\cos(r_x,r_y)=\frac{r_x\cdot r_y}{||r_x||\cdot||r_y||}$

皮尔逊相关系数

$$sim(x,y)=cos(r_x,r_y)=\frac{\sum_{i\in I_{xy}}(r_{xi}-\bar{r}_x)(r_{yi}-\bar{r}_y)}{\sqrt{\sum_{i\in I_{xy}}(r_{xi}-\bar{r}_x)^2}\sqrt{\sum_{i\in I_{xy}}(r_{yi}-\bar{r}_y)^2}}$$

这里减去平均值是为了避免将没评分的结果(0)视为最负面的结果，所以减去平均值后，正面的评分为正，负面的
评分为负，更能够反映用户的喜好。

- 存在的问题：数据稀疏，计算复杂度，人是善变的，冷启动问题
- 稀疏：通常商品非常多，用户购买的只是其中极小一部分
- 计算：计算相似度矩阵是个大活，用户和商品都比较多的时候就难了
- 冷启动：新用户来了怎么办？

物品-物品协同过滤

基于相似物品的用户行为推荐物品。

这种方法的灵感来源于，如果用户x喜欢物品 A,那么他也可能喜欢和 A **相似**的物品 B。

与上面那种方法是类似的，只是这里是计算物品之间的相似度，而不是用户之间的相似度。

实际中，物品-物品联合推荐表现要比用户-用户联合推荐好，因为**物品的特征和相似性更容易捕获**，且相对**单一**。而
用户的品味和爱好是多样的。

- 更流行，现阶段各大网站基本都是用户较多，商品（种类）比用户少的多
- 商品属性通常比较固定，特征获取容易，而且基本不会改变
- 即便上架了新商品，它自身也有各种标签，不会像用户一样是张白纸
- 应用场景更适合当下各种网站，APP（实时的除外，例如新闻）

分析

协同过滤的优点：对于各种物品都适用，无需去挑选特征。

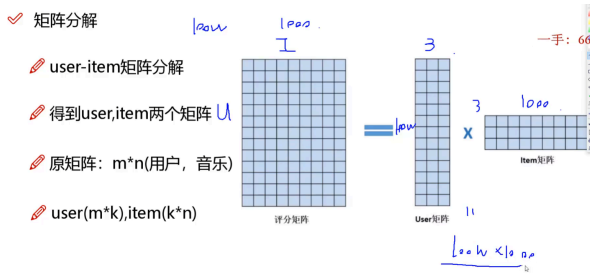
缺点：冷启动问题，对于新用户和新物品（First rater），无法推荐。且对于稀疏数据，效果不佳。而且倾向于推荐热门物品，缺乏多样性。

实践中，一般都结合使用上述方法。

基于矩阵分解的推荐

矩阵分解通过将**用户-物品评分矩阵**分解为低维**用户矩阵**和**物品矩阵**，**预测**用户对未评分物品的评分。

稀疏大矩阵,尝试分解



考虑效用矩阵$U \in \mathbb{R}^{m \times n}$,其中$m$是用户数，$n$是物品数。我们希期找到两个矩阵$P^T \in \mathbb{R}^{m \times k}$和$Q \in \mathbb{R}^{n \times k}$,

使得$U \approx P^T Q$. 近似分解,出现隐向量,代表数据的某种特征,不好解释,但是可以模拟原矩阵

我们希望近似的结果尽可能接近，并加上一些正则项，所以可以目标是最小化：

$$\min_{P,Q} \sum_{i,j} (r_{ij} - \boldsymbol{p}_i^T \boldsymbol{q}_j)^2 + \lambda(\|\boldsymbol{p}_i\|^2 + \|\boldsymbol{q}_j\|^2)$$

仅考虑有评分的项，通过梯度下降来进行训练。最终可以利用$p^T q$来预测未知的评分。

八股文

推荐系统的目的

推荐系统的主要目的是通过分析用户的**历史行为、兴趣和偏好**，向用户**推荐**他们**可能感兴趣**的内容或商品，从而提升用户**体验**、增加用户**粘性**、提高**转化率**，并最终实现**商业目标**（如增加销售额、广告点击率等）。

推荐系统的应用

1. **电子商务**：如亚马逊、淘宝等平台，推荐用户可能购买的商品。
2. **视频流媒体**：如Netflix、YouTube，推荐用户可能喜欢的视频或电影。
3. **社交媒体**：如Facebook、Twitter，推荐用户可能感兴趣的内容或好友。
4. **音乐平台**：如Spotify、网易云音乐，推荐用户可能喜欢的歌曲或歌单。
5. **新闻推荐**：如今日头条，推荐用户可能感兴趣的新闻或文章。

推荐系统的步骤

1. **数据收集**：收集用户行为数据（如点击、购买、评分等）和物品信息（如商品描述、标签等）。
2. **数据预处理**：清洗数据，处理缺失值、异常值等。
3. **特征提取**：从用户和物品数据中提取有用的特征。
4. **模型训练**：使用推荐算法（如协同过滤、基于内容的推荐等）训练模型。
5. **推荐生成**：根据模型预测用户可能感兴趣的物品。
6. **评估与优化**：通过评估指标（如准确率、召回率等）优化推荐效果。

如何基于内容进行分解

基于内容的推荐系统通过分析物品的内容特征（如文本、标签、类别等）来推荐与用户历史偏好相似的物品。步骤如下：

1. **特征提取**：从物品内容中提取特征（如TF-IDF、词嵌入等）。
2. **用户画像**：根据用户的历史行为构建用户偏好模型。
3. **相似度计算**：计算物品特征与用户画像的相似度。
4. **推荐生成**：选择与用户偏好最相似的物品进行推荐。

冷启动是什么

冷启动问题是指在新用户或新物品加入系统时，由于缺乏足够的历史数据，推荐系统难以生成准确的推荐。冷启动分为：

1. **用户冷启动**：新用户没有历史行为数据。
2. **物品冷启动**：新物品没有被用户交互过。
3. **系统冷启动**：新系统没有用户和物品数据。

解决方法包括：

- 利用用户**注册信息**（如年龄、性别等）进行推荐。
- 利用物品的**内容特征**进行推荐。
- 使用**混合**推荐方法（如**结合**协同过滤和基于内容的推荐）。

协同过滤的优缺点

优点：

1. 不需要物品的内容信息，**仅依赖用户行为**数据。
2. 能够发现用户的**潜在兴趣**。
3. 适用于**多种场景**（如商品、电影、音乐等）。

缺点：

1. **冷启动问题**：新用户或新物品难以推荐。
2. **数据稀疏性**：用户-物品交互矩阵通常非常稀疏，影响推荐效果。
3. **可扩展性**：随着用户和物品数量的增加，计算复杂度较高。

如何求解相似用户（相似度）

相似用户的求解通常基于用户的行为数据（如评分、点击等）。常用的相似度计算方法包括：

1. **余弦相似度**：

$$\text{sim}(u, v) = \frac{\sum_i r_{ui} \cdot r_{vi}}{\sqrt{\sum_i r_{ui}^2} \cdot \sqrt{\sum_i r_{vi}^2}}$$

2. **皮尔逊相关系数**：

$$\text{sim}(u, v) = \frac{\sum_i (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\sqrt{\sum_i (r_{ui} - \bar{r}_u)^2} \cdot \sqrt{\sum_i (r_{vi} - \bar{r}_v)^2}}$$

如何计算相似商品的打分

相似商品的打分通常基于用户对商品的评分数据。常用的方法包括：

1. **基于用户的协同过滤**：通过相似用户对商品的评分预测目标用户对商品的评分。
2. **基于物品的协同过滤**：通过相似商品的评分预测目标用户对商品的评分。
3. **矩阵分解**：通过分解用户-物品评分矩阵，**预测缺失的评分**。

如何评估商品的好坏

商品的好坏可以通过以下方式评估：

1. **用户评分**：用户对商品的直接评分。
2. **用户行为**：如点击率、购买率、停留时长等。
3. **内容质量**：如商品描述、图片质量、评论数量等。
4. **销售数据**：如销量、复购率等。

推荐系统的评判指标



1. **准确率 (Accuracy)**
   - **定义**: 准确率是指推荐系统中预测正确的样本占总样本的比例。
   - **公式**:

$$\text{Accuracy} = \frac{\text{True Positives (TP)} + \text{True Negatives (TN)}}{\text{TP} + \text{TN} + \text{False Positives (FP)} + \text{False Negatives (}}$$

   - **适用场景**: 适用于类别平衡的数据集，但在推荐系统中，由于正负样本通常不平衡，准确率可能不是最佳指标。
2. **精确率 (Precision)**
   - **定义**: 精确率是指推荐系统中预测为正样本的样本中，实际为正样本的比例。
   - **公式**:

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

   - **适用场景**: 适用于关注推荐结果中正确推荐的比例，即推荐结果的质量。
3. **召回率 (Recall)**
   - **定义**: 召回率是指实际为正样本的样本中，被正确预测为正样本的比例。
   - **公式**:

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

- **适用场景**: 适用于关注系统是否能够尽可能多地捕捉到正样本，即推荐结果的覆盖率。

4. **F1 分数 (F1 Score)**
   - **定义**: F1 分数是精确率和召回率的调和平均数，用于平衡精确率和召回率。
   - **公式**:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

   - **适用场景**: 适用于需要同时考虑精确率和召回率的场景，尤其是在正负样本不平衡的情况下。

- **准确率** 适用于类别平衡的数据集，但在推荐系统中可能不常用。
- **精确率** 关注推荐结果的质量。
- **召回率** 关注推荐结果的覆盖率。
- **F1 分数** 是精确率和召回率的平衡指标。

基于检索的评判指标

1. **命中率（Hit Rate）**：推荐列表中是否**包含**用户实际感兴趣的物品。
2. **平均倒数排名（MRR）**：相关物品在推荐列表中的排名的倒数平均值。

   评估系统在返回**第一个正确答案**方面的表现;值越接近 1，表示系统性能越好

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

3. **归一化折损累计增益（NDCG）**：考虑物品排序的相关性评分。

$$\mathrm{NDCG} = \frac{\mathrm{DCG}}{\mathrm{IDCG}}$$

其中，DCG（折损累计增益）和 IDCG（理想折损累计增益）分别表示实际和理想的排序得分。

通过以上整理，可以系统地理解推荐系统的核心概念、方法及其评估方式。